Introduction to Deep Learning and Convolutional Neural Networks

Pablo Mesejo
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA



Recommended Bibliography

- Bishop, C. M., & Bishop, H. (2024). *Deep learning: Foundations and concepts*. Springer Nature.
- Prince, S. J. (2023). *Understanding deep learning*. MIT Press.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning.* Nature, *521*(7553), 436-444.
- Schmidhuber, J. (2015). *Deep learning in neural networks: An overview.* Neural networks, 61, 85-117.
- Bengio, Y.; Courville, A.; Vincent, P. (2013). *Representation Learning: A Review and New Perspectives*. IEEE Transactions on Pattern Analysis and Machine Intelligence. 35 (8): 1798–1828.
- Bengio, Y. (2009). *Learning deep architectures for AI.* Foundations and Trends in Machine Learning, 2(1), 1-127.

Recommended online resources

- “CS231n: Deep Learning for Computer Vision” (Stanford): http://cs231n.stanford.edu/
- “Neural Networks for Machine Learning” (course by Geoffrey Hinton, University of Toronto/Coursera): https://www.youtube.com/watch?v=cbeTc-Urqak
- “Deep Learning Specialization” (course by Andrew Ng, Stanford/deeplearning.ai): https://www.youtube.com/watch?v=CS4cs9xVecg&list=PLkDaE6sCZn6Ec-XTbcX1uRg2_u4xOEky0
- “Dive into Deep Learning. Interactive deep learning book with code, math, and discussions” (A. Zhang, Z.C. Lipton, M. Li, A.J. Smola): https://d2l.ai/
- “CS230 Deep Learning” (Stanford): https://cs230.stanford.edu/lecture/
- Andrej Karpathy’s blog: https://karpathy.github.io/ and https://karpathy.ai/zero-to-hero.html
- Christopher Olah’s blog: https://colah.github.io/ (especially for RNNs)

Recommended online resources

- https://distill.pub/
- “Practical Deep Learning” (Jeremy Howard): https://www.fast.ai/
- “Neural Networks and Deep Learning” (Michael Nielsen): http://neuralnetworksanddeeplearning.com/
- Nice animations from Simon J.D. Prince’s book: https://udlbook.github.io/udlfigures/
- “Neural Networks and Deep Learning” (Jimmy Ba): https://uoft-csc413.github.io/2023/
- Collection of foundational AI and deep learning papers, based on a reportedly recommended reading list by Ilya Sutskever: https://30papers.com/

Index

1 Introduction

Neural networks, hierarchical representations, applications and historical context

2 Convolutional Neural Networks

Fundamental Concepts

Origins, convolution variants, receptive fields, fully-connected and convolutional layers, pooling, activation functions, and building a ConvNet

Training and Regularization

Backpropagation, optimization, learning rates, overfitting, training curves, normalization, and regularization techniques

3 Final Remarks

Training stochasticity, representative ConvNet architectures, and main conclusions

4 Appendix

Additional research insights and hyperparameter considerations

1. Introduction

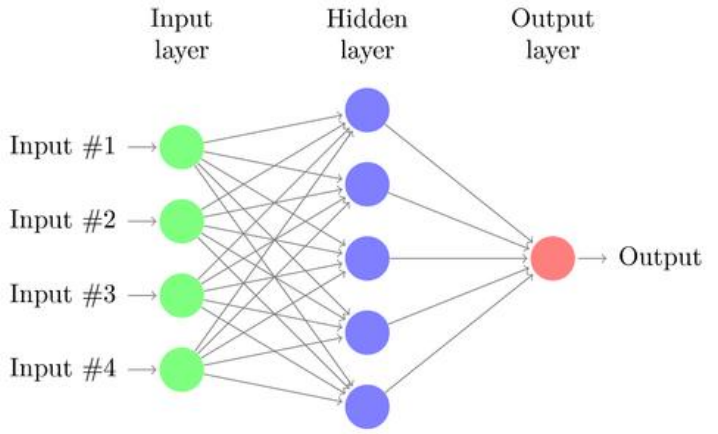
Main AI paradigms

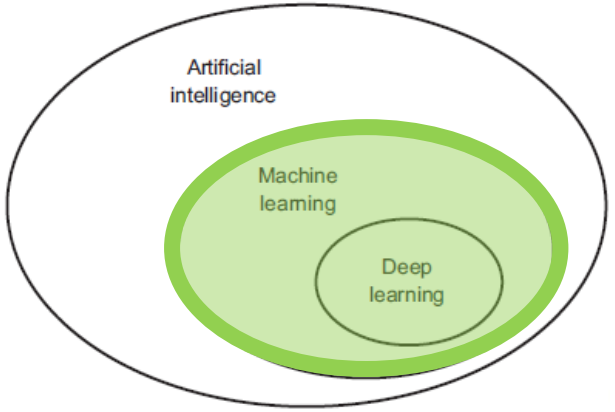
Symbolic/classic or GOFAI (*Good Old-Fashioned Artificial Intelligence*)

```
IF
        CARD_NUMBER= verified        &
        DATE       = not expired     &
        CODES      = correct         &
        ATTEMPTS   = not exceeded    &
        BALANCE    = sufficient      &
        LIMIT      = not exceeded    &
THEN
        PAYMENT    = authorized
```

Example: rule-based systems.
It starts from knowledge (high-level information)

Sub-symbolic/connectionist





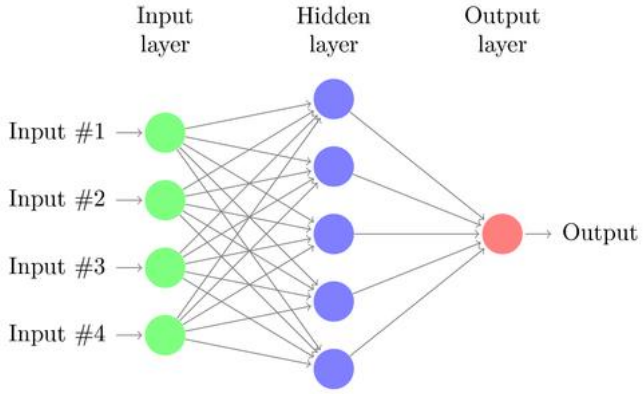
Example: artificial neural networks.
It starts from data (low-level information)

These categories are not mutually exclusive, and learning from data does not necessarily require neural networks. For example, symbolic regression can discover mathematical expressions directly from data using genetic programming (La Cava et al., 2021). **Neuro-symbolic AI** combines neural and symbolic approaches (Garcez & Lamb, 2023), and some methods can learn explicit symbolic representations and rules from data (Daniele et al., 2023).

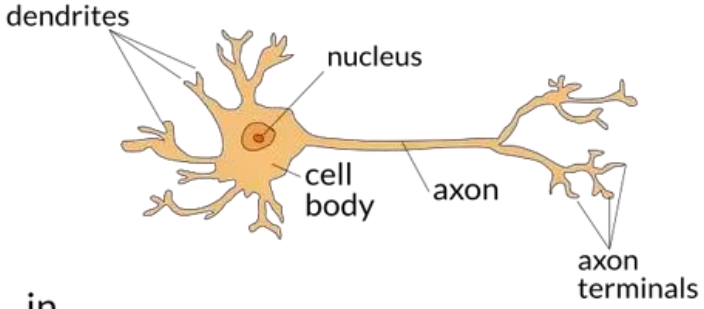
Garcez, A. d'Avila, & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd Wave. Artificial Intelligence Review, 56(11), 12387-12406.
Daniele, A., Campari, T., Malhotra, S., & Serafini, L. (2023). Deep Symbolic Learning: Discovering Symbols and Rules from Perceptions. Proceedings of IJCAI 2023, 3597-3605.
La Cava, W., et al. (2021). Contemporary Symbolic Regression Methods and their Relative Performance. NeurIPS Datasets and Benchmarks Track. 1-16.

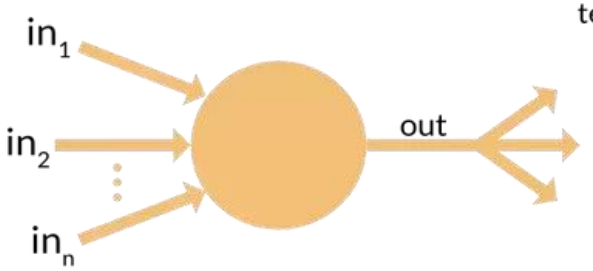
Neural Networks

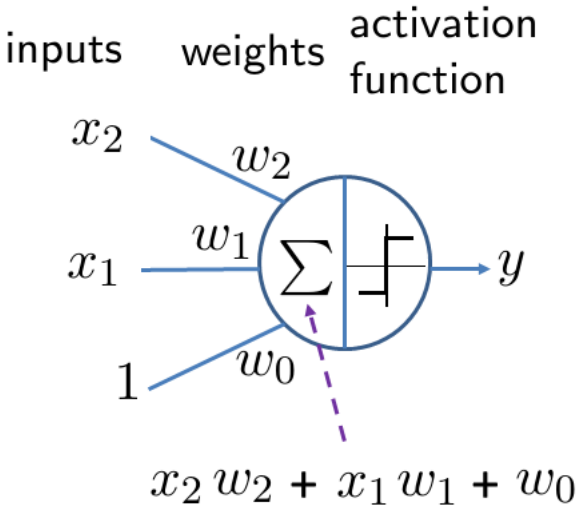
A set of simple processing units that are highly interconnected and, through a learning process, solve a specific task



What is an artificial neuron?

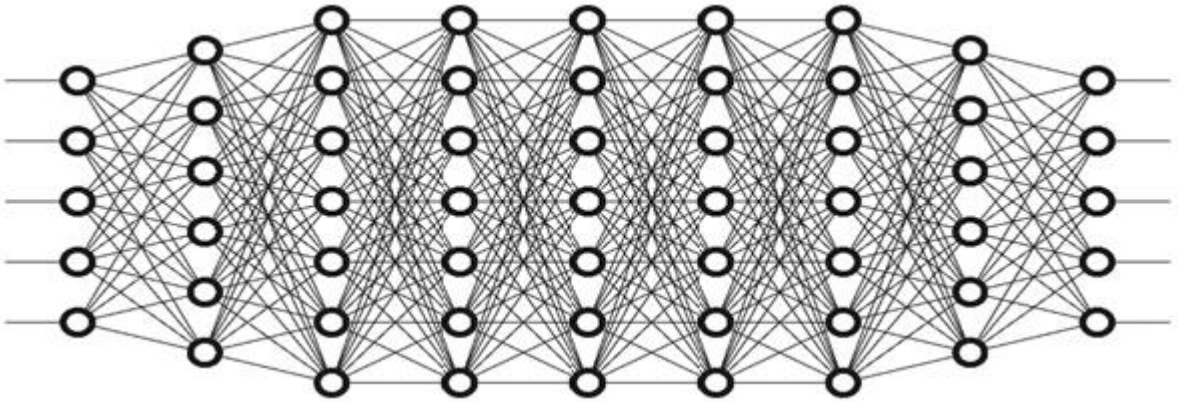




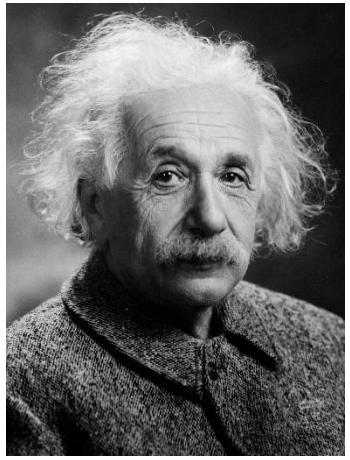


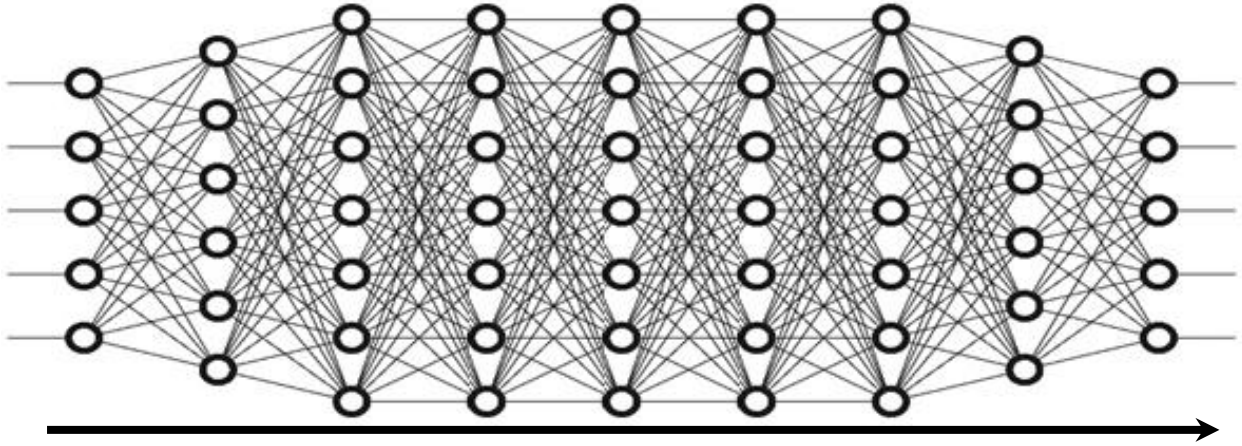
What are Deep Neural Networks?

- Official definition
  - computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction (LeCun et al., 2015)
- In practice:
  - Neural networks with "many" hidden layers
    - And that, as a consequence, can approximate more complex functions (i.e. solve more difficult problems)

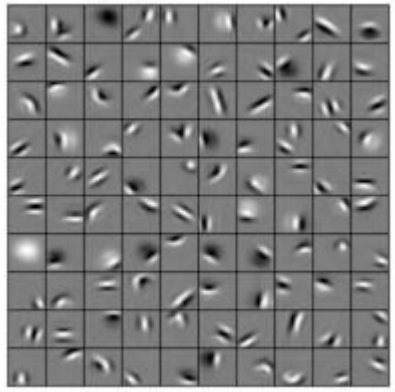


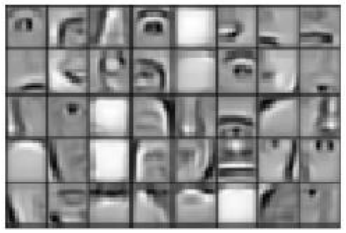
What are Deep Neural Networks?





“Albert Einstein”







Images taken from Lee et al. "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations." *ICML* 2009.

They learn hierarchical representations of data (multiple levels of abstraction).

What are Deep Neural Networks?



François Chollet 
@fchollet

"Neural networks" are a sad misnomer. They're neither neural nor even networks. They're chains of differentiable, parameterized geometric functions, trained with gradient descent (with gradients obtained via the chain rule). A small set of highschool-level ideas put together

Traducir post

8:58 p. m. · 12 ene. 2018

Can we have Deep Learning without neural networks?

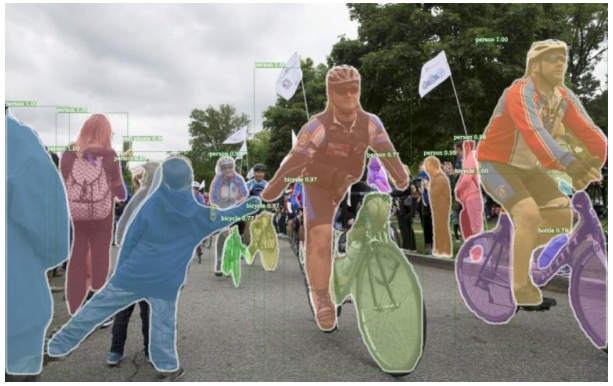
- DL is defined by a hierarchy of learned representations (representation learning)
  - Not by which particular model is used to define these representations.
  - See https://www.deeplearningbook.org/contents/intro.html

- We are effectively learning a series of transformations of our data (typically, using neural nets).

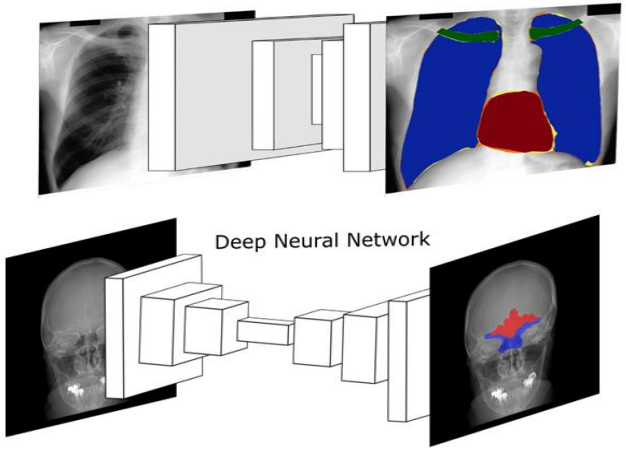
- However, the use of neural nets is not mandatory.
  - See Deep Gaussian Processes (https://proceedings.mlr.press/v31/damianou13a.pdf, https://arxiv.org/abs/1711.00165).
  - You can do DL with non-neural architectures that learn hierarchical representations. PCA, being linear, does not qualify (though nonlinear analogues of PCA, such as deep kernel PCA, can qualify, see https://www.sciencedirect.com/science/article/pii/S089360802300669X).

Applications (1)

Object Recognition



Mask R-CNN output from
https://github.com/facebookresearch/Detectron

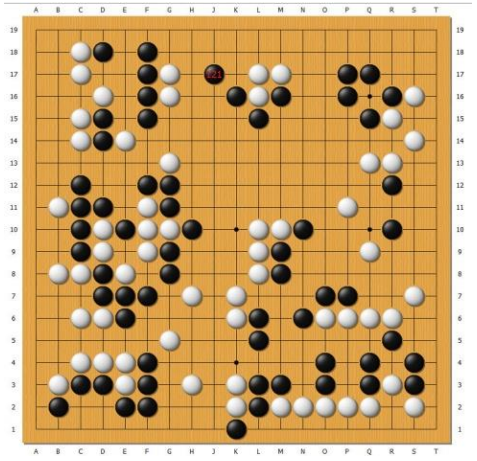




"Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields" (CVPR'17)

Reinforcement Learning in Games

AlphaGo









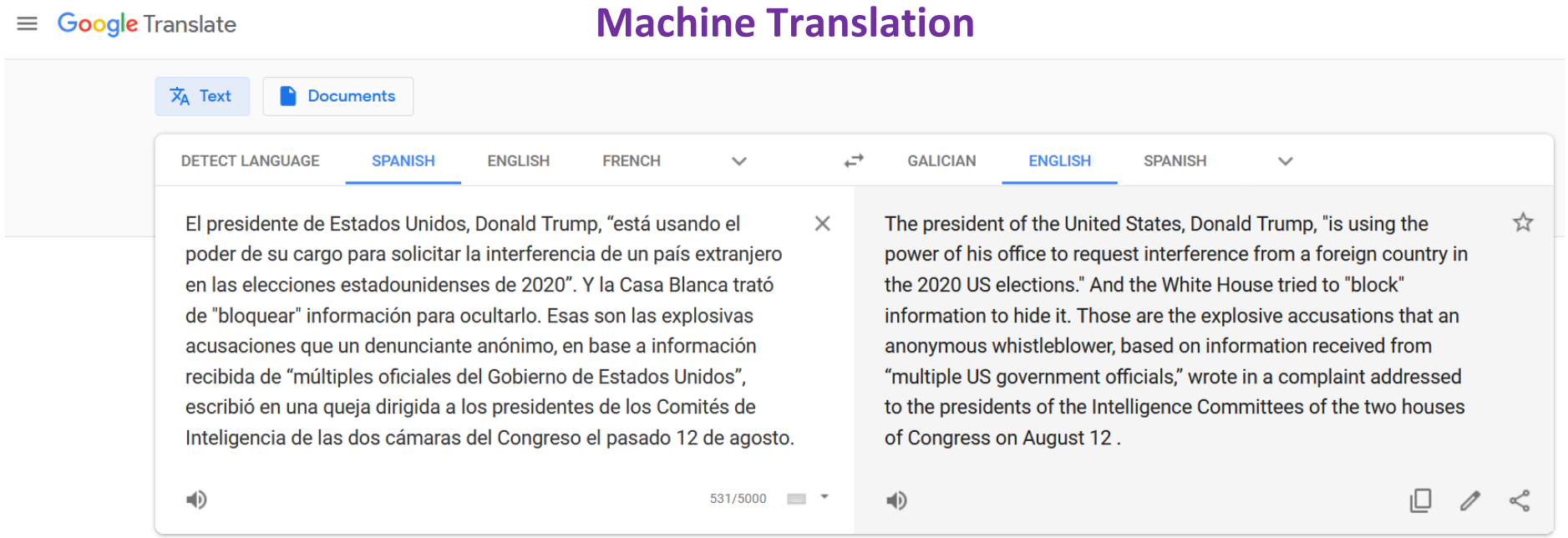
Applications (2)

Automatic Speech Recognition



Image taken from https://www.trzcacak.rs/imgb/iRoJxhx/

Machine Translation



Applications (3): style transfer

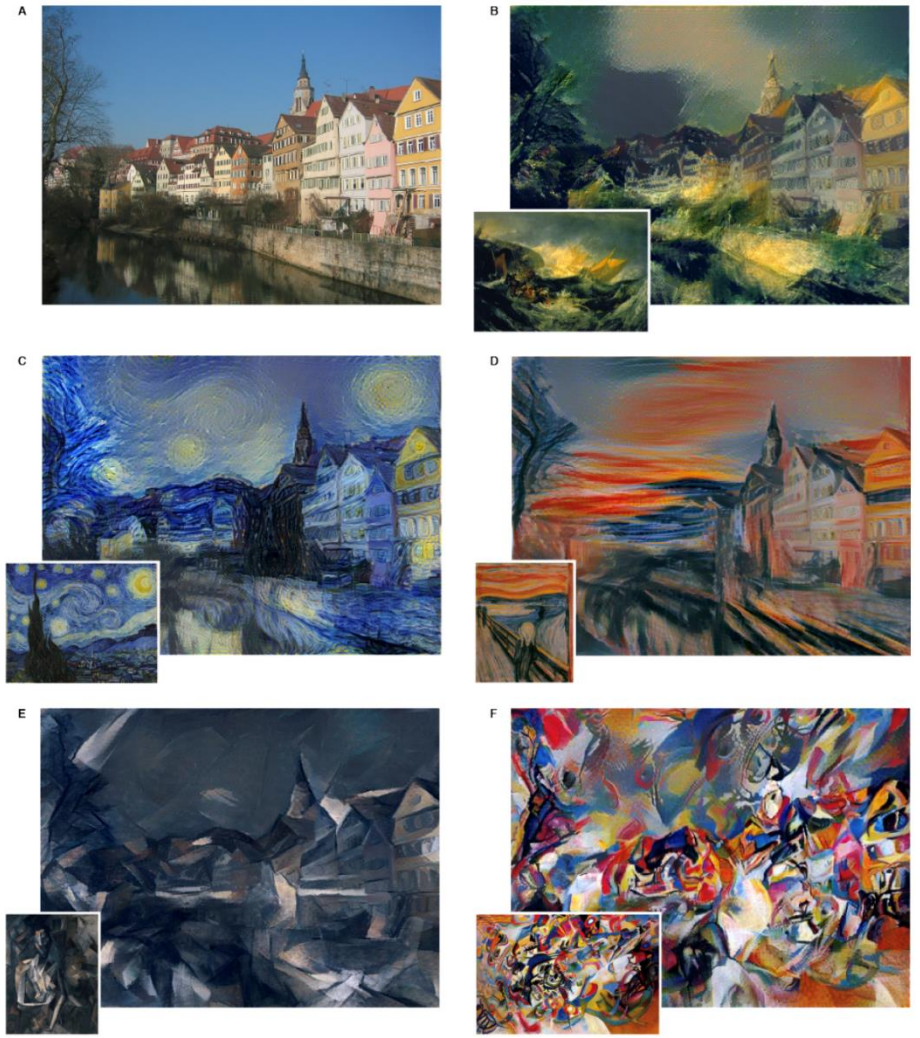


Image taken from “A Neural Algorithm of Artistic Style” (Gatys et al., 2015)

Applications (4): image colorization



Image taken from “Colorful Image Colorization” (Zhang et al., 2016)

Applications (5): image inpainting/reconstruction

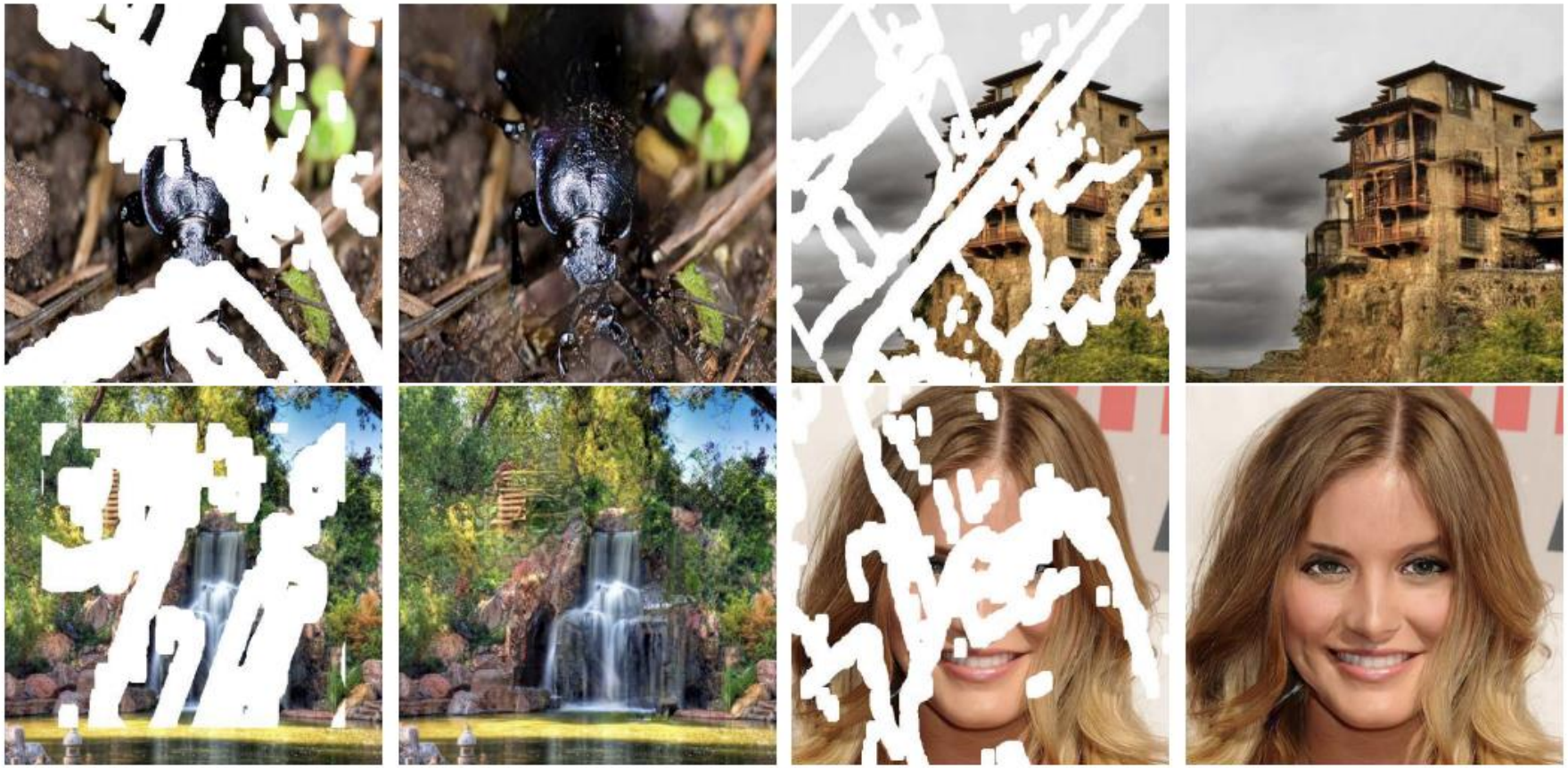


Image taken from “Image Inpainting for Irregular Holes Using Partial Convolutions” (Liu et al., 2018)

Applications (6): image synthesis



Image taken from “Progressive Growing of GANs for Improved Quality, Stability, and Variation” (Karras et al., 2017)

Applications (and 7): image captioning



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



"two young girls are playing with lego toy."



"boy is doing backflip on wakeboard."



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."



"young girl in pink shirt is swinging on swing."

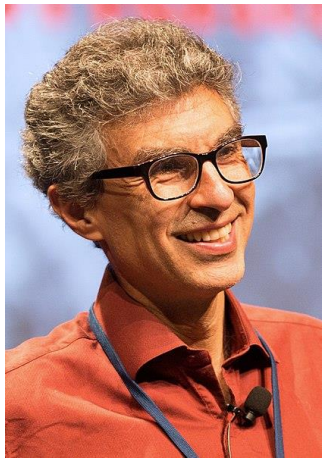


"man in blue wetsuit is surfing on wave."

Karpathy, A., & Fei-Fei, L. (2015). Deep visual-semantic alignments for generating image descriptions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 3128-3137).

Clear success and progress

So evident that three of the field's leading researchers received in 2018 the **Turing Award** ("Nobel Prize in Computer Science")



Yoshua Bengio
University of Montreal

GANs, Neural Machine Translation, among other things…



Geoffrey Hinton
University of Toronto & Google

Popularization of backpropagation, Boltzmann machine, AlexNet, t-SNE, dropout, among other things…



Yann LeCun
New York University & Facebook

ConvNets, Energy-based Models (EBMs), self-supervised learning, gradient-based learning, among other things,…

Clear success and progress

Fundación Princesa de Asturias

#PremiosPrincesadeAsturias

PREMIOS PRINCESA DE ASTURIAS | PREMIADOS

GALARDONADOS CON LOS PREMIOS PRINCESA DE ASTURIAS 2022

Todas las categorías | 2022 | Nombre | BUSCAR

LISTA COMPLETA DE PREMIADOS ›



ARTES
CARMEN LINARES Y MARÍA PAGÉS



CIENCIAS SOCIALES
EDUARDO MATOS MOCTEZUMA



COMUNICACIÓN Y HUMANIDADES
ADAM MICHNIK



CONCORDIA
SHIGERU BAN



COOPERACIÓN INTERNACIONAL
ELLEN MACARTHUR



DEPORTES
FUNDACIÓN Y EQUIPO OLÍMPICO DE REFUGIADOS



INVESTIGACIÓN CIENTÍFICA Y TÉCNICA
GEOFFREY HINTON, YANN LECUN, YOSHUA BENGIO Y DEMIS HASSABIS



LETRAS
JUAN MAYORGA

Clear success and progress

The Nobel Prize in Physics 2024



Ill. Niklas Elmehed © Nobel Prize Outreach
John J. Hopfield
Prize share: 1/2



Ill. Niklas Elmehed © Nobel Prize Outreach
Geoffrey E. Hinton
Prize share: 1/2

The Nobel Prize in Physics 2024 was awarded jointly to John J. Hopfield and Geoffrey E. Hinton "for foundational discoveries and inventions that enable machine learning with artificial neural networks"

Hopfield network

Popularization of backpropagation, Boltzmann machine, AlexNet, t-SNE, dropout

The Nobel Prize in Chemistry 2024



Ill. Niklas Elmehed © Nobel Prize Outreach
David Baker
Prize share: 1/2



Ill. Niklas Elmehed © Nobel Prize Outreach
Demis Hassabis
Prize share: 1/4



Ill. Niklas Elmehed © Nobel Prize Outreach
John M. Jumper
Prize share: 1/4

The Nobel Prize in Chemistry 2024 was divided, one half awarded to David Baker "for computational protein design", the other half jointly to Demis Hassabis and John M. Jumper "for protein structure prediction"

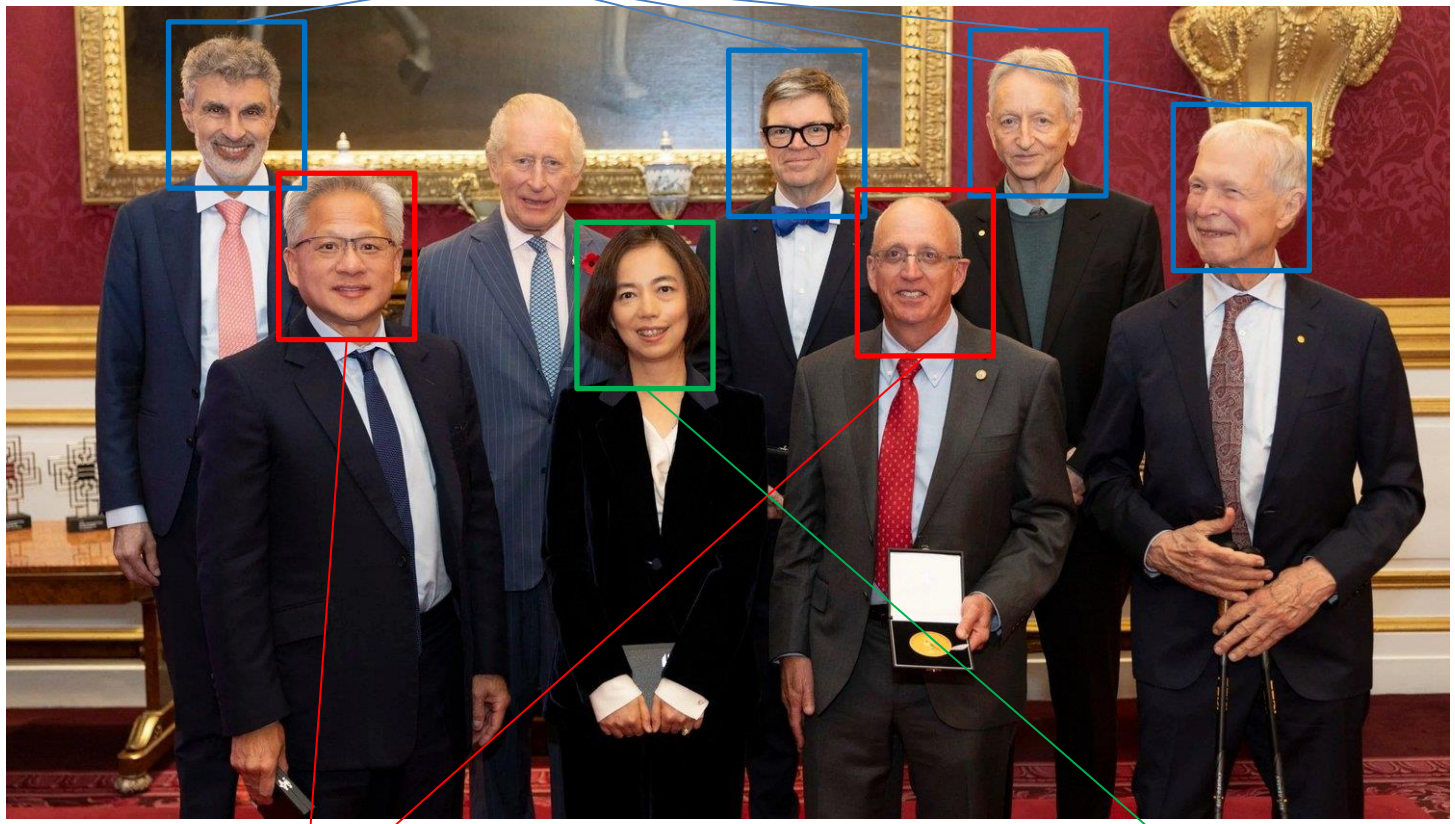
Co-founder and Chair of Google DeepMind (AlphaGo, AlphaFold)

Former AlphaFold lead at Google DeepMind, now at Anthropic

Clear success and progress

Queen Elizabeth Prize for Engineering 2025
(https://qeprize.org/winners/modern-machine-learning)

Yoshua Bengio, Yann LeCun, Geoffrey Hinton, and John Hopfield (artificial neural nets)



Jensen Huang and Bill Dally (hardware)

Fei-Fei Li (high quality datasets)

Clear success and progress

MIT Technology Review

10 Breakthrough Technologies 2013

Deep Learning

With massive amounts of computational power, machines can now recognize objects and translate speech in real time. Artificial intelligence is finally getting smart.

Why now?

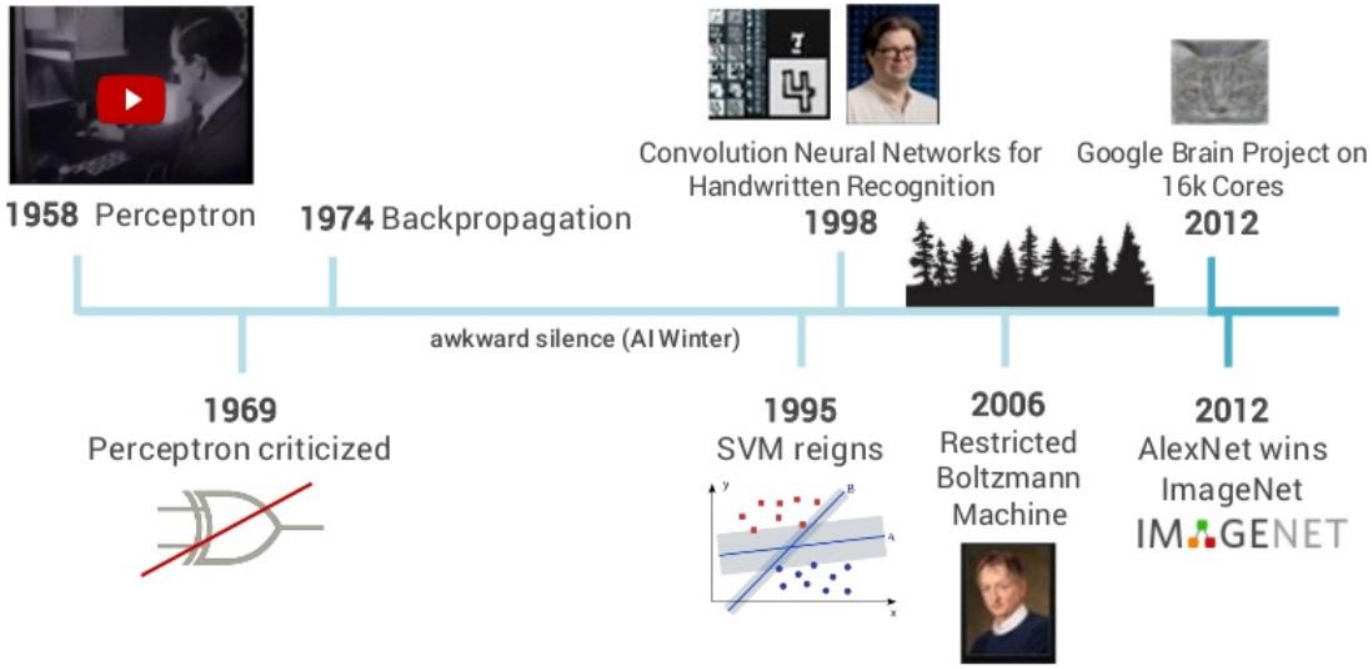


Image taken from https://www.slideshare.net/LuMa921/deep-learning-a-visual-introduction

- The rapid growth of deep learning in the early 2010s (with AlexNet in 2012 as a major milestone) was driven by:
  - Huge amounts of labeled data (e.g. ImageNet)
  - High computational power (e.g. GPU-computing)
  - Algorithmic and architectural advances developed over several years (e.g. ReLU, Dropout, Pretraining)

Some of the main names

- **Geoffrey Hinton** (University of Toronto; formerly Google Brain)
- **Yann LeCun** (NYU / AMI Labs; formerly Meta FAIR)
- **Andrew Ng** (Stanford / DeepLearning.AI; formerly Google Brain & Baidu)
- **Yoshua Bengio** (U. Montréal / Mila / LawZero)
- **Jürgen Schmidhuber** (KAUST / IDSIA-SUPSI)
- **Ruslan Salakhutdinov** (Carnegie Mellon / Sooth Labs; formerly Apple & Meta)
- **Quoc V. Le** (Discovery Loop; formerly Google Brain & Google DeepMind)
- **Léon Bottou** (Flatiron Institute / NYU; formerly Meta FAIR & Microsoft Research)
- **Jeff Dean** (Discovery Loop; formerly Google Brain & Google DeepMind)
- **Max Welling** (U. Amsterdam / CuspAI)
- **Fei-Fei Li** (Stanford / World Labs; formerly Google Cloud AI)
- **Ilya Sutskever** (Safe Superintelligence; formerly OpenAI & Google Brain)
- **Ian Goodfellow** (Co-founder of an undisclosed AI startup; formerly Google DeepMind & Apple)
- **Kaiming He** (MIT / Google DeepMind; formerly Meta FAIR)
- **David Silver** (UCL / Ineffable Intelligence; formerly Google DeepMind)
- **Sepp Hochreiter** (JKU Linz / ELLIS / NXAI)

Some of the main companies























Main companies

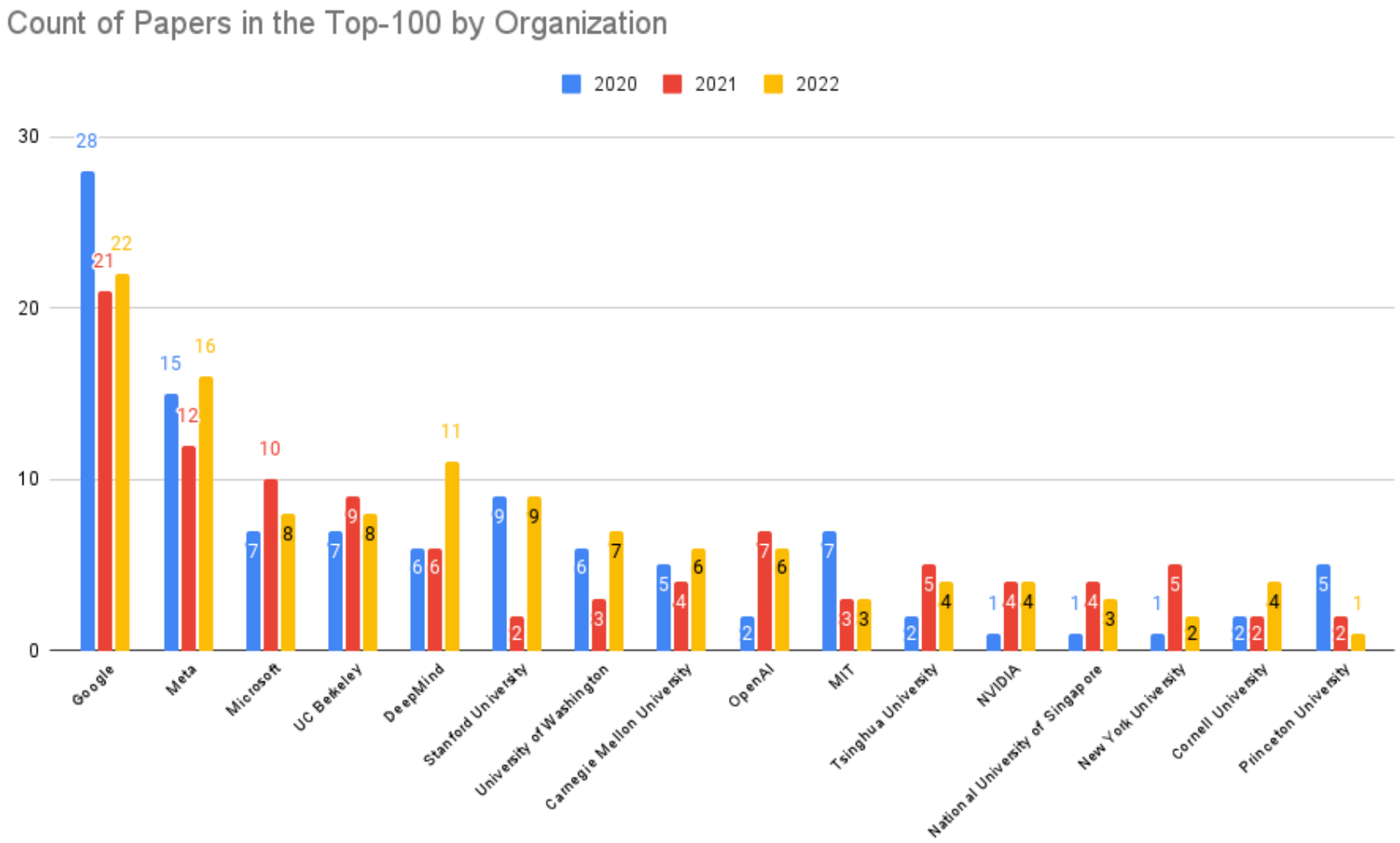


Figure 4 from https://www.zeta-alpha.com/post/must-read-the-100-most-cited-ai-papers-in-2022

Main companies

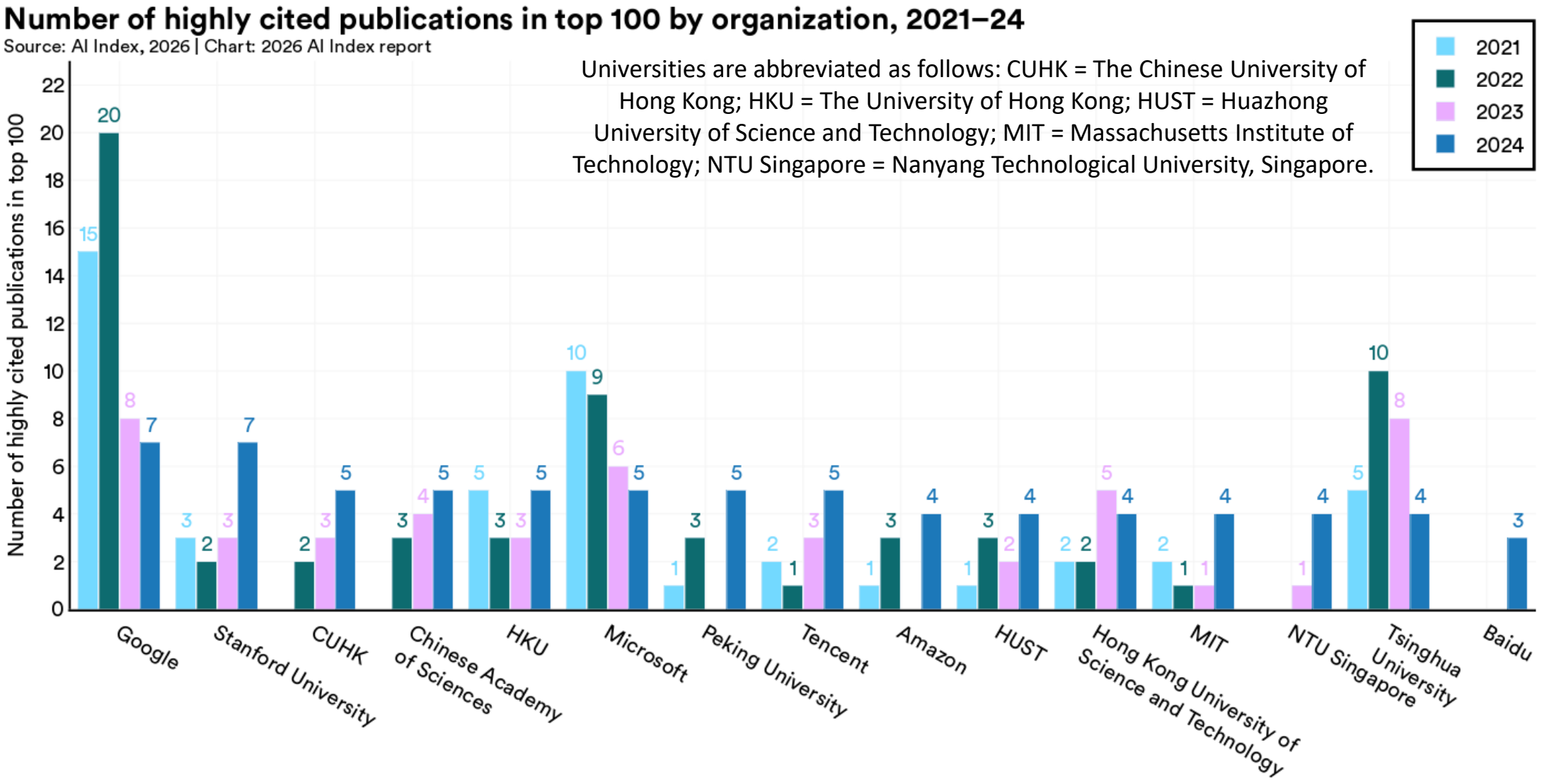


Figure 1.6.13 from https://hai.stanford.edu/assets/files/ai_index_report_2026.pdf

2. Convolutional Neural Networks

Fundamental Concepts

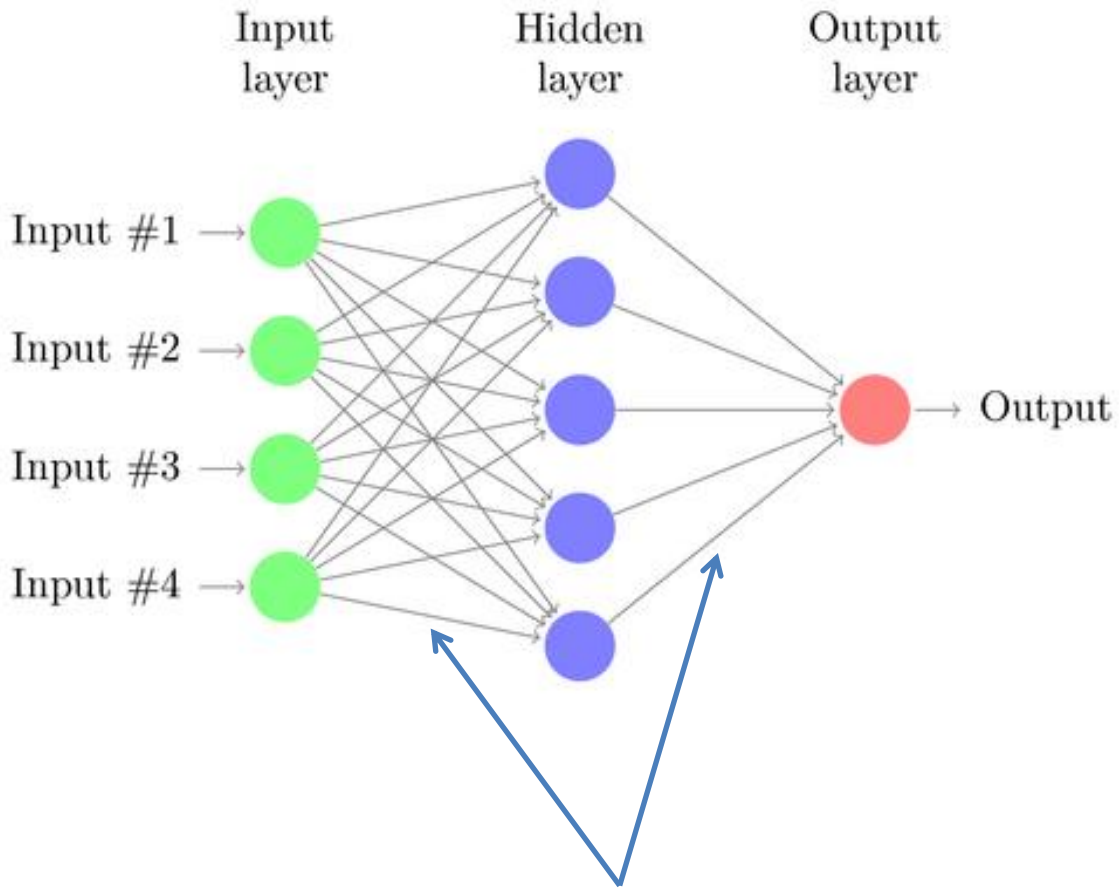
Recommended reading

(part of these slides are taken from this source)

- Highly recommended to consult the Stanford course:
  - CS231n: Convolutional Neural Networks for Visual Recognition (now called Deep Learning for Computer Vision) (https://cs231n.stanford.edu/schedule.html)

- In particular, section on Convolutional Neural Networks (ConvNets): https://cs231n.stanford.edu/slides/2026/lecture_5.pdf https://cs231n.github.io/convolutional-networks/

“Shallow” Neural Networks



Shallow neural networks: “few” layers with weights.
In the example on this slide, two.

There were theoretical reasons for this…

Multi-layer perceptron with a single hidden layer of units

- Networks with two or more layers are universal approximators:
  - Any continuous function can be uniformly approximated to arbitrary accuracy, given enough hidden units.
  - This is true for many definitions of $h(\cdot)$, but excluding polynomials.
  - The key problem is how to find suitable parameter values given a set of training data.



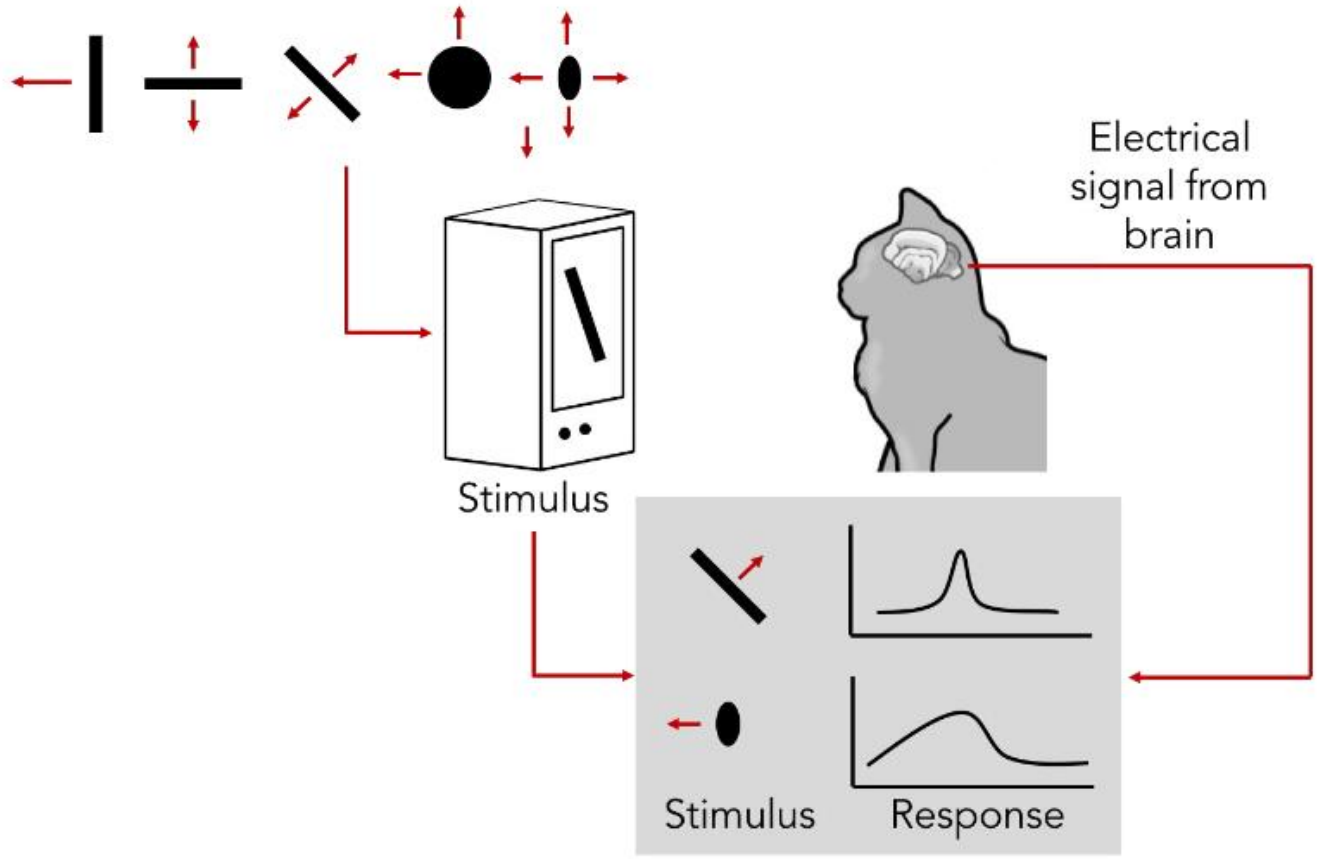
https://project.inria.fr/deeplearning/files/2016/05/session1.pdf

Chapter 5. Bishop, Christopher M. Pattern recognition and machine learning. Springer, 2006.

You can also check this thread by Tivadar Danka about the Universal Approximation Theorem:
https://x.com/TivadarDanka/status/1953501434038956331

Origin

- The origin of convolutional neural networks can be found in the Neocognitron (Fukushima, 1980): a model inspired by the work of Hubel and Wiesel (50s and 60s) on the structure and function of the visual cortex.



How do visual stimuli affect the activation of a neuron?

More information regarding the History of ConvNets:
https://people.idsia.ch/~juergen/who-invented-convolutional-neural-networks.html

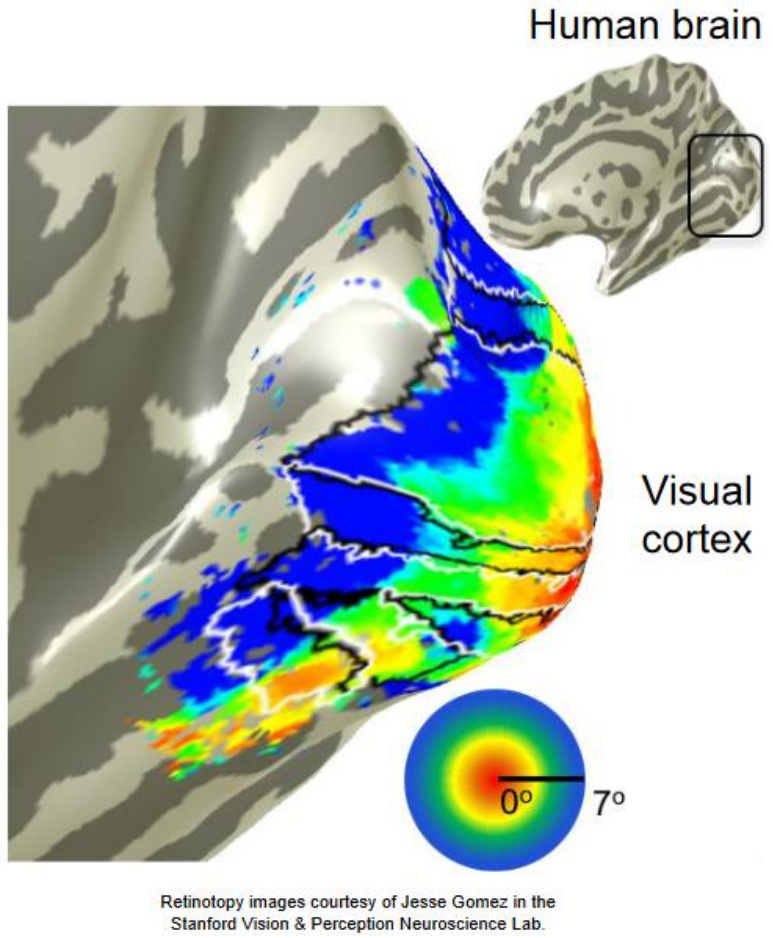
Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung.
http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Origin

- The visual cortex is retinotopically organized:
  - Nearby regions of the visual field tend to be represented in nearby regions of the visual cortex.

- Visual processing exhibits a **partially hierarchical organization**.

- Neurons in the primary visual cortex (V1) exhibit different response properties:
  - **Simple cells**: respond preferentially to edges or bars with a particular orientation and position.
  - **Complex cells**: also respond to oriented edges or bars, but are less sensitive to their exact position.
  - **End-stopping:** property of some neurons whose responses decrease when an oriented stimulus extends beyond a preferred length.

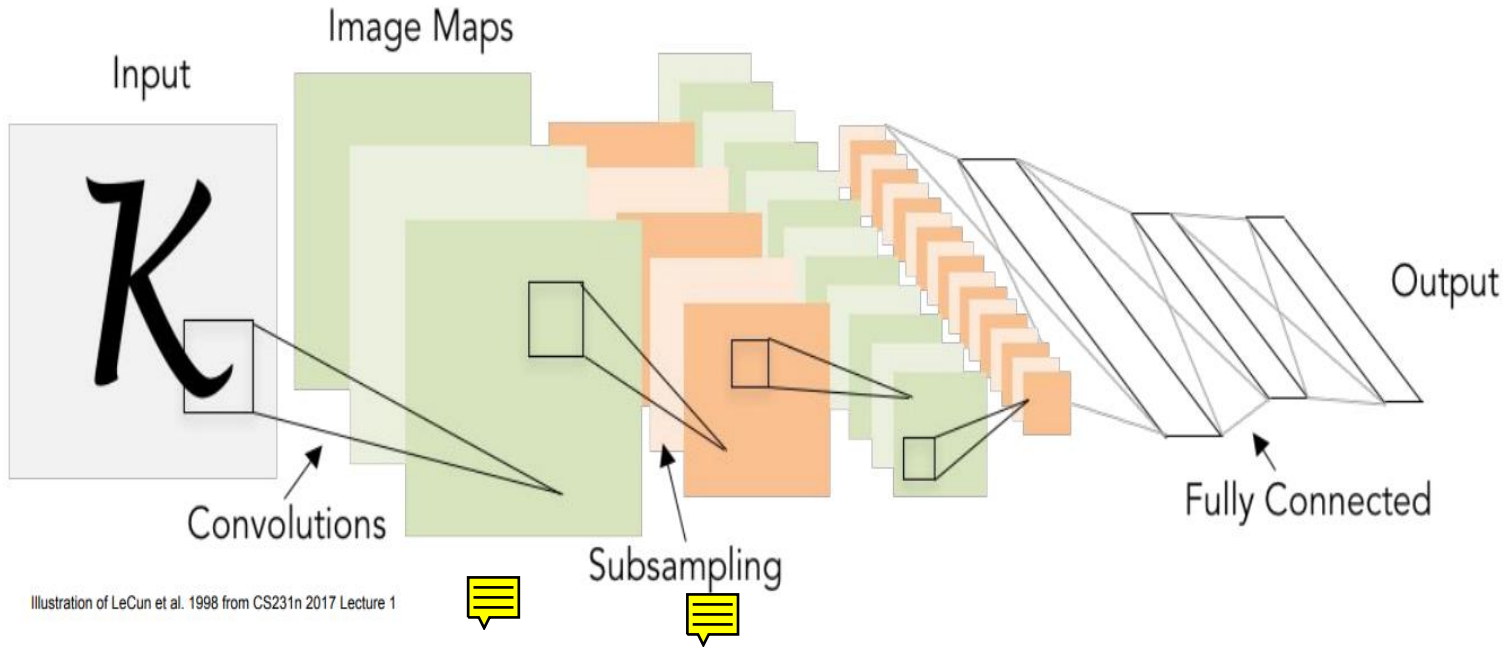


Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.



Definition

- What is a Convolutional Neural Network (ConvNet or CNN)?
  - A neural network with a convolution in, at least, one of its layers.
  - Used in problems where information is presented in matrix/grid format (e.g. images)
- Typical architecture of a ConvNet:



Traditional Image Processing

- **Local linear operation** with a mask.
  - The values of the mask determine the result (features to be extracted)

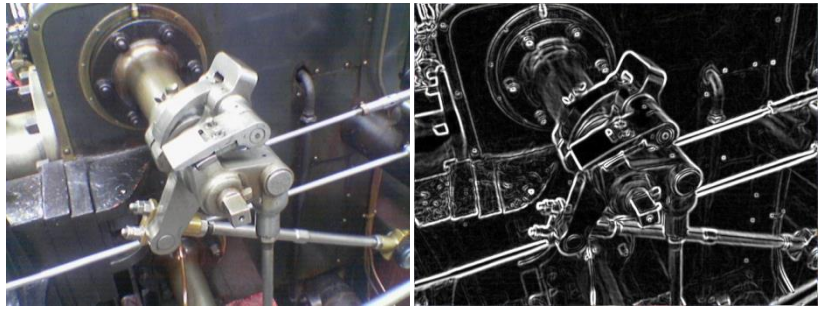
Gaussian filter (attenuates high frequencies → image smoothing)

$$\frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$



Sobel filter (attenuates low frequencies → edge detection)

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



Key idea

In ConvNets, these values are learned! They are not defined by a human expert. They are free network parameters and are trained like any other weight.



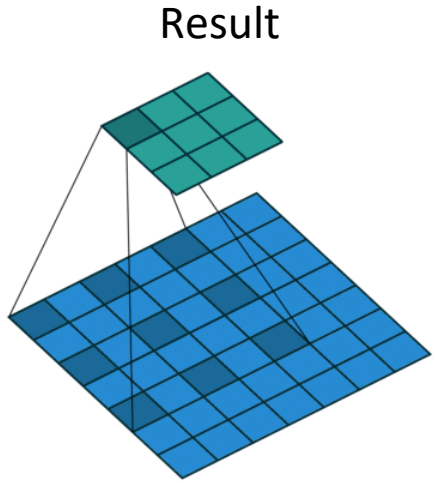
1989: LeCun et al. used **back-propagation to directly learn the coefficients of convolutional filters** from images of handwritten digits.

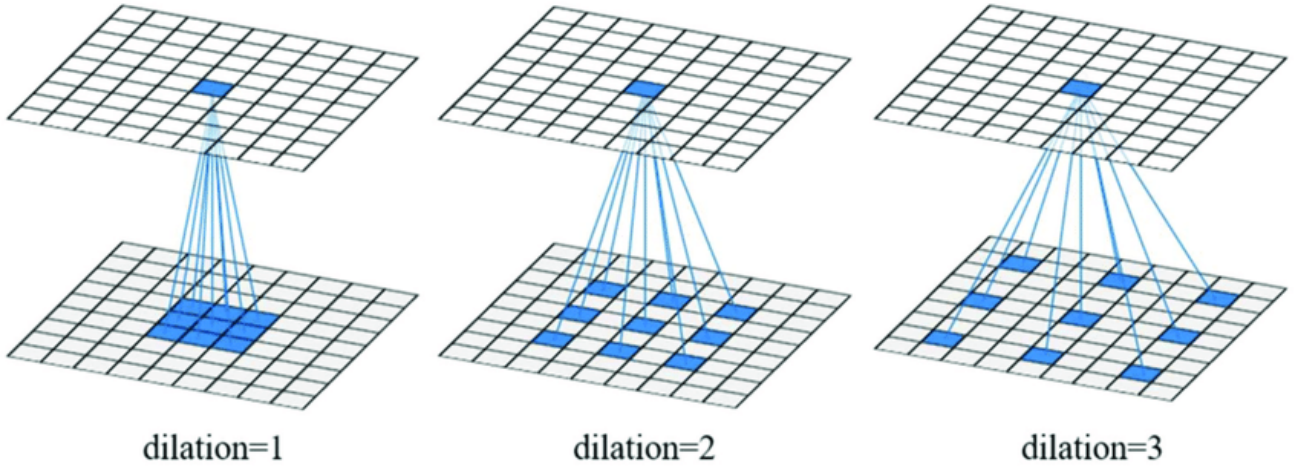
1998: LeCun et al. presented **LeNet-5**, a ConvNet that could automatically recognise handwritten digits in checks, and **demonstrated strong performance on this task**.

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4), 541-551.
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.

Other ways of convolution

- So far we have seen the classical/canonical form of convolution, but there are others. It is worth at least knowing that they exist.
- Dilated convolutions (atrous convolutions).
  - Popular in image segmentation.
  - A *dilation rate*, which defines a spacing between kernel/filter values, is established.
  - The receptive field is widened without increasing the computational cost.

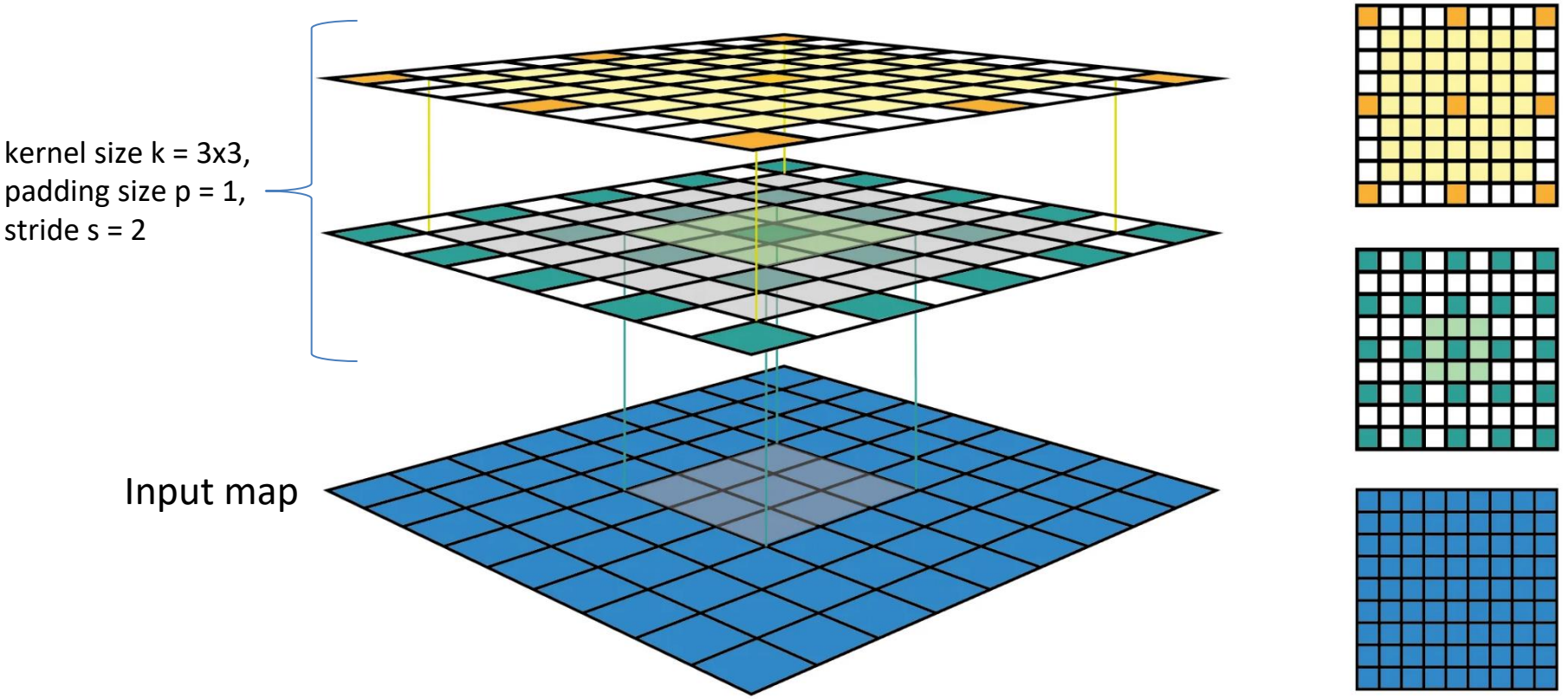




What is the receptive field?

– The region of the input that affects a particular unit of the neural network.

  • In other words, the region in the input space that a particular ConvNet feature is looking at.

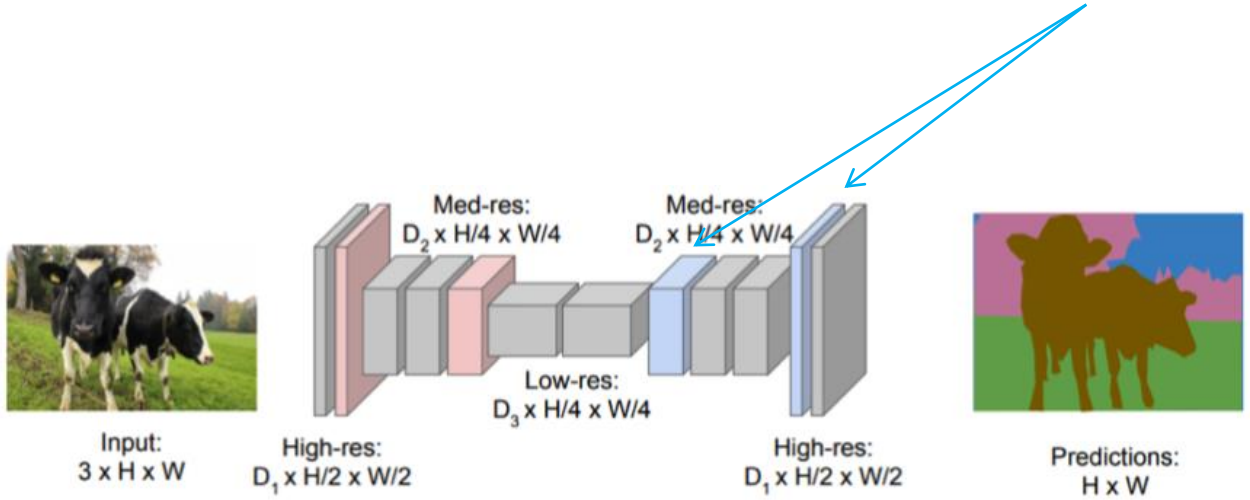


"large receptive fields are necessary for high-level recognition tasks"
https://distill.pub/2019/computing-receptive-fields/

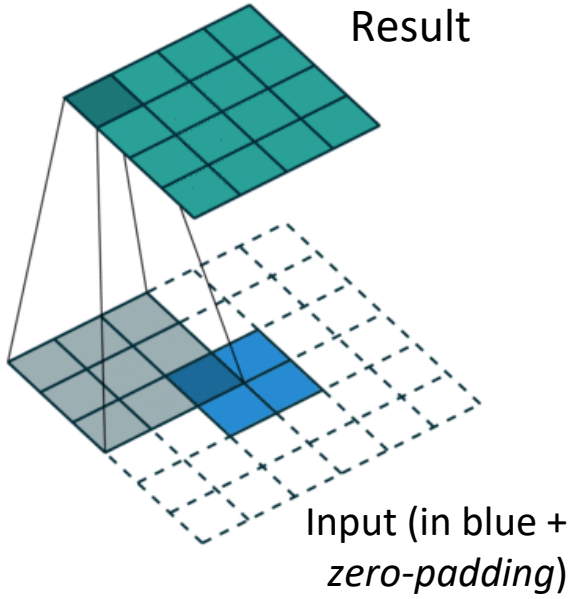
https://blog.mlreview.com/a-guide-to-receptive-field-arithmetic-for-convolutional-neural-networks-e0f514068807

Other ways of convolution

- Transposed convolutions (sometimes, erroneously referred to as *deconvolution*)
  - Popular in super-resolution and segmentation (in the *up-sampling* part) of images.



https://towardsdatascience.com/transposed-convolution-demystified-84ca81b4baba

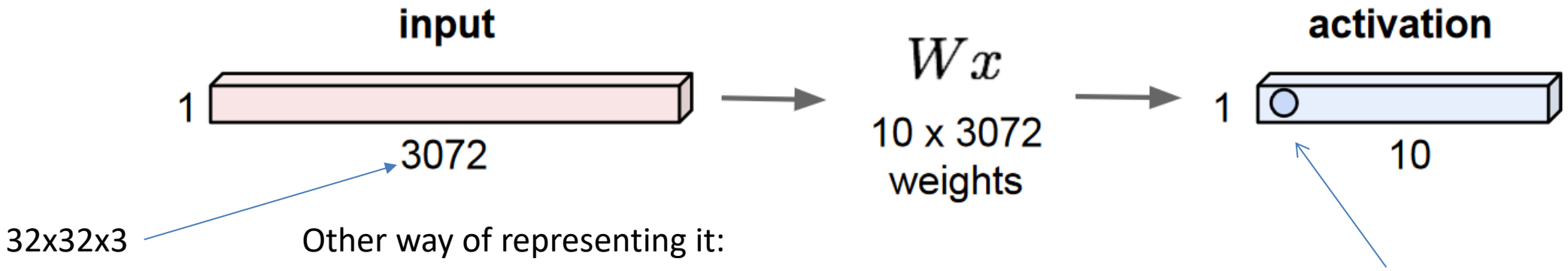


Interesting reading: Vincent Dumoulin, Francesco Visin - A guide to convolution arithmetic for deep learning

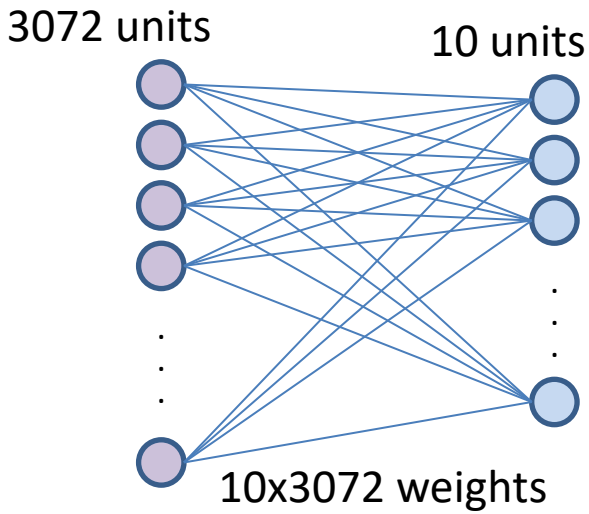
Here more visualizations: https://github.com/vdumoulin/conv_arithmetic

Fully-connected Layer

All units/neurons of a layer are connected (weights) with all units/neurons in the next layer.



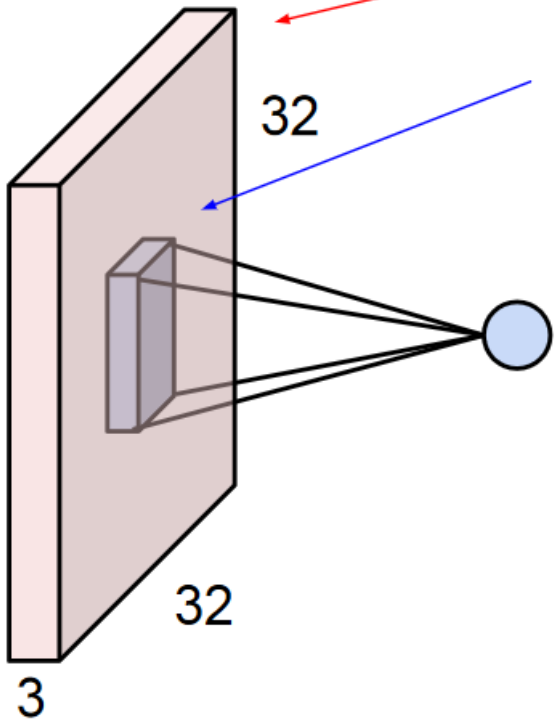
Other way of representing it:



1 number: Each output value is computed as the dot product between one row of W (3072 weights) and the input vector (3072 values), plus an optional bias.

If we include the bias, we would have 30730 parameters.

Convolutional Layer



32x32x3 image

5x5x3 filter w

The 3072 elements we saw before!

1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

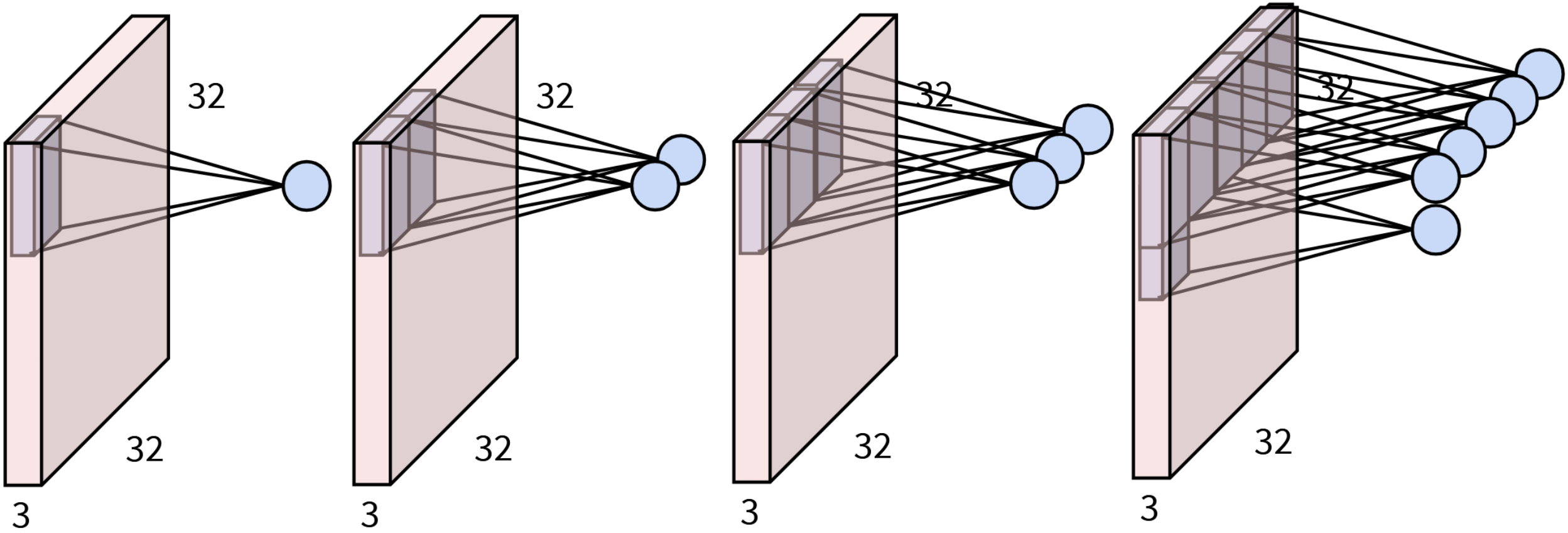
$$w^T x + b$$

Great advantage:
76 vs 30730 parameters!

Note: *bias* brings flexibility to learning (richer representations of the input space), allowing to translate/adapt the output of the unit/neuron. Simple and intuitive example: https://stackoverflow.com/questions/2480650/what-is-the-role-of-the-bias-in-neural-networks

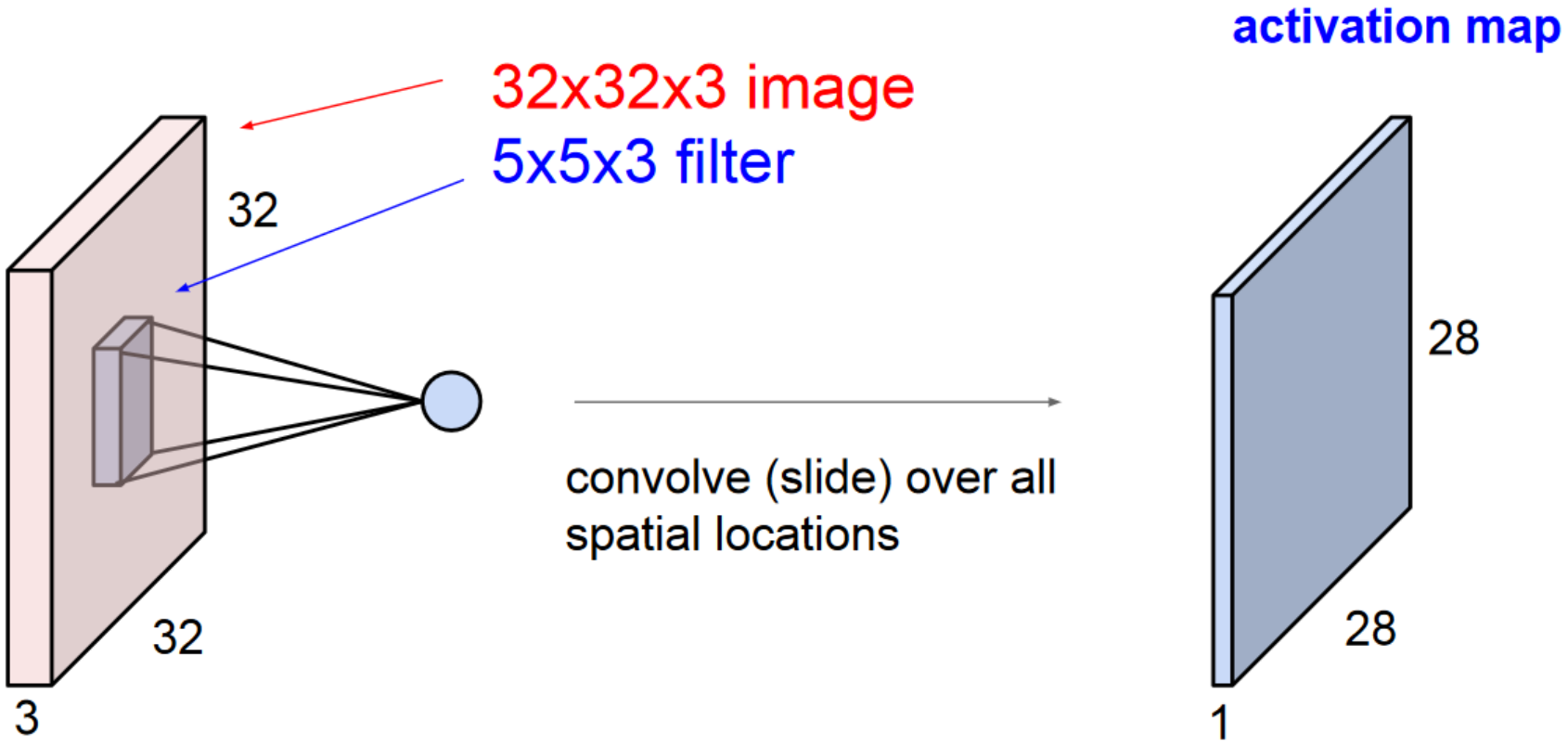
Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung. http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Layer



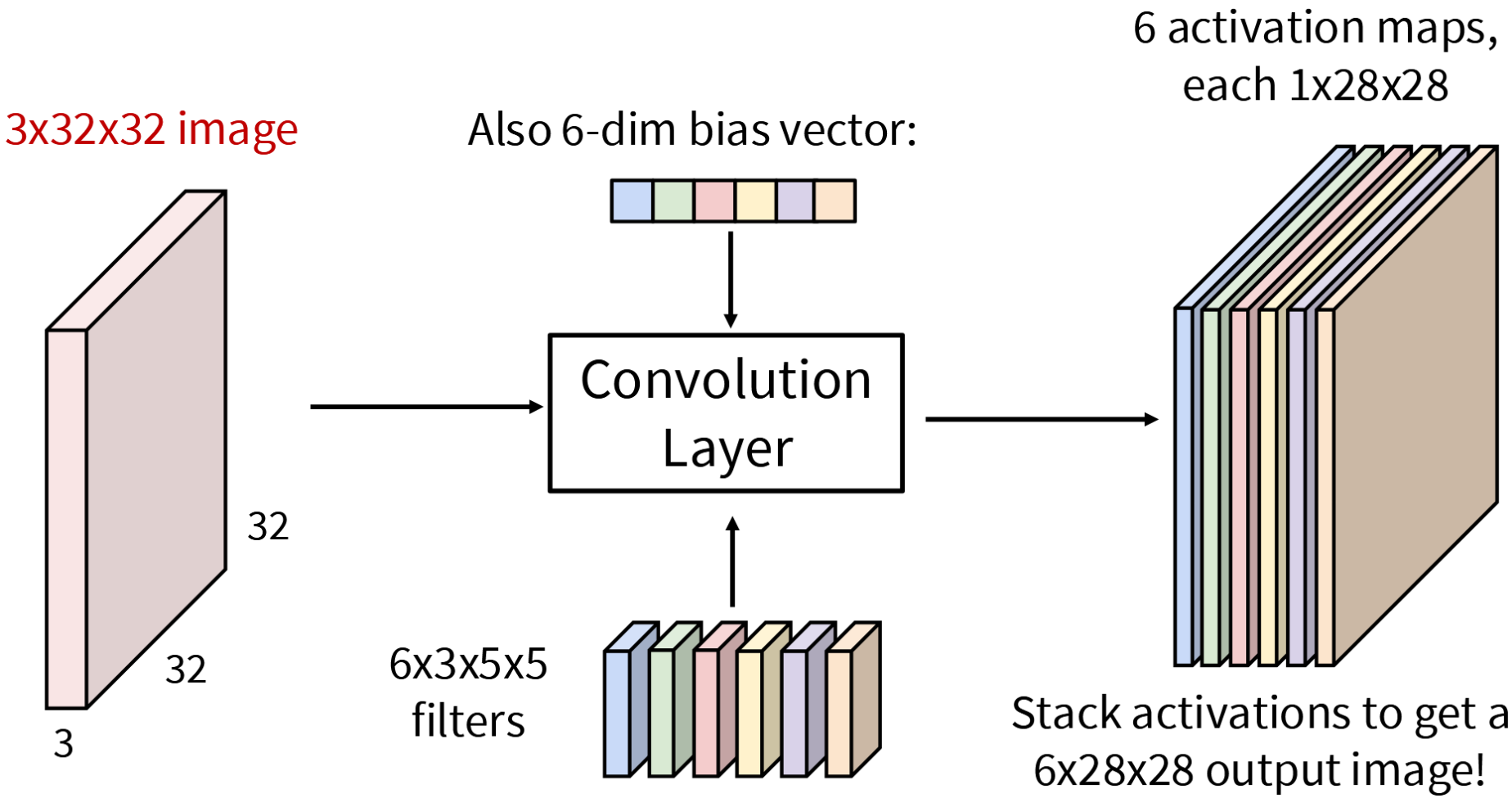
Slide credit: Justin Johnson. https://cs231n.stanford.edu/slides/2025/lecture_5.pdf

Convolutional Layer



Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung. http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Layer

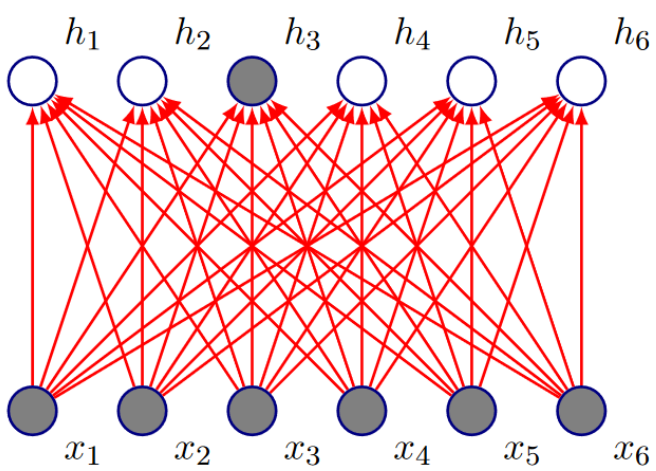


Even now the number of weights is much lower than what we had before: 456 vs 30730 parameters!

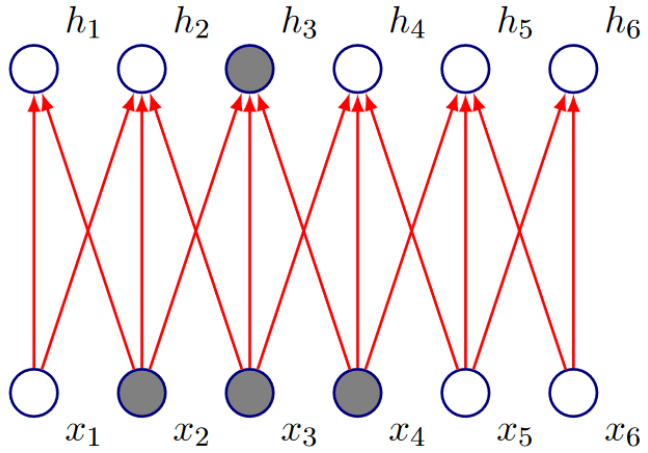
Slide credit: Justin Johnson. https://cs231n.stanford.edu/slides/2025/lecture_5.pdf

Advantages of ConvNets

- **Sparse connectivity**
  - *Fully-connected layers* operate globally (each neuron sees the complete input), while *convolutional layers* operate locally → Fewer operations!



vs



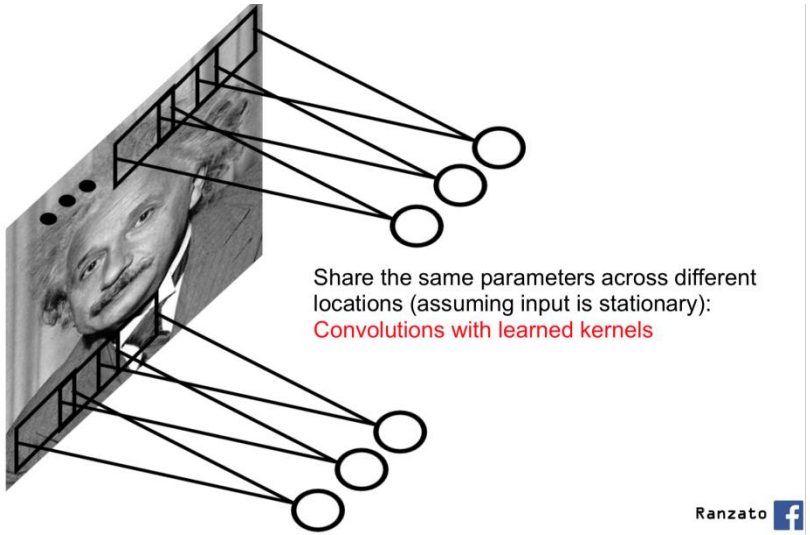
- Fully connected network: h_3 is computed by full matrix multiplication with no sparse connectivity

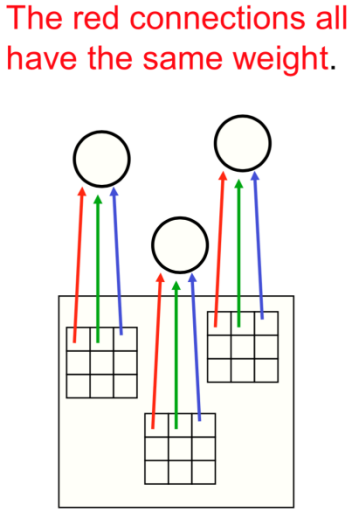
- Kernel of size 3, moved with stride of 1
- h_3 only depends on x_2, x_3, x_4

https://ttic.uchicago.edu/~shubhendu/Pages/Files/Lecture7_pauses.pdf

Advantages of ConvNets

- **Weight/Parameter Sharing**
  - The same filter is applied to the whole image.
    - Instead of learning a parameter for each localization in the image, only a reduced set of parameters (those corresponding to the filter) are learned.
    - The number of parameters to learn and store is largely reduced! → Regularization!





https://www.cs.toronto.edu/~lczhang/aps360_20191/lec/w03/convnet.html

Advantages of ConvNets

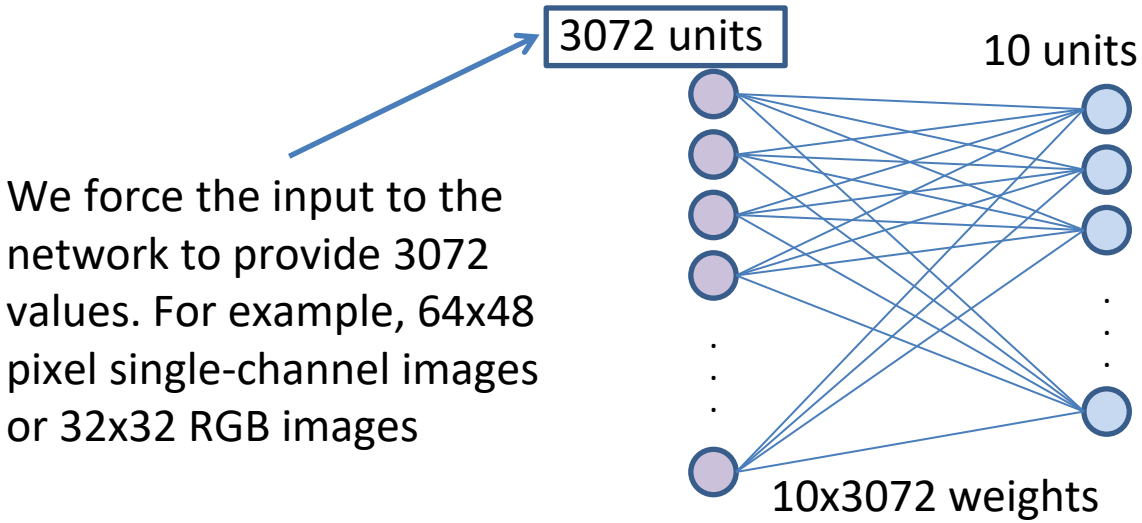
- **Equivariant representations**
  - Standard stride-1 convolutions are translation-equivariant, except potentially at image boundaries (depending on the padding used).

> *f* is equivariant with respect to *g* if $f(g(\boldsymbol{x})) = g(f(\boldsymbol{x}))$

  - If we translate an object in the image, its representation will be translated the same distance in the output.
    - It allows to generalize the detection of edges, textures and shapes at different locations in the image!
  - Convolution is not equivariant with respect to scale or rotation.

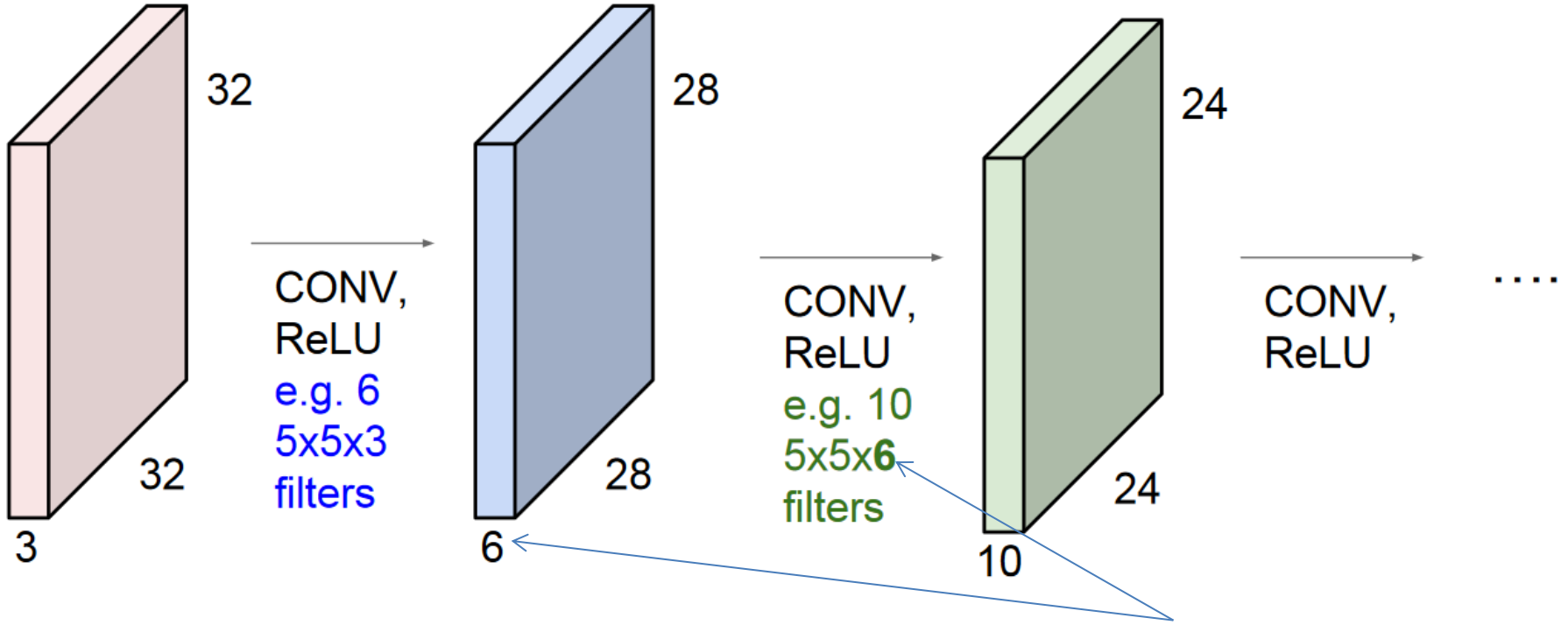
Advantages of ConvNets

- Ability to **operate on variable size inputs**
  - A *fully-connected layer* forces the input to have a certain size.



Convolutional Networks

- A ConvNet is a sequence of Convolutional Layers, interspersed with activation functions.

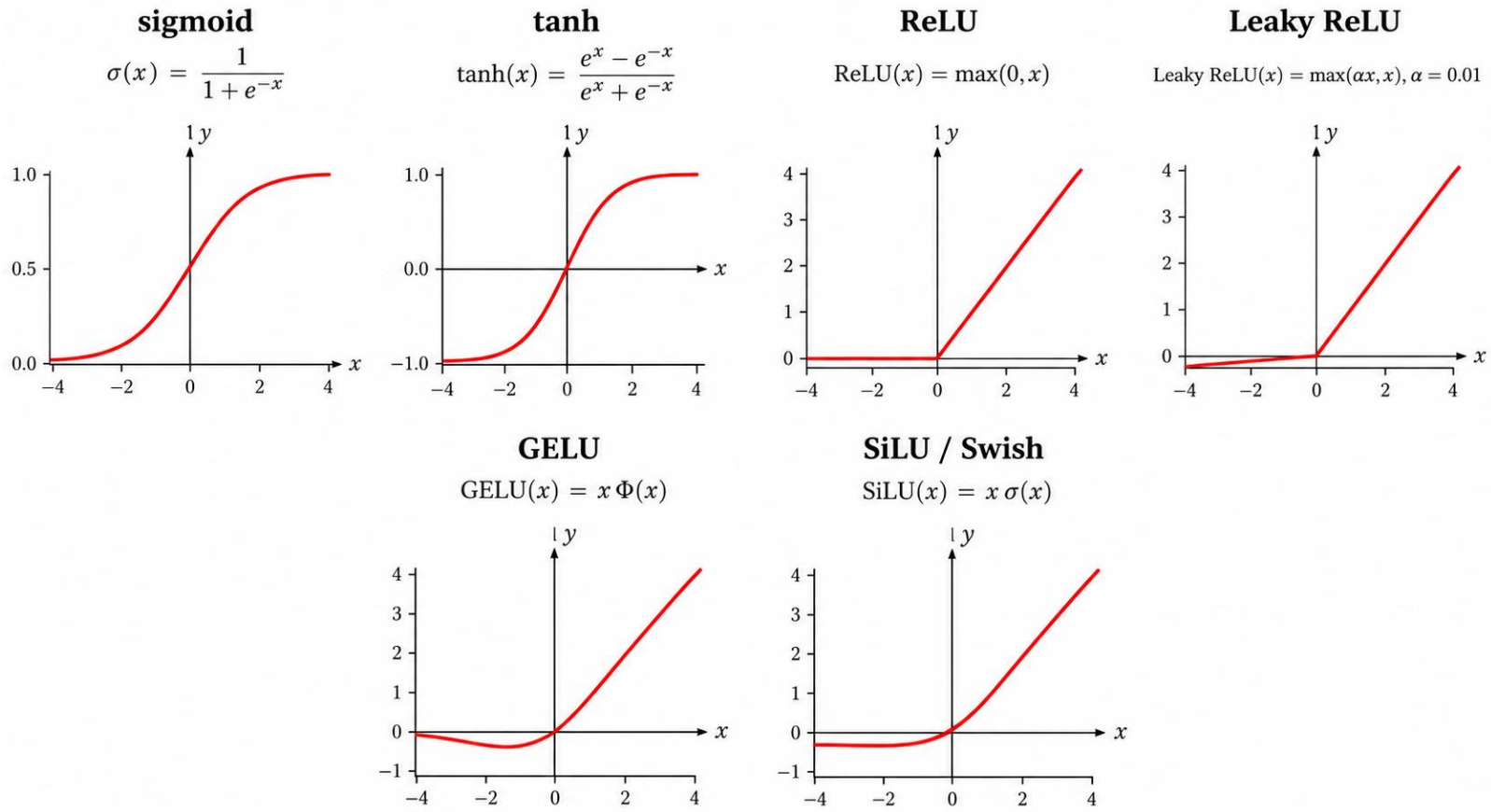


Notice that the filters, in conventional convolutions, always have the same depth as the previous convolutional block, i.e. they extend the full depth of the input volume/tensor.

http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Networks

- Convolutional layers are interspersed with **(nonlinear)** activation functions.
  - Traditionally, the activation functions were sigmoids, such as the *logistic sigmoid* or the hyperbolic tangent.
  - In deep networks it is generally recommended to use the rectified linear unit (ReLU), and its variants, among other reasons because it allows a faster training.



See dying ReLU problem and vanishing gradient problem ("Yes, you should understand backprop" by Andrej Karpathy)

Convolutional Networks

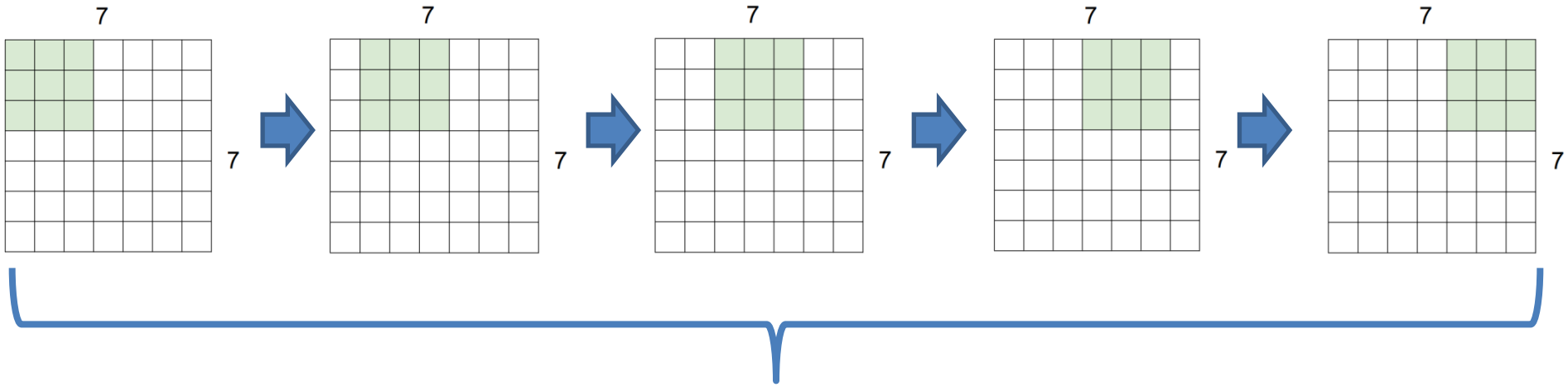
- Convolutional layers are interspersed with **(nonlinear)** activation functions.
  - This non-linearity is necessary to learn complex representations of the input data.
  - Otherwise, if only linear activation functions were used, the neural network would behave like a linear function.

If the activation functions of all the hidden units in a network are taken to be linear, then for any such network we can always find an equivalent network without hidden units.

"Pattern Recognition and Machine Learning" (C. M. Bishop, 2006) p.229

Convolutional Networks

- Let's pay attention now to the dimensions
  - Example: 7x7 image and 3x3 filter

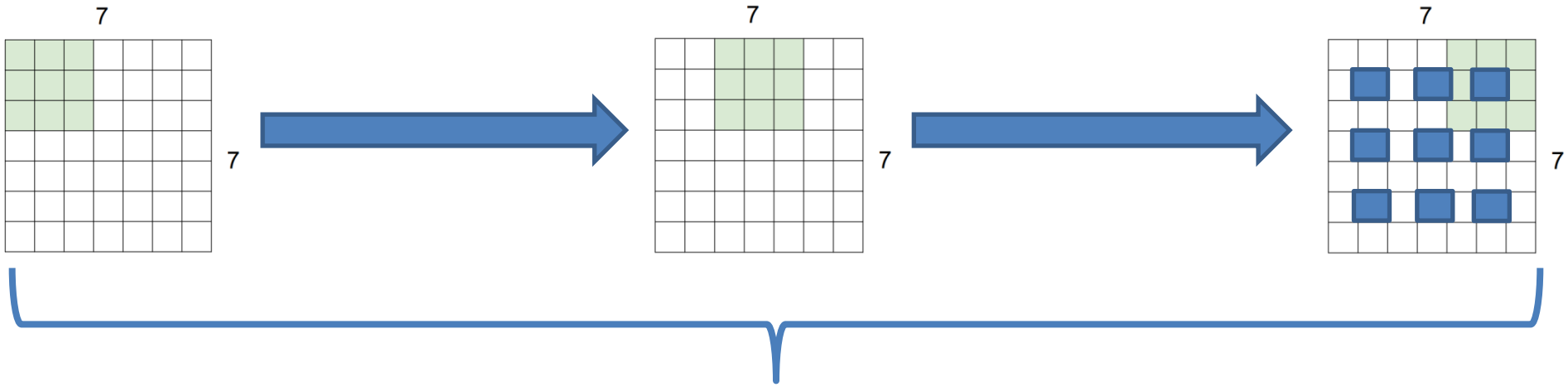


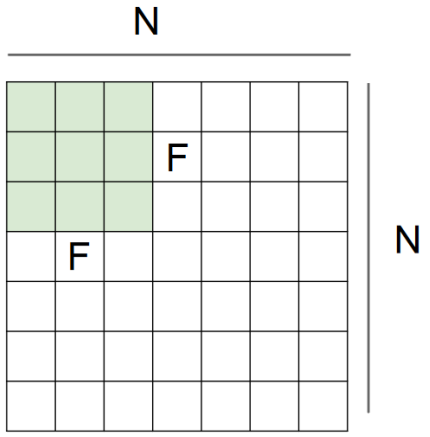
The output is a 5x5 image
(notice that we do not "replace" the edges)

- Two important concepts: stride and padding.

Convolutional Networks

- Let's pay attention now to the dimensions
  - Example: 7x7 image and 3x3 filter, with stride 2





$$\text{Output size} = \left\lfloor \frac{N + 2P - F}{s} \right\rfloor + 1$$

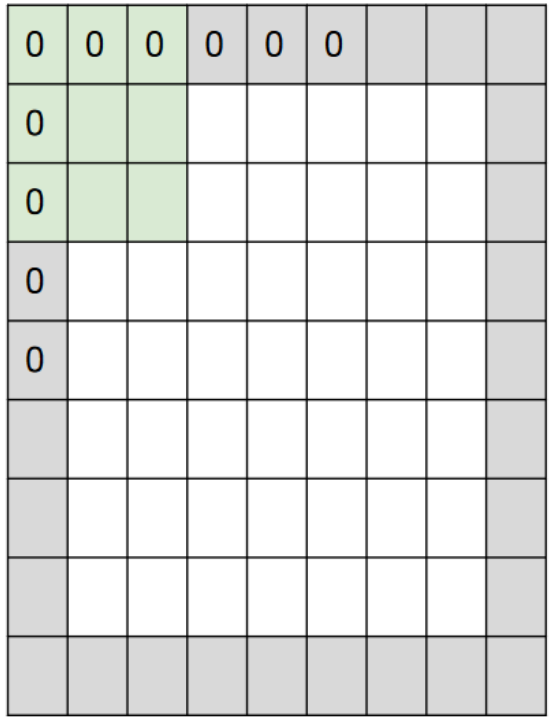
e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33

With stride 3, some input positions are not covered. **Standard implementations such as PyTorch still allow this operation, producing a 2×2 output.**

http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Networks

- We progressively reduce the size of the maps/volumes.
  - If we want to keep the size → padding
  - Example: *zero-padding*



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

In many cases it is good to use *padding*, as empirical evidence advises not to reduce dimensionality too quickly!

http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Networks

- Complete example:
  - Input volume: **16**x**16**x**3**
  - Filters used: **10** filters of **3**x**3** with stride **1** and padding **1**

Output_size = ((**N**+2*__P__-**F**)/**stride**) +1

  - Output volume?
    - (**16**+2*__1__-**3**)/**1** + 1 = 16 → 16x16x**10**

  - Number of parameters in that layer?
    - Each filter has **3**x**3**x**3** + 1 = 28 parameters

      → 28x**10** = 280 parameters

+1 for the *bias*

Convolutional Networks

- (Another) complete example:
  - Input volume: **32**x**32**x**3**
  - Filters used: **10** filters of **5**x**5** with stride **1** and padding **0**

Output_size = ((**N**+2*__P__-**F**)/**stride**) +1

  - Output volume?
    - (**32**+2*__0__-**5**)/**1** + 1 = 28 → 28x28x**10**

  - Number of parameters in that layer?
    - Each filter has **5**x**5**x**3** + 1 = 76 parameters

      → 76x**10** = 760 parameters

+1 for the *bias*

Convolutional Networks

- (Another) complete example (x2):
  - Input volume: **32**x**32**x**3**
  - Filters used: **10** filters of **5**x**5** with stride **1** and padding **2**
  - Output volume?
    - (**32**+2*__2__-**5**)/**1** + 1 = 32 → 32x32x**10**
  - Number of parameters in that layer?
    - Each filter has **5**x**5**x**3** + 1 = 76 parameters

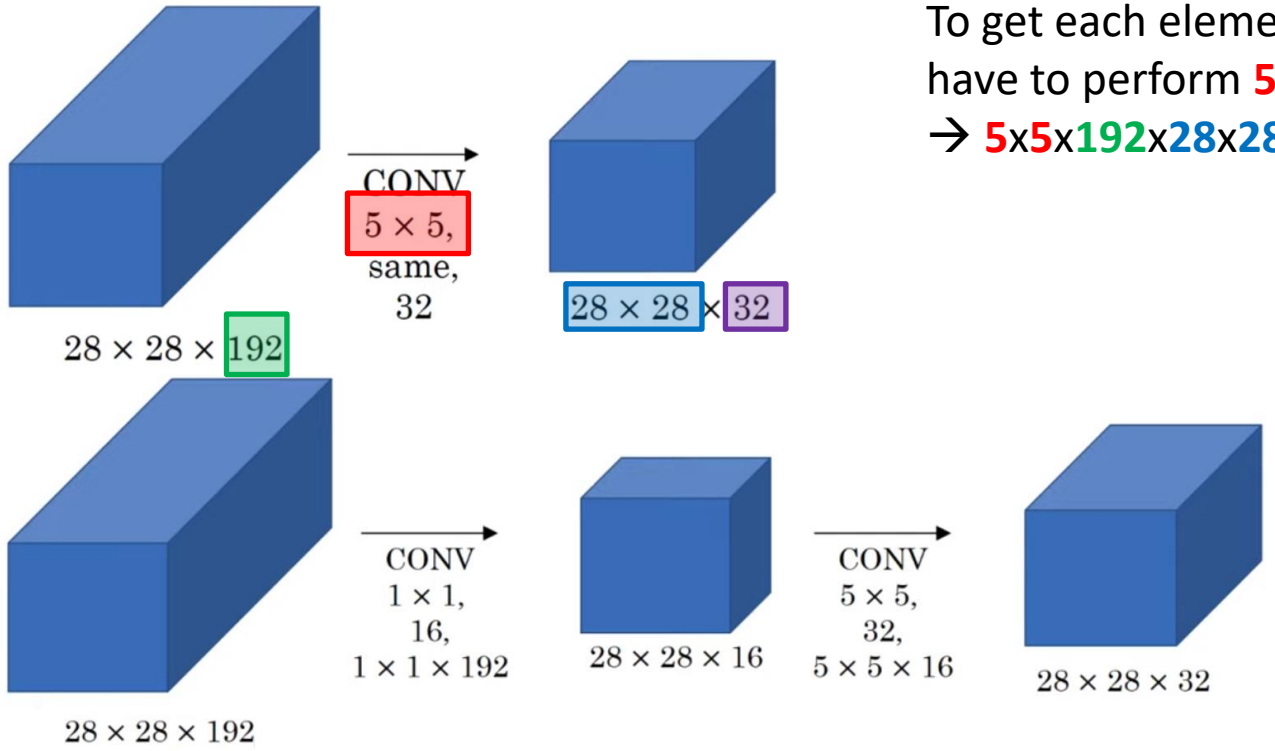
      → 76x**10** = 760 parameters
  - Number of multiplications?
    - Each output is the inner product of two 3x5x5 tensors (75 elems)
    - We have 32x32x10 = 10240 outputs

    → 75x10240=768000 (sums or mults) → FLOPs = 1536000

The bias is just added at the end of every dot product

Convolutional Networks

- **Convolutions 1x1**
  - They are used to linearly combine information from different channels and reduce (or increase) the number of filters (*feature maps*).
  - It can reduce dimensionality and computational cost.



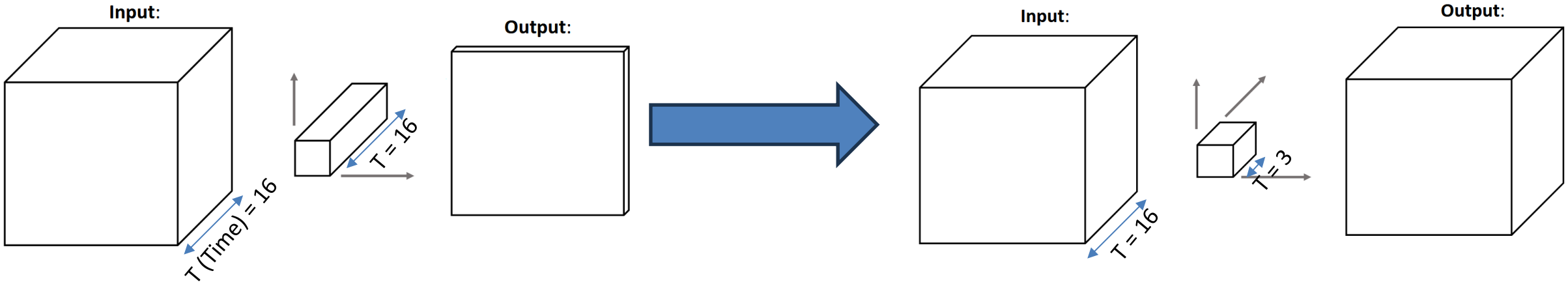
To get each element of the output volume we have to perform **5**x**5**x**192** operations (products)
→ **5**x**5**x**192**x**28**x**28**x**32** = ~120M multiplications

Now, we have to perform:
a) 1x1x192x28x28x16 = ~2.4M mult.
b) 5x5x16x28x28x32 = ~10M mult.

Total (a+b): ~12.4M operations

Convolutional Networks

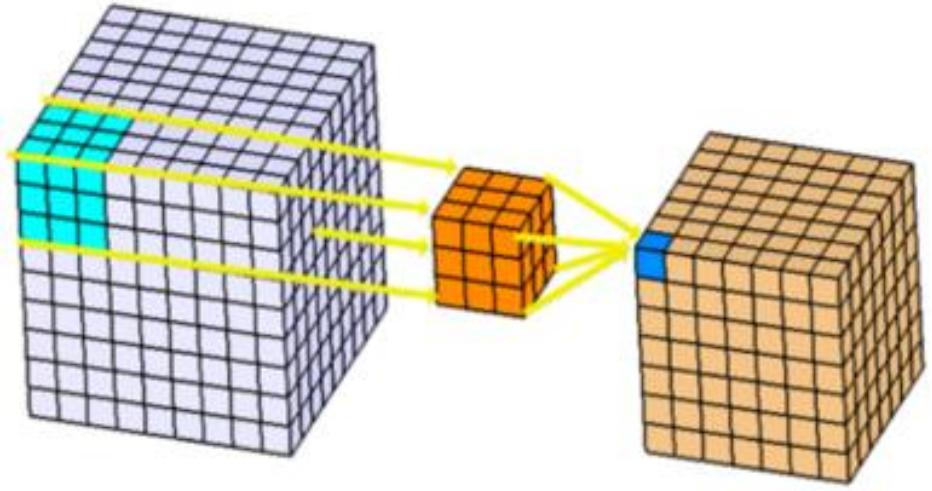
- **3D Convolutions**
  - They are used to process 3D information (for instance, spatio-temporal data, like videos), moving the kernels in 3 dimensions:
    - **2D convolutions:** When applied independently to each frame, they extract spatial features without directly combining information across time.
    - **3D convolutions:** They jointly extract spatial and temporal features, sharing filter weights across spatial and temporal positions.



https://web.eecs.umich.edu/~justincj/slides/eecs498/498_FA2019_lecture18.pdf
YouTube video: https://youtu.be/A9D6NXBJdwU?list=PL5-TkQAfAZFbzxjBHtzdVCWE0Zbhomg7r&t=1653

Convolutional Networks

- **3D Convolutions**
  - They are used to process 3D information (for instance, spatio-temporal data, like videos), moving the kernels in 3 dimensions:
    - **2D convolutions:** When applied independently to each frame, they extract spatial features without directly combining information across time.
    - **3D convolutions:** They jointly extract spatial and temporal features, sharing filter weights across spatial and temporal positions.

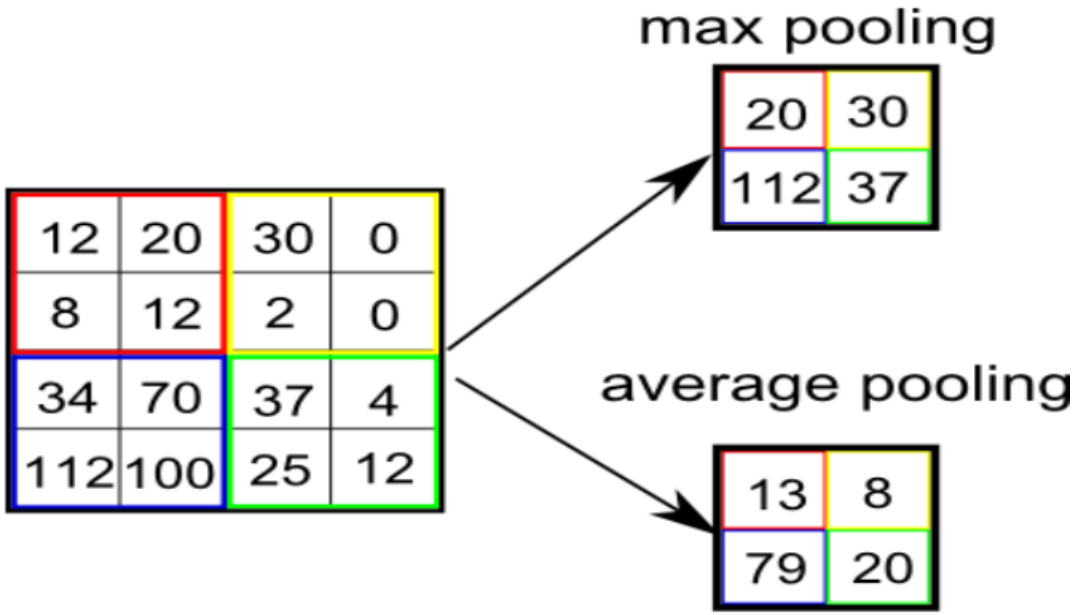


https://www.kaggle.com/code/shivamb/3d-convolutions-understanding-use-case

Convolutional Networks

- **Pooling**. Reduces dimensionality and introduces certain invariance to (small) translations of the input
  - If the input is shifted by a small amount, *pooling* output values may not change.

Example of *max* and *average pooling* with 2x2 kernels/filters and stride 2



Convolutional Networks

- **Pooling** (technical note)

```
import torch.nn as nn

pool1 = nn.MaxPool2d(2, 2)

pool2 = nn.AvgPool2d(3, stride=2, padding=1)

pool3  = nn.AdaptiveAvgPool2d((1,1))
```

Kernel size

Stride

Kernel size

Stride

Output size

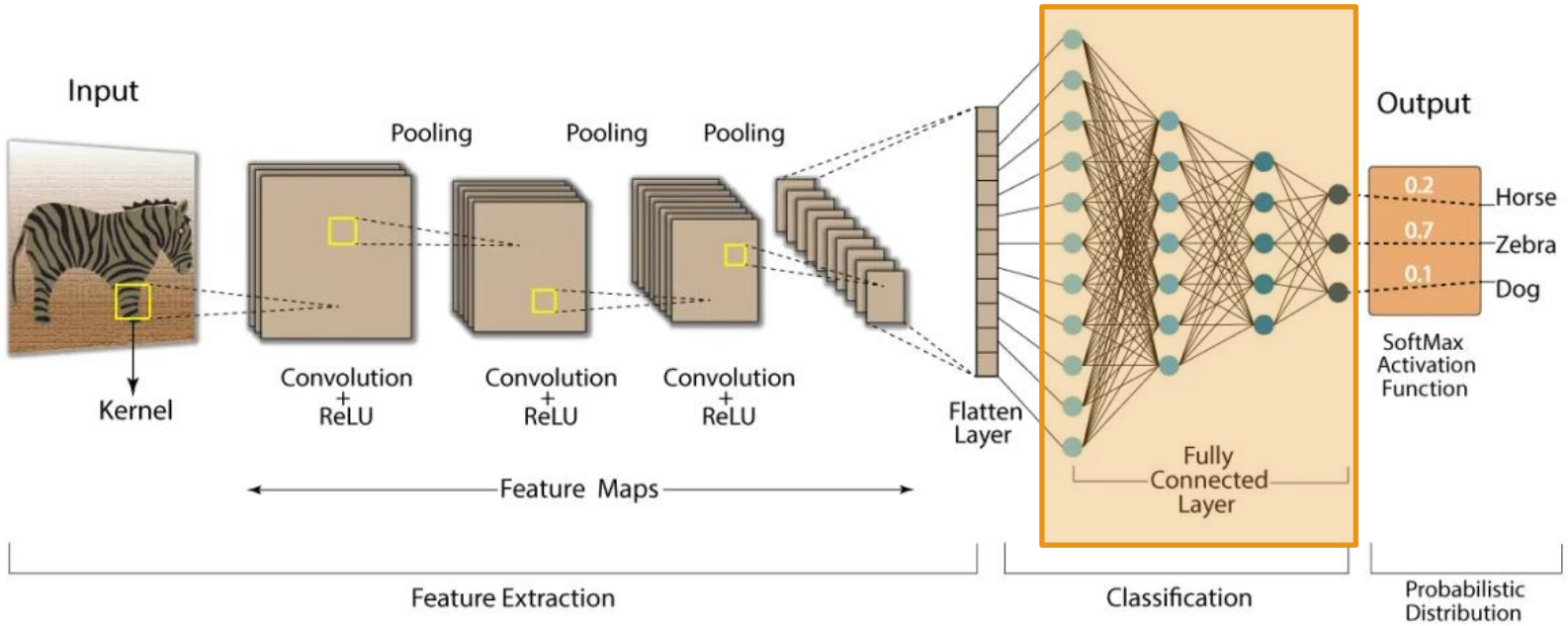
With adaptive pooling, you give the desired output size, and PyTorch computes the suitable kernel/stride.

This is called Global Average Pooling

In the example, if the input is `[Batch, Channels, Height, Width]` then the output is `[Batch, Channels, 1, 1]`

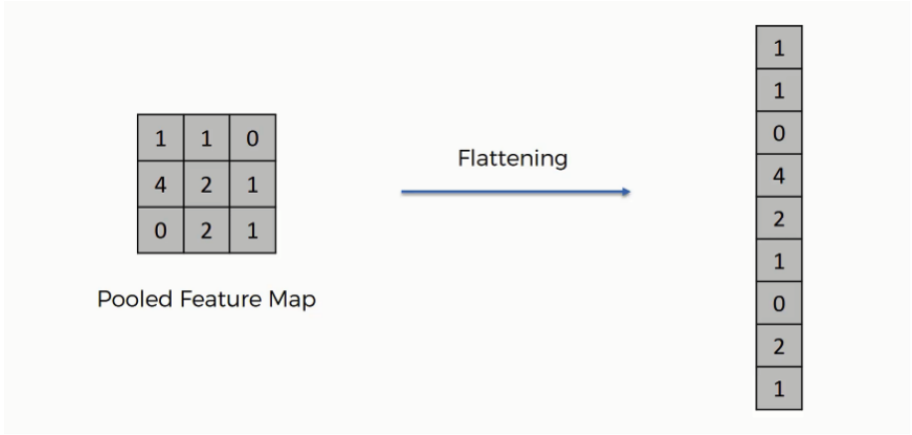
Convolutional Networks

- **Fully-connected or dense layers**.
  - Convolutional networks may or may not have fully-connected layers.
  - When these are present, they usually appear at the end of the network.
  - All units in one layer are connected to all units of the next layer.
  - They usually contain many parameters, so be careful!

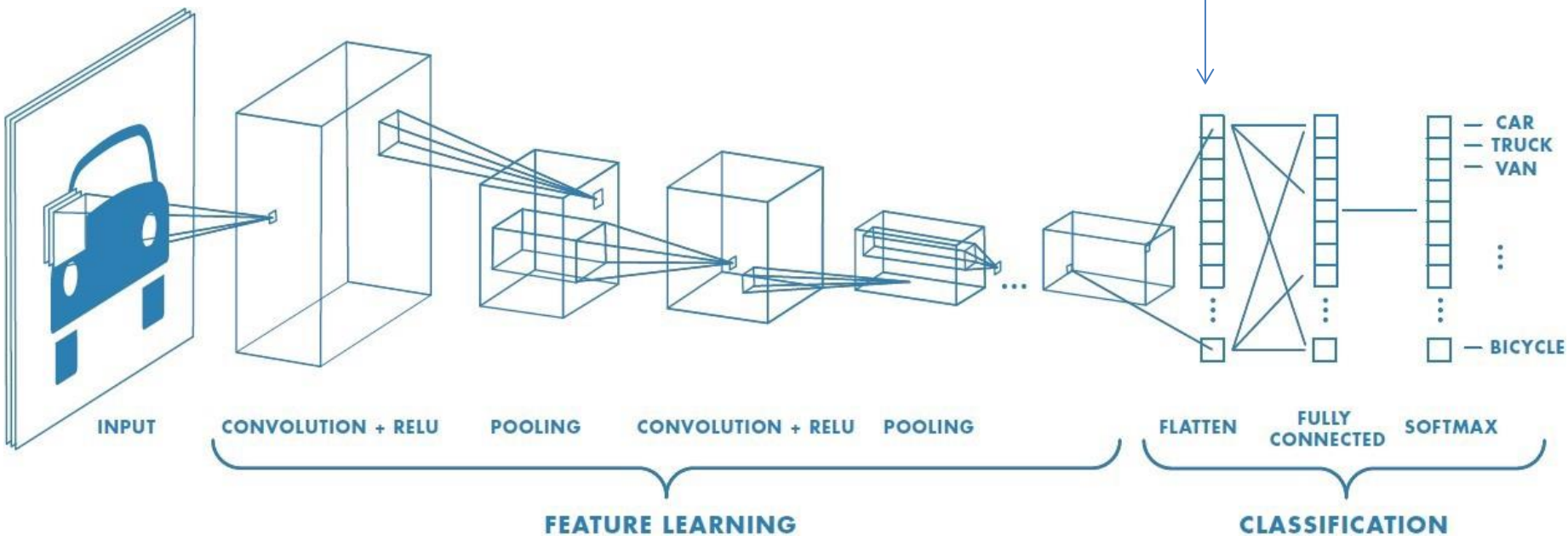


https://developersbreach.com/convolution-neural-network-deep-learning/

Combining everything



Flatten: Layer that "flattens" the *features*, allowing to pass from a volume (set of feature maps) to a vector (that can be processed by a fully-connected layer)



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Combining everything

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

σ = softmax

$\vec{z}$ = input vector

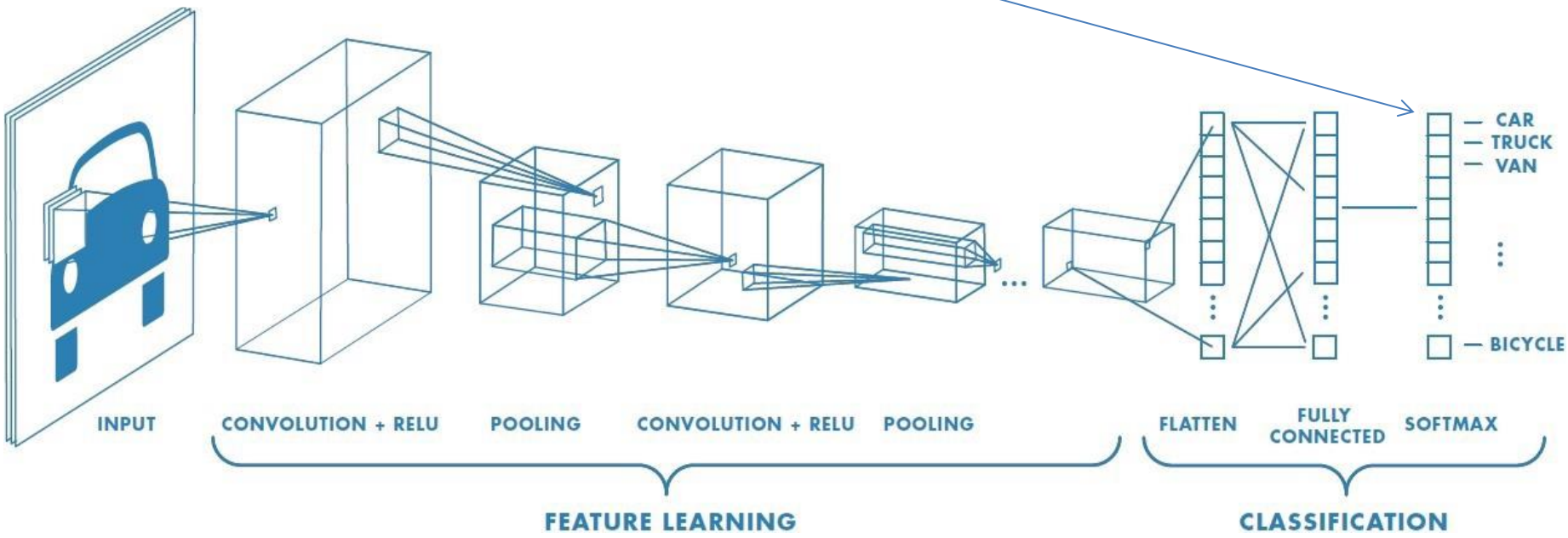
e^{z_i} = standard exponential function for input vector

K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

Softmax: activation function that generalizes the sigmoid function to multiple classes. Softmax converts logits into normalized scores that sum to 1 and can be interpreted as estimated class probabilities. However, these probabilities are not necessarily well calibrated.

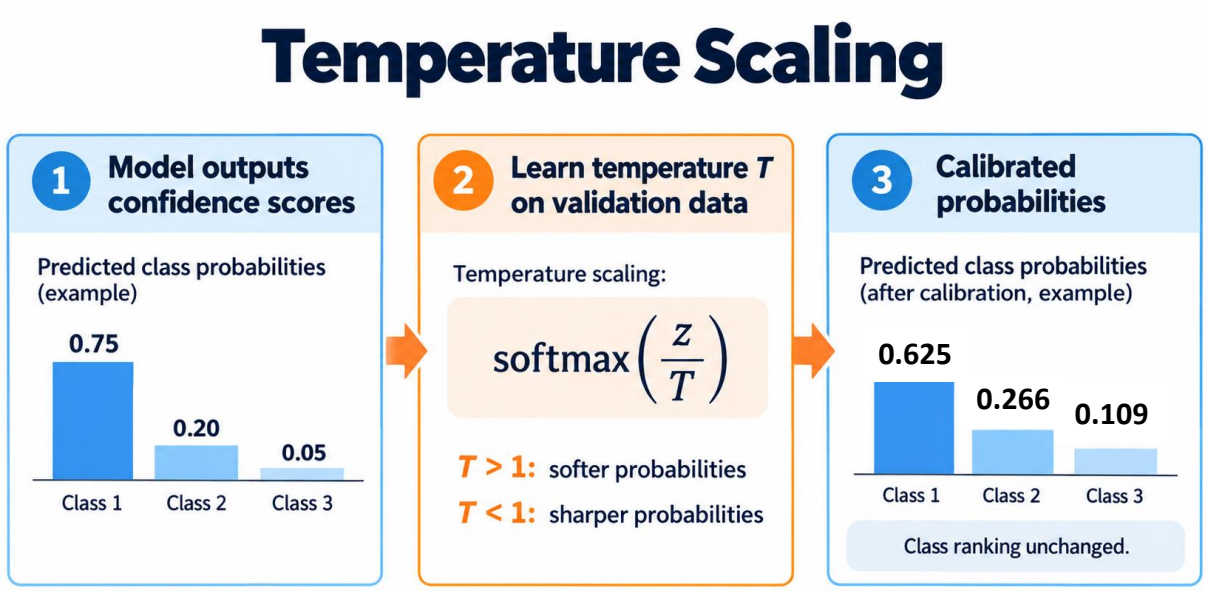
A **well-calibrated model** should be correct approximately p% of the time when making predictions with p% confidence. An **overconfident model** overestimates its confidence. For example, it may assign 90% confidence to its predictions but be correct only 60% of the time. An **underconfident model** underestimates its confidence.



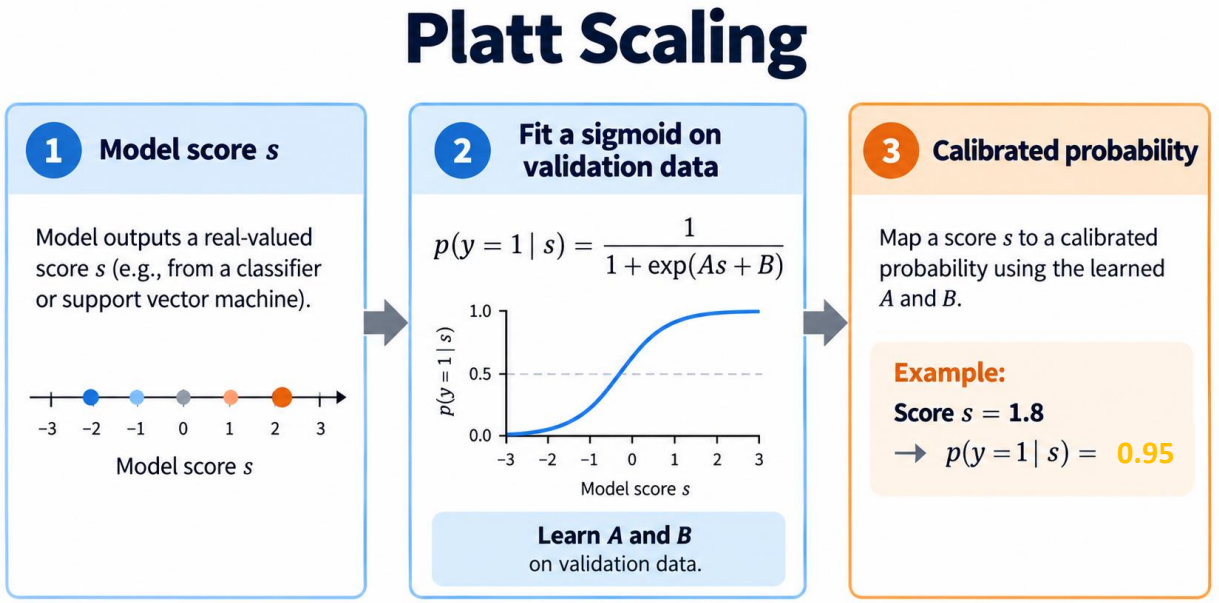
https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

How can we calibrate predicted probabilities?

There are several strategies for calibrating those probabilities, two of which are:



Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning (ICML), 70, 1321–1330.

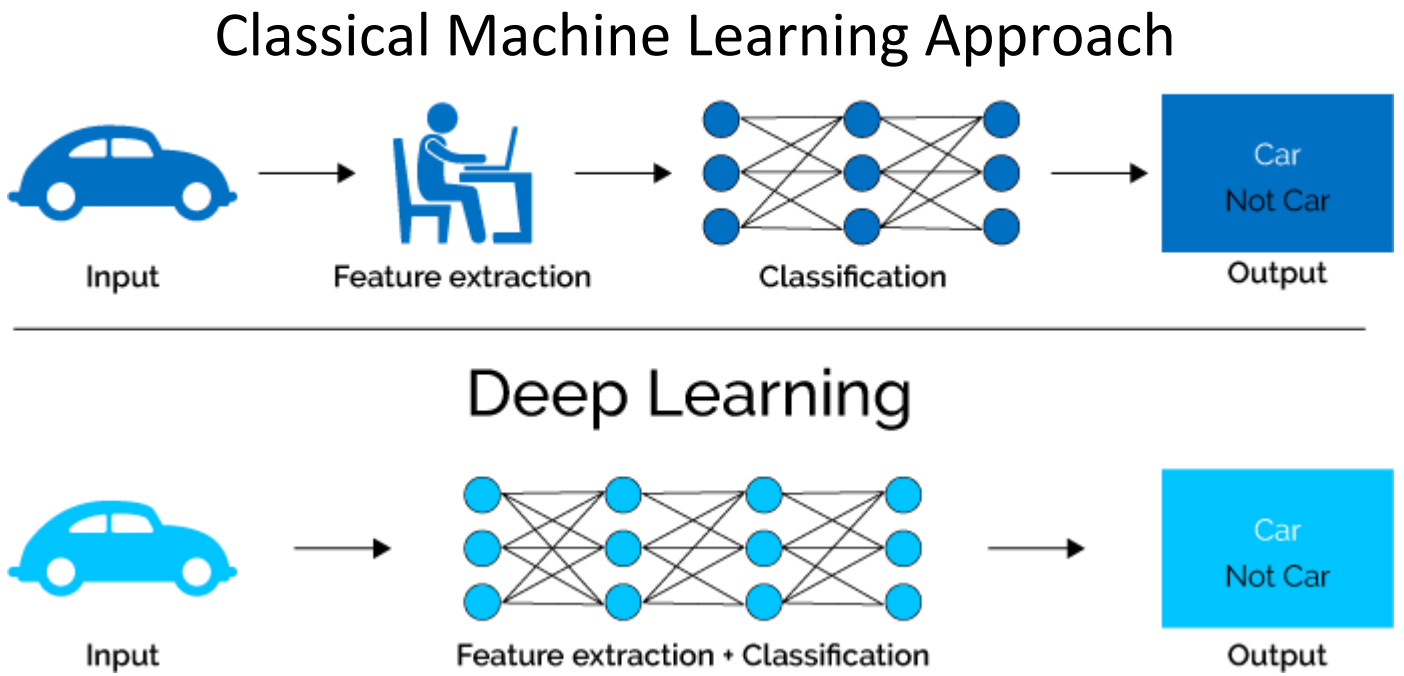


Platt, J. C. (1999). Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In Advances in Large Margin Classifiers (pp. 61–74). MIT Press.

The figures in this slide were created with the assistance of ChatGPT (GPT-6).

Machine Learning vs Deep Learning

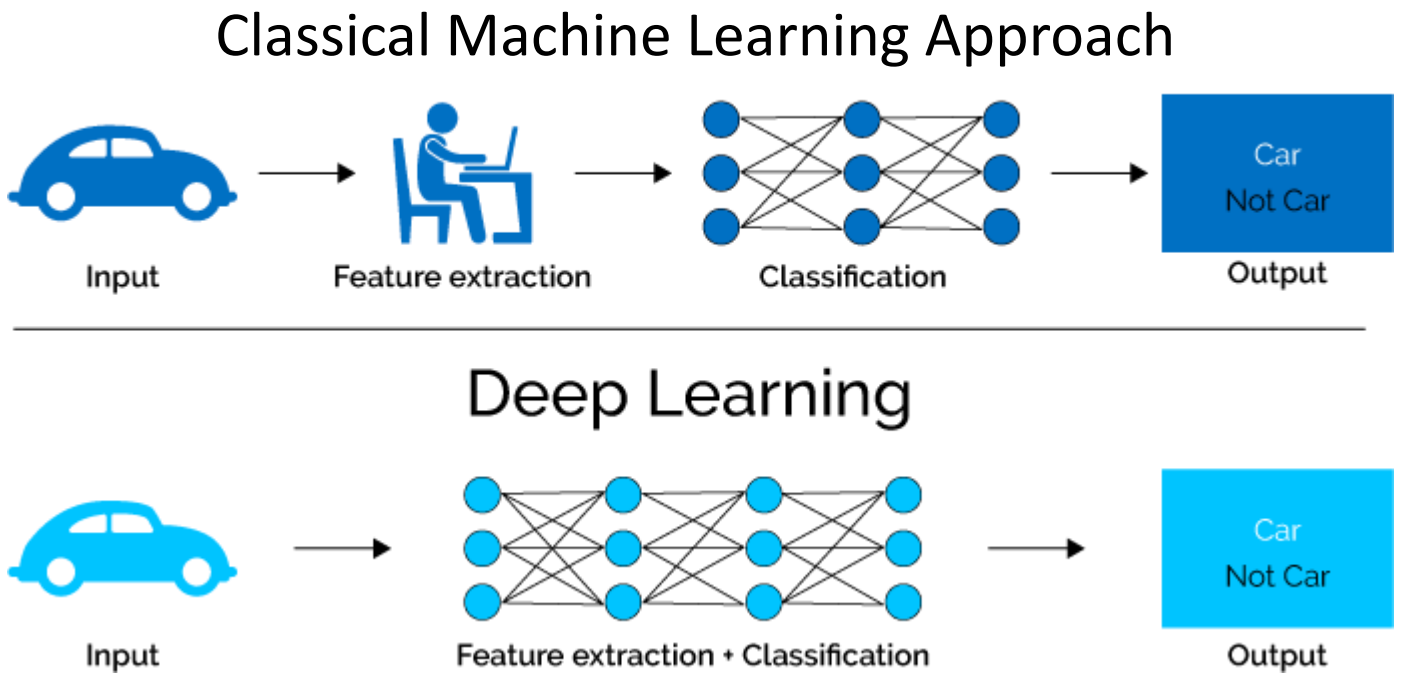
- Methodological level: ConvNets allow *feature/representation learning* instead of designing feature extractors by hand (*feature engineering*).
- Empirical level: they provide superior results in many vision tasks (classification, regression, segmentation, etc.)



https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e

Machine Learning vs Deep Learning

- Deep Learning blurs the boundaries between feature extraction and image classification (*end-to-end learning*).
- The key is to have a good internal representation of the input data.



https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e

Visualizing how it works

- ConvNetJS. Deep Learning in your browser: https://cs.stanford.edu/people/karpathy/convnetjs/
- A Neural Network Playground: https://playground.tensorflow.org/
- An Interactive Node-Link Visualization of Convolutional Neural Networks: https://adamharley.com/nn_vis/
- CNN Explainer: https://poloclub.github.io/cnn-explainer/
- ConvNet Playground: https://convnetplayground.fastforwardlabs.com/
- Topological visualisation of a convolutional neural network: https://terencebroad.com/works/cnn-vis

ConvNets training and regularization

How are ConvNets trained?

- *Backpropagation* is used.
  - Parameter learning becomes an optimization problem.

- Do not confuse *backprop* and gradient descent!
  - Gradient descent uses backprop!

Optimization method for minimizing a loss function using the gradient

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \gamma \nabla F(\mathbf{a}_n)$$

Method for calculating the gradient (i.e. the partial derivatives of the loss function with respect to the weights)

- *Backprop* efficiently applies the chain rule to compute gradients!

Intuitive explanations of *Backprop*: https://www.jeremyjordan.me/neural-networks-training/ and https://www.youtube.com/watch?v=Ilg3gGewQ5U
Interesting post in https://karpathy.medium.com/yes-you-should-understand-backprop-e2f06eab496b
Neural Networks and Backpropagation: http://cs231n.stanford.edu/slides/2023/lecture_4.pdf (https://www.youtube.com/watch?v=i94OvYb6noo)

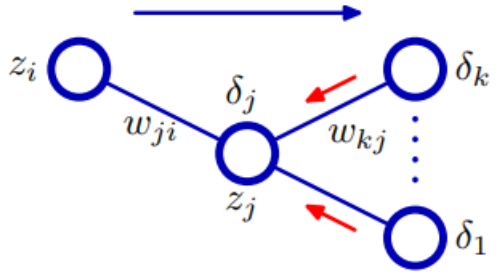
How are ConvNets trained?

Given a training example

1. Propagate it forward (*forward pass*), calculate activations and the output error
2. Go backwards (*backward pass*) calculating the error signals (δ_j) of each unit *j* on each layer, and the gradients with respect to the parameters
3. We update the parameters/weights using the calculated gradient. $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j}$$

$$z_j = h(a_j)$$



See Ch.5.3 in Bishop (2006)

Batch gradient descent: we use the average of the gradients of all training examples to update the weights

Mini-batch gradient descent: we use the average of the gradients of a subset of the training examples to update the weights

Stochastic gradient descent: we use the gradient of a single training example to update the weights

How are ConvNets trained?

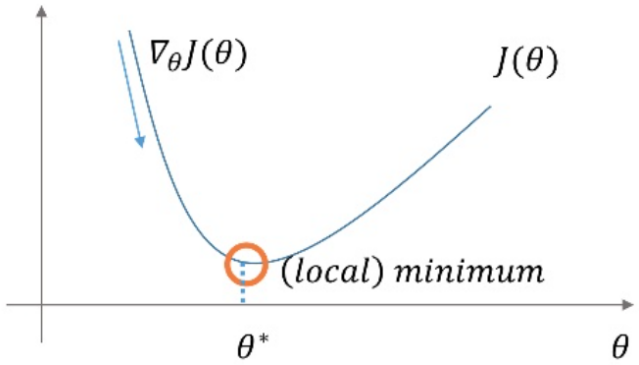
- Batch: a subset of training examples processed together during a training iteration.
  - Batch Gradient Descent. Batch Size = Training Set Size
  - Stochastic Gradient Descent. Batch Size = 1
  - Mini-Batch Gradient Descent. 1 < Batch Size < Training Set Size
- Epoch: each time the network, during training, sees the entire training set.
- Iteration: one optimization step (model's parameters update). Typically, the number of iterations per epoch is the number of training examples divided by the batch size (rounded up when necessary).

Example: a dataset with 1000 images, using mini-batches of 100 images
→ it will take 10 iterations to complete a single epoch.

How are ConvNets trained?

- Vanilla gradient descent has a single learning rate for all the weights and does not change throughout the training.

$$\theta = \theta - \eta \cdot \nabla_\theta J(\theta)$$

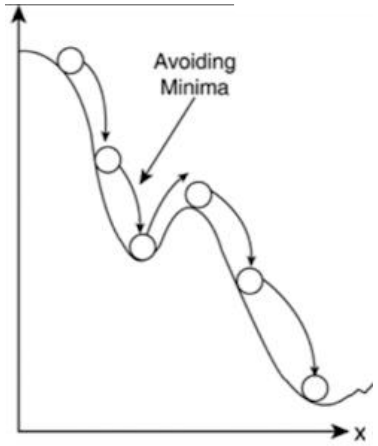


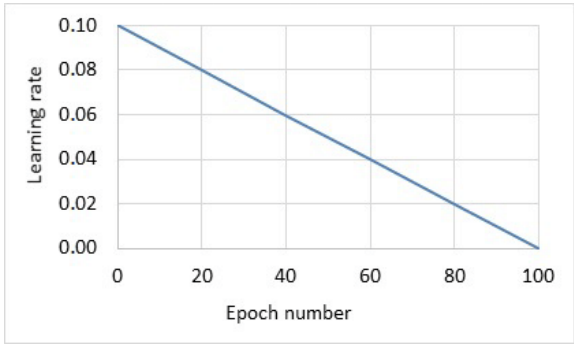
https://www.slideshare.net/SebastianRuder/optimization-for-deep-learning

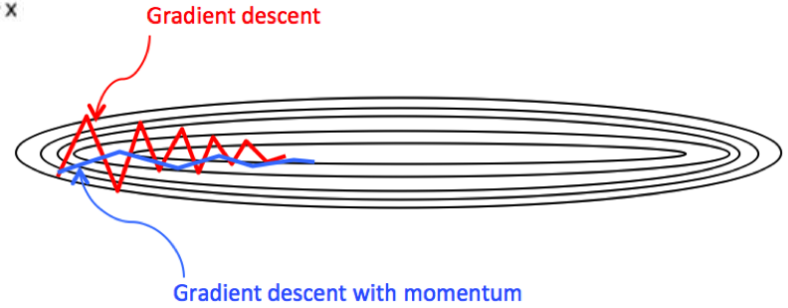
- There are many other algorithms and optimization strategies (https://www.ruder.io/optimizing-gradient-descent/, https://d2l.ai/chapter_optimization/index.html)

How are ConvNets trained?

- Learning rate decay
- SGD with momentum:
  - Accelerates the training (trajectory with less oscillations) and allows to skip local minima (https://distill.pub/2017/momentum/, https://www.youtube.com/watch?v=k8fTYJPd3_I)



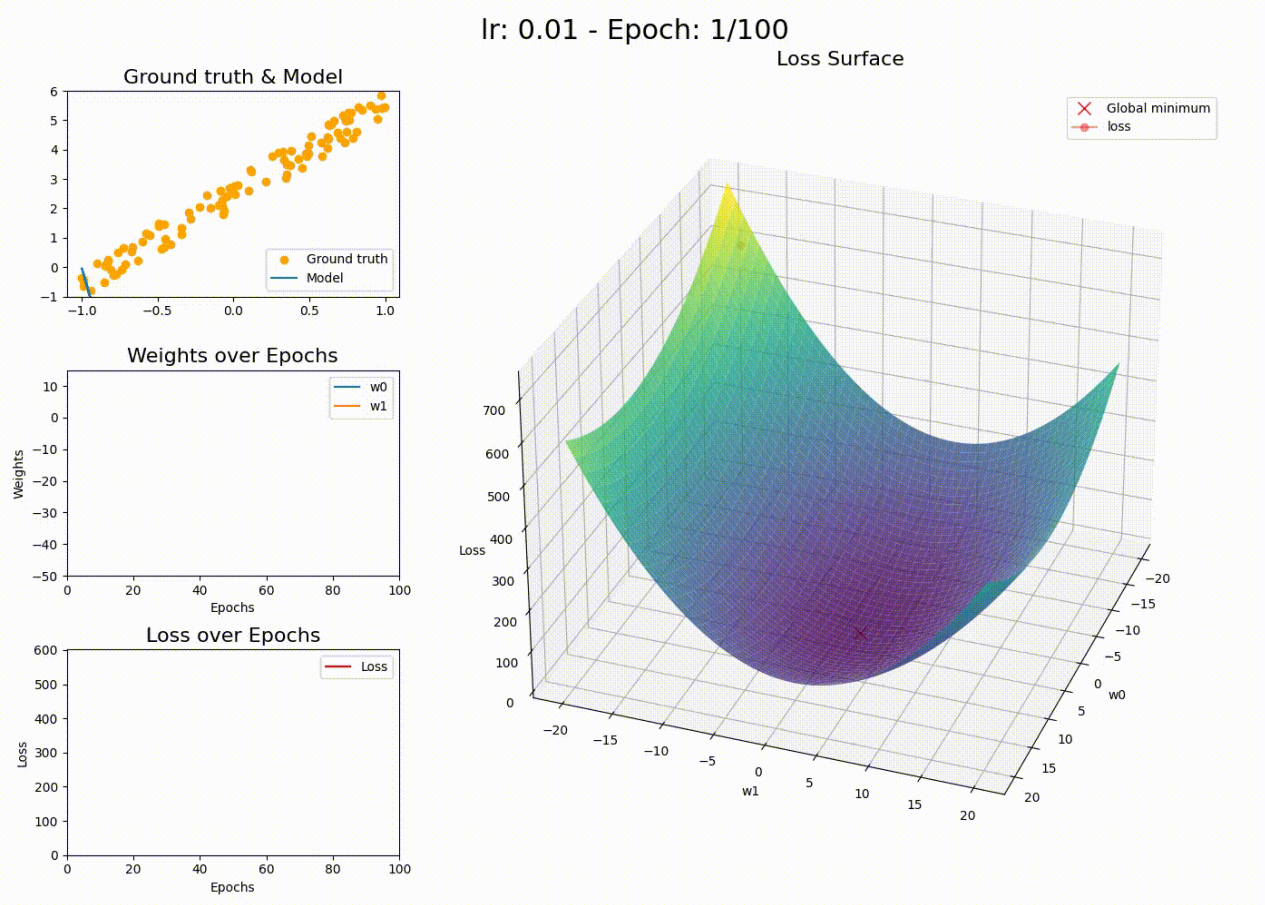




- Adam [Kingma and Ba, 2015]: RMSProp + momentum
  - Learning rate for each parameter
  - This learning rate is adaptive
- AdamW [Loshchilov and Hutter, 2019]: Adam + weight decay
  - Empirically provides models with better generalization
  - Corrects a bug in the way weight decay is commonly implemented in Adam (https://www.fast.ai/posts/2018-07-02-adam-weight-decay.html, https://towardsdatascience.com/why-adamw-matters-736223f31b5d)
    - AdamW decouples weight decay from the gradient-based optimization step.

Importance of learning rate

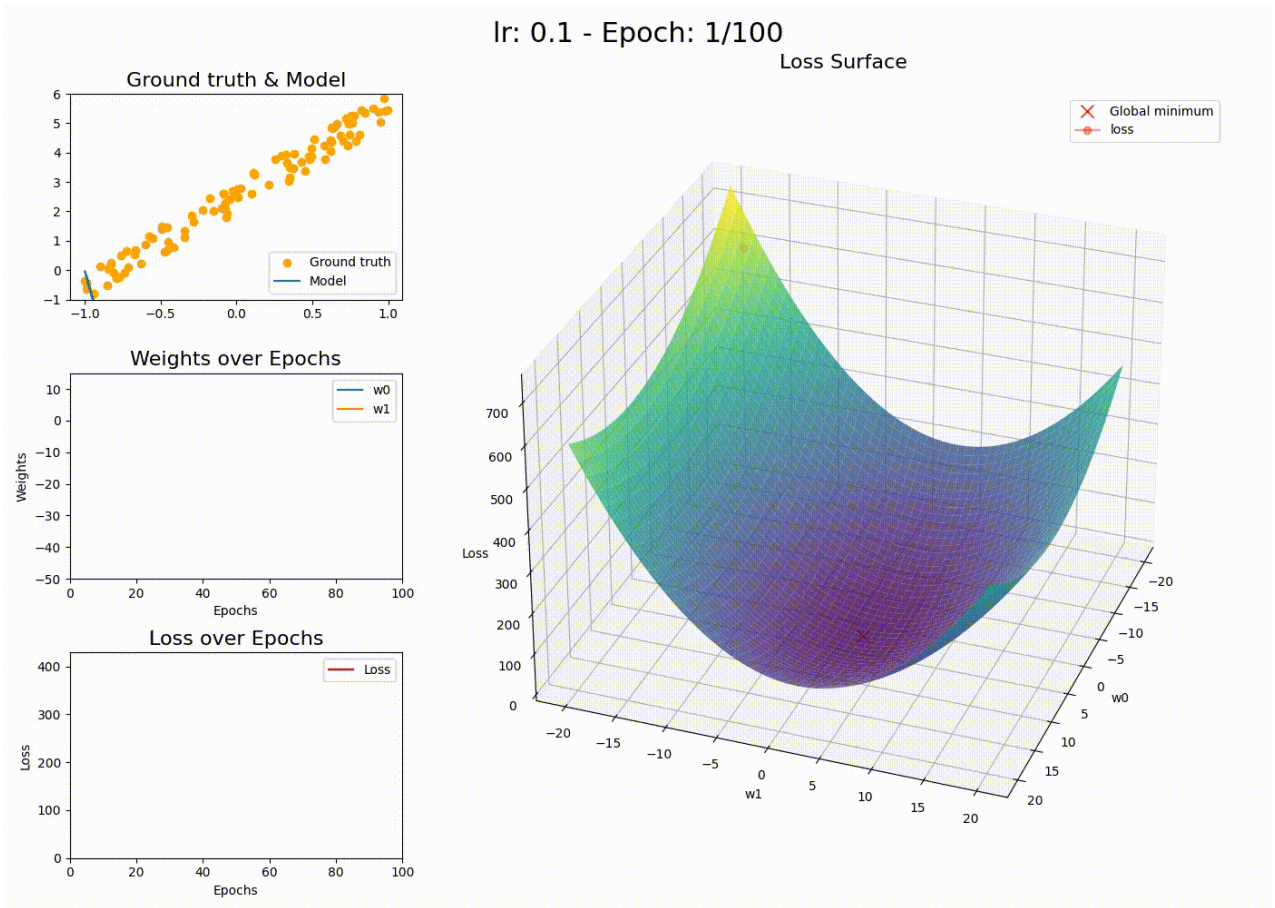
Learning rate too small: the optimization of the model progresses too slowly. In fact, it does not reach the optimum at the indicated epochs.



https://iconof.com/1cycle-learning-rate-policy/

Importance of learning rate

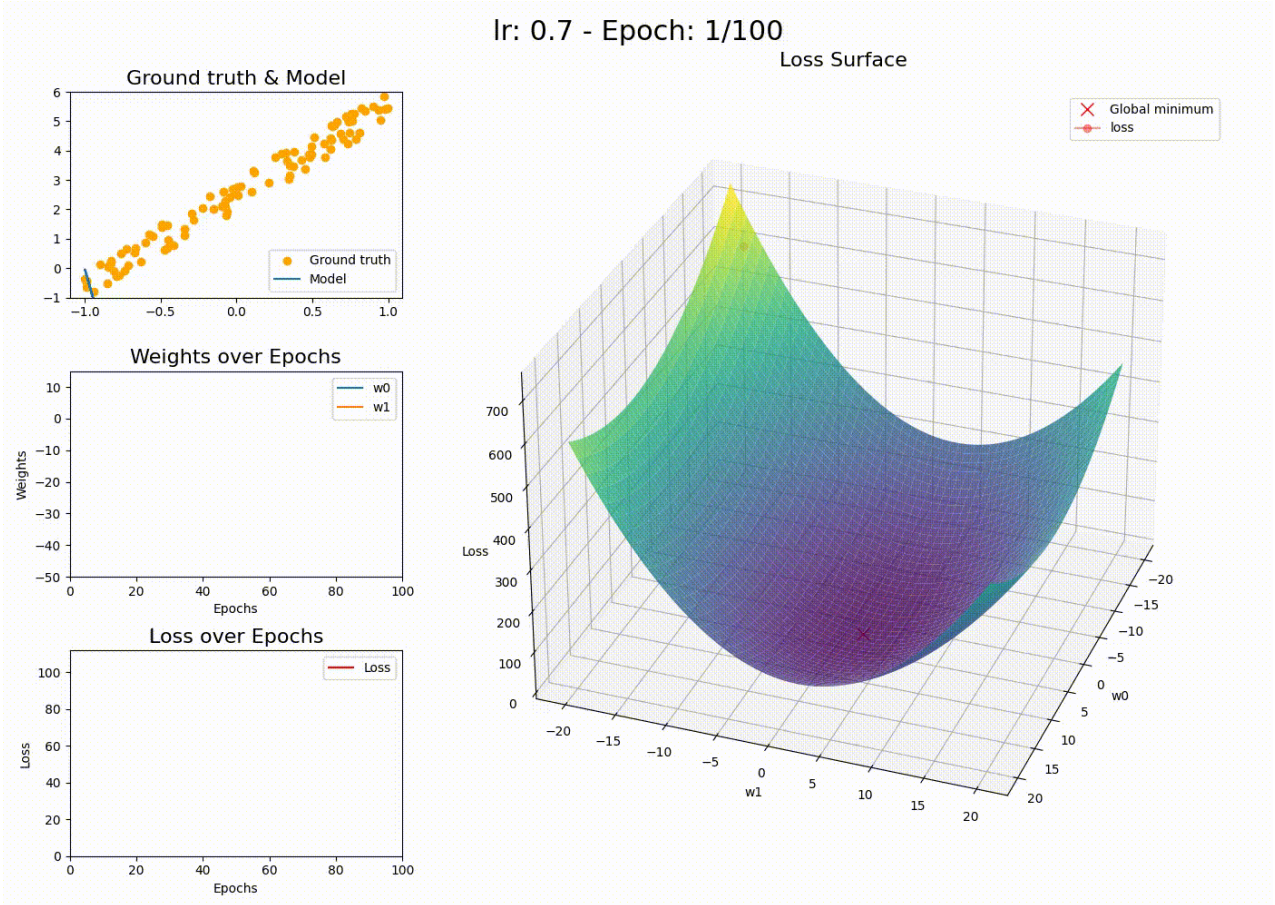
Adequate learning rate: the model converges within the indicated epochs.



https://iconof.com/1cycle-learning-rate-policy/

Importance of learning rate

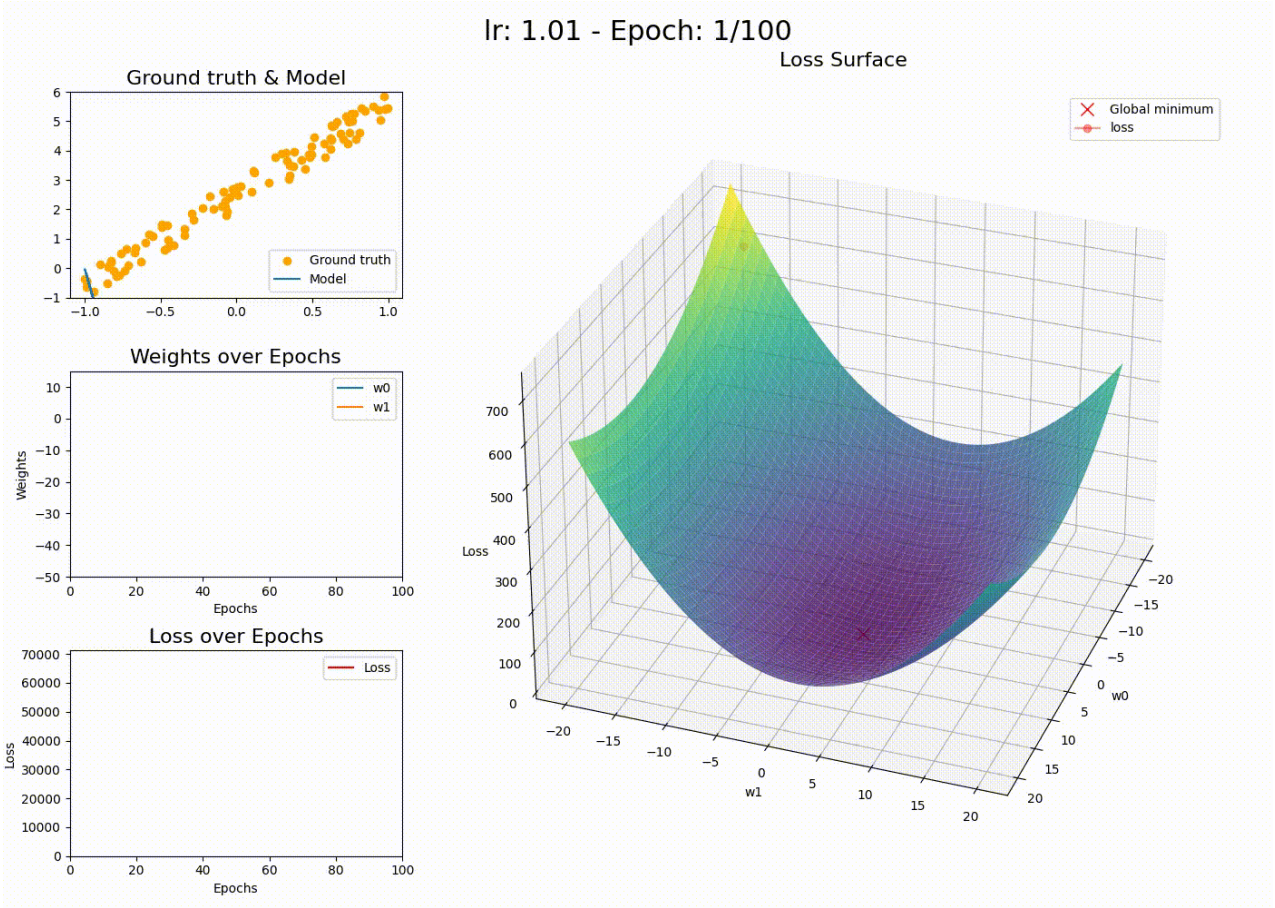
Optimal learning rate: the model reaches the optimum very quickly (less than 10 epochs).



https://iconof.com/1cycle-learning-rate-policy/

Importance of learning rate

Learning rate too high: the model diverges.



https://iconof.com/1cycle-learning-rate-policy/

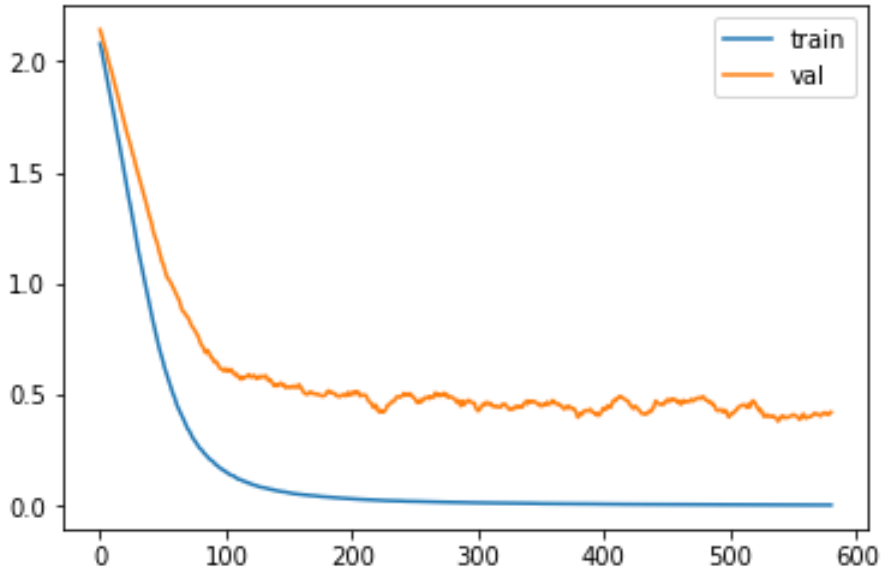
Ways to improve training

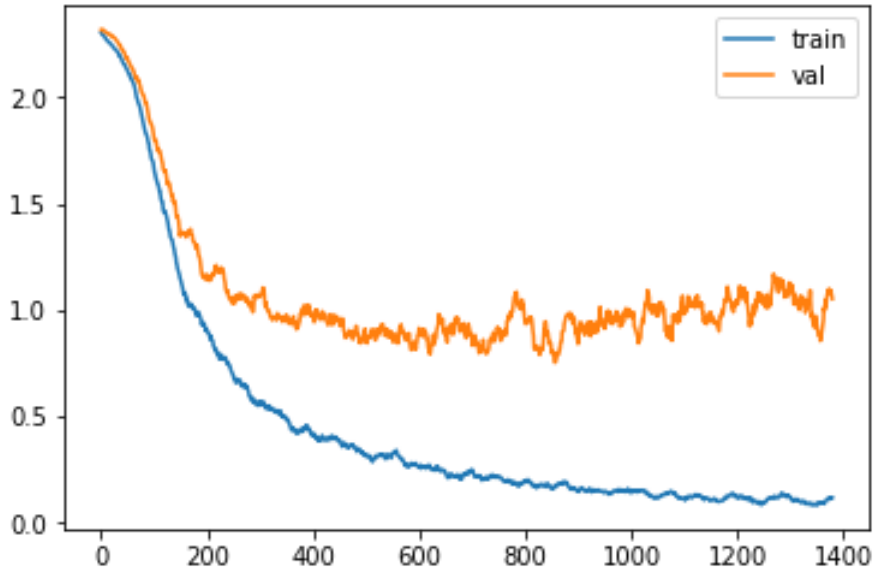
- Deep networks have many parameters → they often need large amounts of data to train and have a high risk of overfitting.

- What can we do to improve our training (i.e. accelerate it and/or regularize it)?
  - Introduce loss penalties
  - Use a different optimizer or select a better learning rate
  - Early Stopping
  - Data augmentation
  - Batch normalization and other ways of normalization
  - Dropout

Overfitting

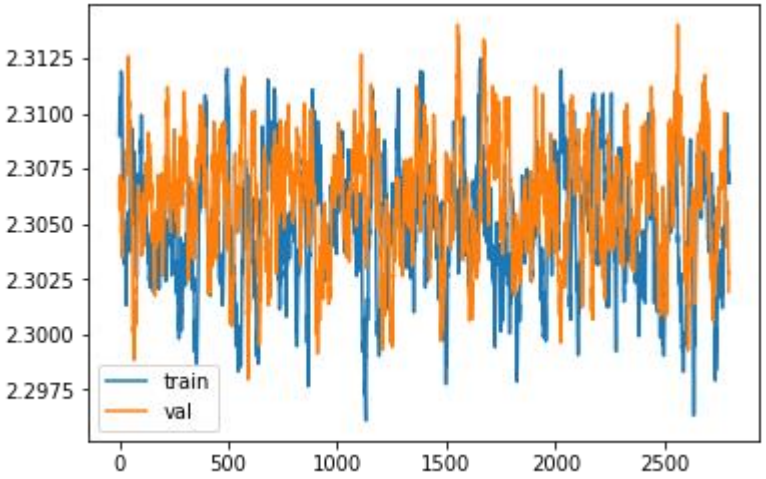
- The model does not generalize properly
- The model is too complex / we have too little data



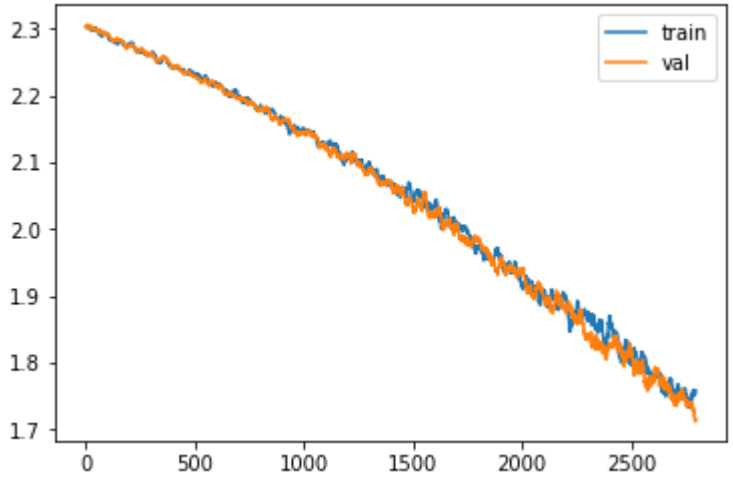


Look at the training curves because they give you a lot of information about what might be happening to you

Looking at training curves



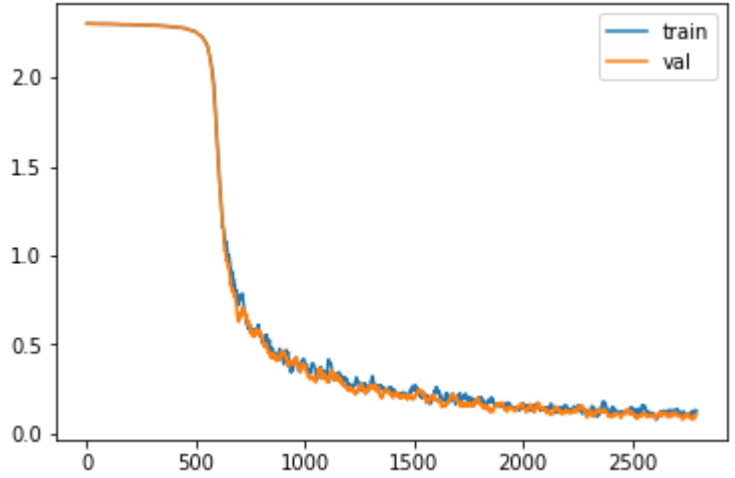
No apparent learning: check gradients, parameter updates, and learning rate.



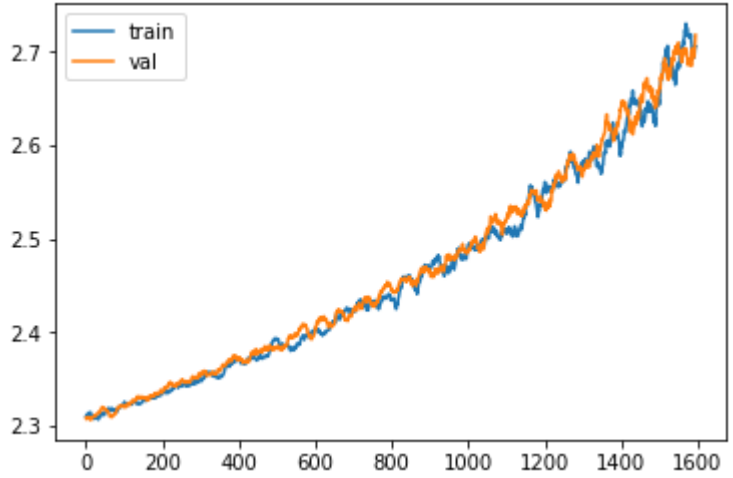
Loss still decreasing: training may benefit from additional iterations.

[How to use Learning Curves to Diagnose Machine Learning Model Performance]

[Tips and tricks for tuning NNs (Fei & Nish)]



Slow initial convergence: possibly related to initialization or learning-rate settings.



You might be performing gradient ascent instead of gradient descent (e.g., adding the gradient instead of subtracting it).

Looking at training curves

It is very important to carefully examine training curves, as they can provide valuable information. E.g., let's take a closer look at one of the graphs from the previous slide.

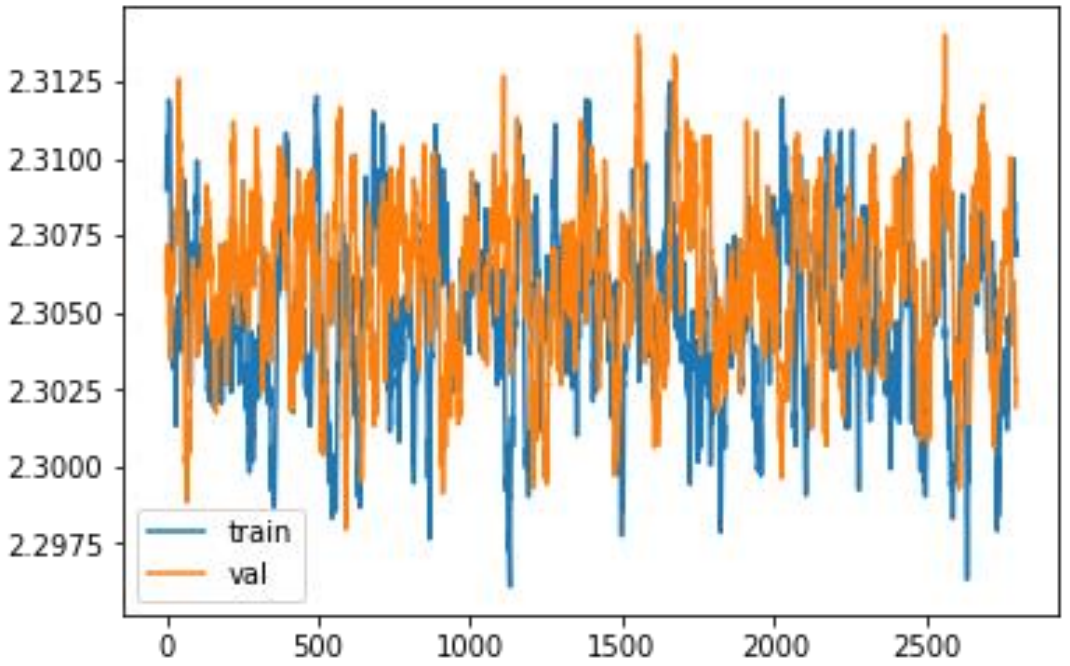
Categorical cross-entropy

$$L = -\sum_{i=1}^{K} y_i \ln(p_i)$$

y_i: ground-truth label (one-hot encoded; only the true class contributes)

p_i: predicted probability for class *i*

$$L = -\ln(p_{true})$$



Interpretation in this case

If the model predicts almost uniform probabilities over 10 classes, then $p_{true} \approx 0.1$.

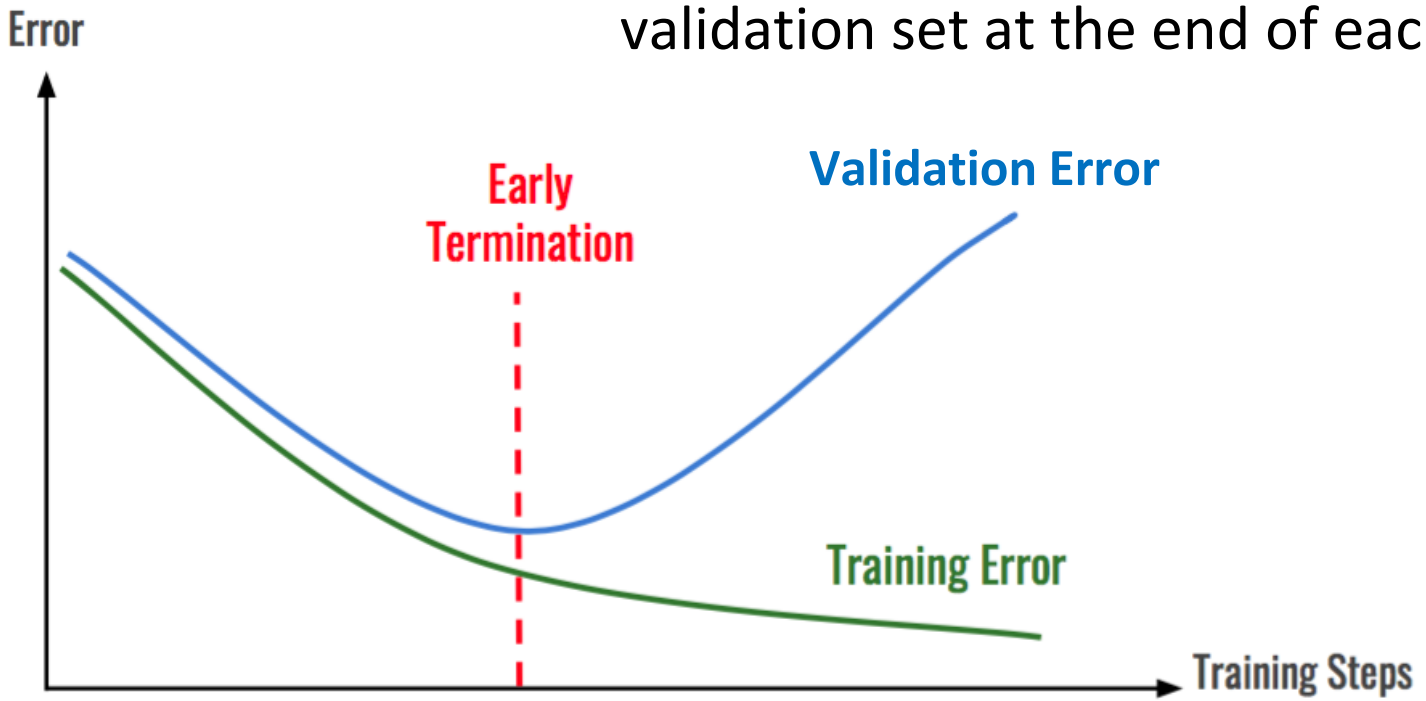
$$L = -\ln(0.1) \approx 2.30$$

Loss values around 2.30 suggest the model is behaving close to random guessing.

If this is a 10-class classification problem…
Training and validation curves around 2.30 suggest **no apparent learning**.

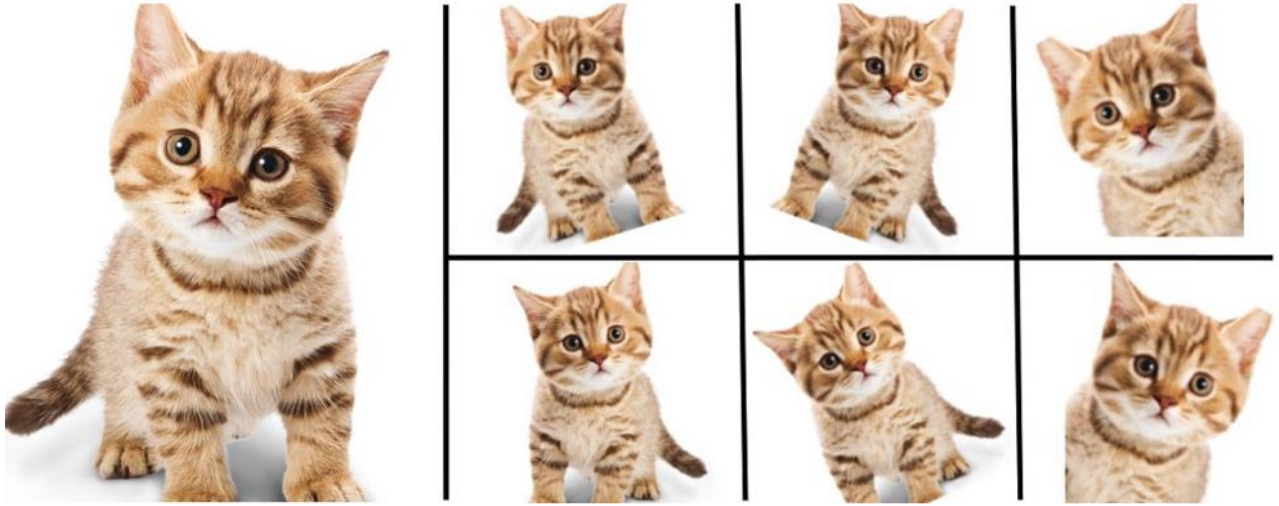
Early Stopping

Typically, the model is evaluated on the validation set at the end of each epoch.

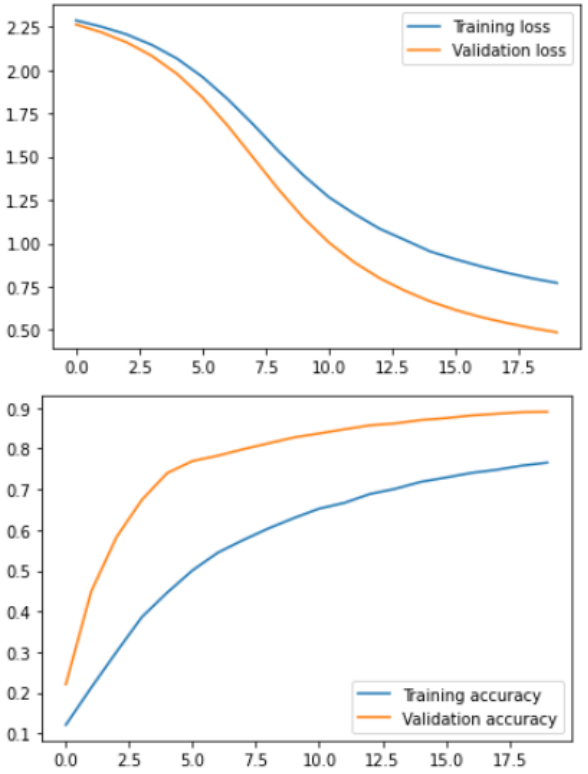


Data Augmentation

- We apply different **transformations to the training data** to increase their diversity and improve generalization.
  - E.g. in images: rotations, contrast changes, noise insertion, *mirroring*,…
  - Depending on the problem, some transformations will be valid and others will not. E.g. if you want to classify digits and you rotate a 6 a lot, it could become a 9…



On the evolution of errors



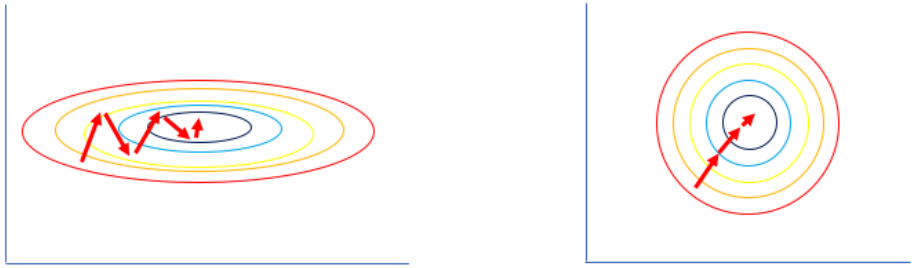
Possible behavior: the network provides better results in validation than in training.

Possible reasons:

1) Error in visualization/implementation
2) The validation set is "easier" than the training set
   - This can be verified by testing with different partitions and percentages of data, and see if the behavior is the same.
   - Be careful: data augmentation can make training more difficult than validation!
3) Influence of the way the *framework* computes errors:
   - For example, if the *training loss* is the average of the *losses* of all *batches* within an *epoch* (note that at the beginning of the epoch errors are generally higher), while *validation* is only at the end of the epoch (with the trained model).
4) Influence of Dropout (or other elements, such as BN).
   - Dropout is activated in training and deactivated in test/validation. Training accuracy is sacrificed for robustness and generalization in validation.

Ways to improve training

- Batch normalization [Ioffe & Szegedy, 2015]
  - It is known that normalizing input accelerates training



  - The idea is to normalize the inputs to the hidden layers (usually just before applying the activ. func.).

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

γ, β are parameters to be learned. Statistics are calculated for each batch and each channel. These params allow to have any mean and std (we maintain expressive power).

Ways to improve training

- Batch normalization [Ioffe & Szegedy, 2015]
  - γ, β are parameters to be learned (like any other weight).
  - statistics (μ, σ^2) are not learned, but are "stored" as part of the "state" of that BN layer for each channel.
    - In test, an *exponential moving average* of statistics (μ, σ^2) is used.

      - During training, we store $moving_{avg}$ and $moving_{var}$:

        $$moving_{avg}(t) = (1 - \alpha) \cdot moving_{avg}(t-1) + \alpha \cdot \mu(t)$$

        $$moving_{var}(t) = (1 - \alpha) \cdot moving_{var}(t-1) + \alpha \cdot \sigma^2(t)$$

        With $\alpha = 0.1$ in PyTorch.

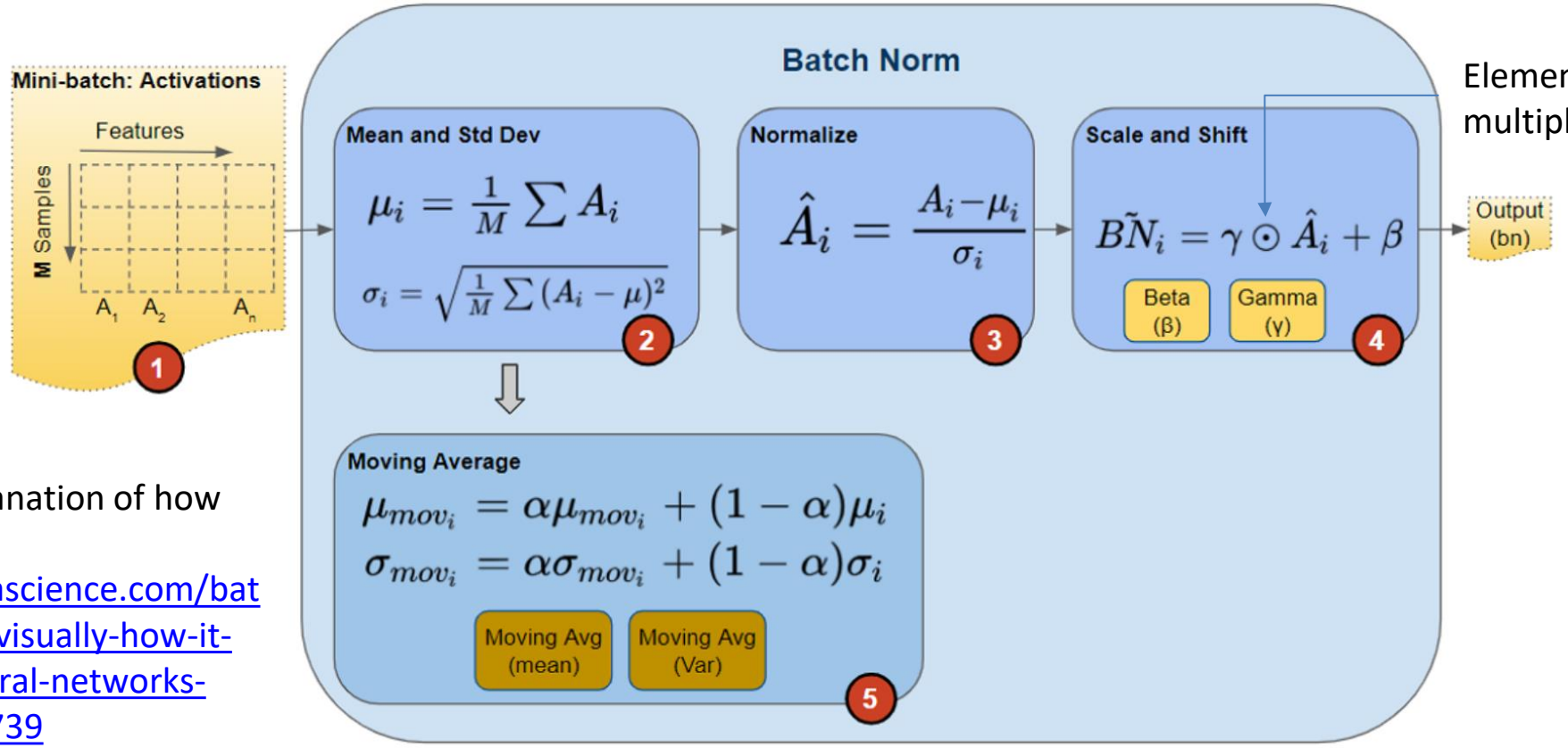
      - Then, in test, we apply:

        $$x_{new} = \frac{x - moving_{avg}}{\sqrt{moving_{var} + \epsilon}} \qquad y = \gamma \cdot x_{new} + \beta$$

  - These 4 values (2 learnable parameters and 2 running statistics) are maintained for each channel of each BN layer.

Ways to improve training

- Batch normalization [Ioffe & Szegedy, 2015]



Intuitive visual explanation of how BN works: https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739

In this example BN is included after activation. What is called *Features* in the figure are the channels.

The diagram shows the moving average of the standard deviation σ, but in PyTorch it is standard practice to store the moving average of the variance σ^2

Ways to improve training

- Other ways of normalization

We normalize each batch, taking each channel separately

We normalize each batch image separately, including all channels

We normalize each channel of each image (of each batch) separately

We normalize a set of channels from each image (from each batch)

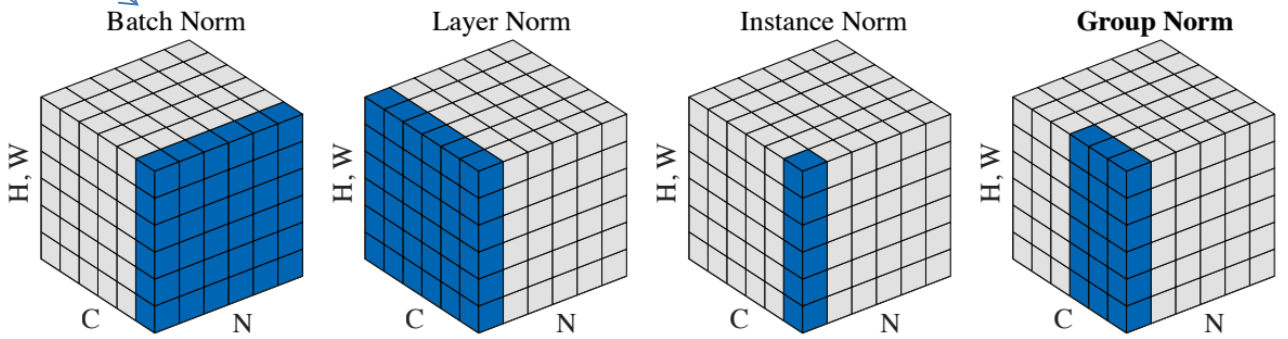
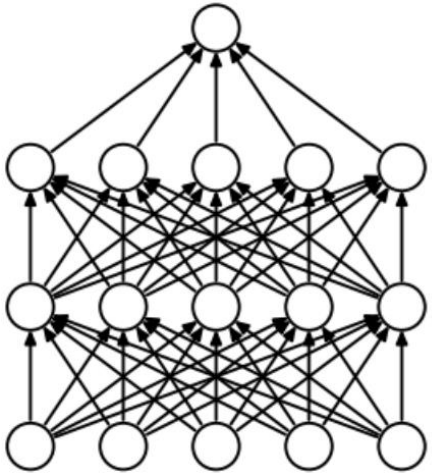


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

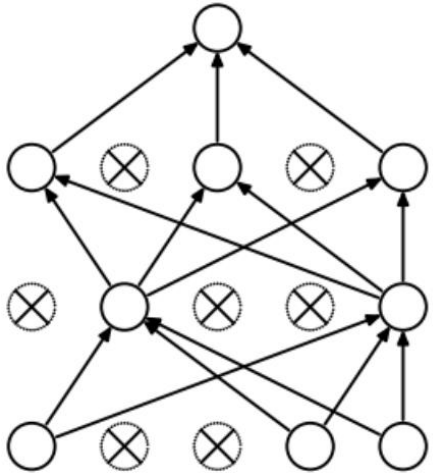
Image taken from Wu, Y. and He, K., 2018. Group normalization. https://arxiv.org/abs/1803.08494

Ways to improve training

- Dropout [Hinton et al., 2012]
  - We randomly eliminate hidden units during training.
  - It helps reduce co-adaptation between neurons, acting as a regularization mechanism.
  - Generally, it is applied in fully-connected layers, and with different units in each iteration (even if variants like torch.nn.Dropout2d also exists).



(a) Standard Neural Net



(b) After applying dropout.

Ways to improve training

- Dropout (technical note)

```
nn.Flatten(),
nn.Linear(in_features=1024, out_features=128),
nn.Dropout(p=0.5),
nn.ReLU(),
nn.Linear(in_features=128, out_features=25)
```

Dropout affects the activations at the output of the previous linear layer, setting a certain fraction of them to zero during training. $p = 0.5$ **means that each activation has an independent 50% probability of being set to zero**.

nn.Dropout2d zeroes out entire channels.
https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout2d.html

Ways to improve training

- Dropout (technical note)
  - In `nn.Dropout(p=0.25)`, `p` represents the probability of setting each activation to zero
    - Each activation has, on average, a 25% probability of being set to zero and a 75% probability of being retained during each training pass.

Classical dropout: scaling during inference

- Suppose a neuron with an activation value of 6.
- During training, with $p = 0.25$, its output will be:
  - 0 with probability 0.25.
  - 6 with probability 0.75.
- Expected activation:

$$E[a_{train}] = 0.25(0) + 0.75(6) = 4.5$$

- During inference, when dropout is no longer applied, we should multiply its activation by $1 - p$:

$$a_{test} = 6 \cdot 0.75 = 4.5$$

Inverted dropout: scaling during training

- Commonly used in frameworks such as PyTorch. Instead of modifying the activations during inference, we scale the activations that survive dropout during training.
- The original activation with a value of 6 will become:
  - 0 with probability 0.25.
  - $1 / (1 - p) = 1 / 0.75 = 4 / 3 \approx 1.333$
    → $6/0.75 =$ 8 with probability 0.75.
- Its expected value will be:

$$E[a_{train}] = 0.25(0) + 0.75(8) = 6$$

- And during inference:

$$a_{test} = 6$$

Classical and inverted dropout can produce identical outputs with appropriately rescaled weights. With identical weights, their outputs differ. In practice, training learns weights adapted to the chosen convention.

3. Final Remarks

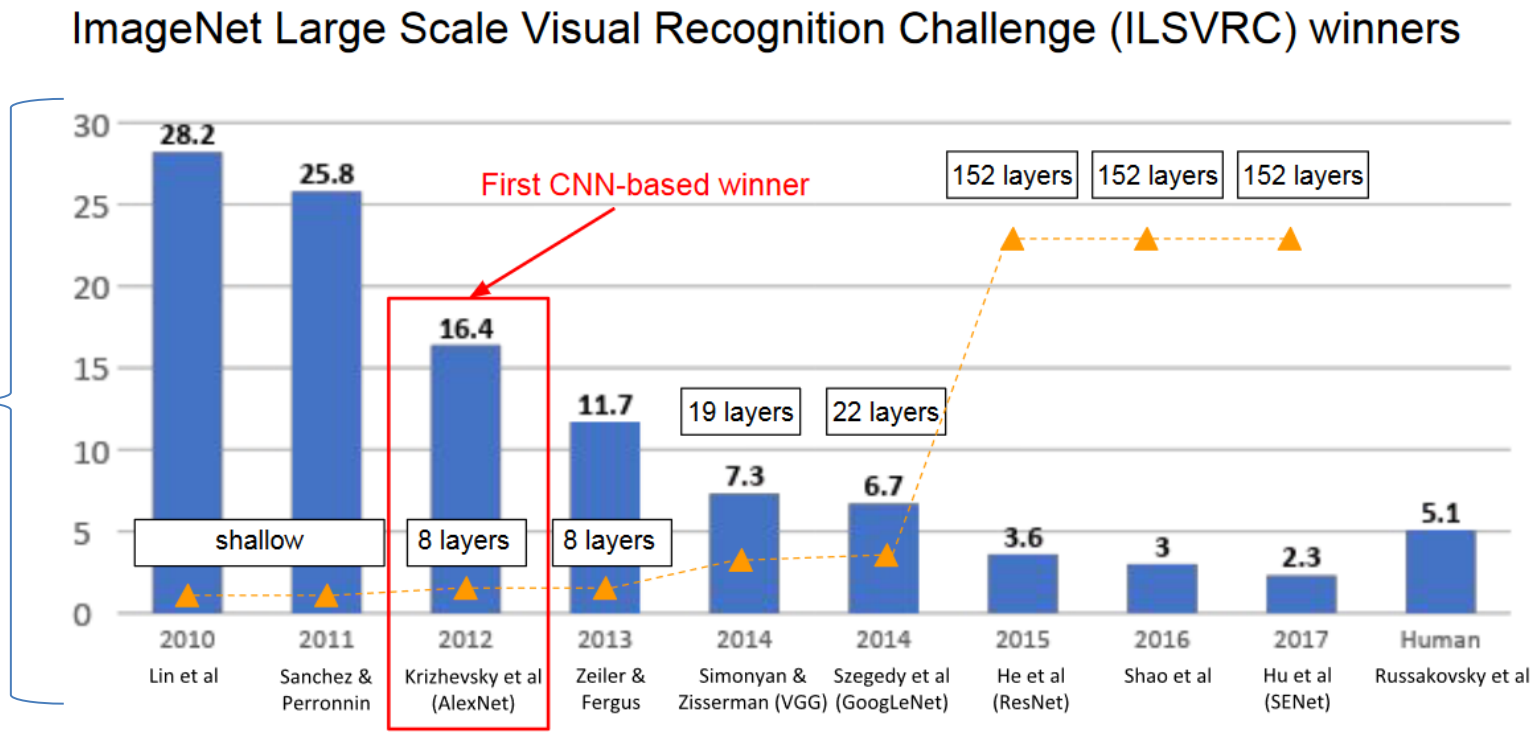
On stochasticity in neural networks

- You will see that when training a model you get different results each time
  - Neural network training often involves stochastic processes
    - Random weights initialization
    - Random selection of batches for training
    - Neurons randomly “turned off” in Dropout
  - The hardware architecture (specific GPU or TPU) you use for training can cause variations in the results, even if only slightly:
    - different underlying algorithms and optimizations,
    - varying precision/formats,
    - floating-point arithmetic is not associative,...
- During inference, the weights are fixed and the model is static, so the same input should generally produce the same output.
  - Unless stochastic mechanisms are introduced at inference, e.g., probabilistic decoding in LLMs/VLMs or latent/noise sampling in generative image models.

Some popular models

- LeNet-5 [LeCun et al., 1998]
- AlexNet [Krizhevsky et al., 2012]
  - Popularized the use of ReLU (already explored in earlier works, e.g., Nair&Hinton, 2010)
  - It was the take off of deep learning

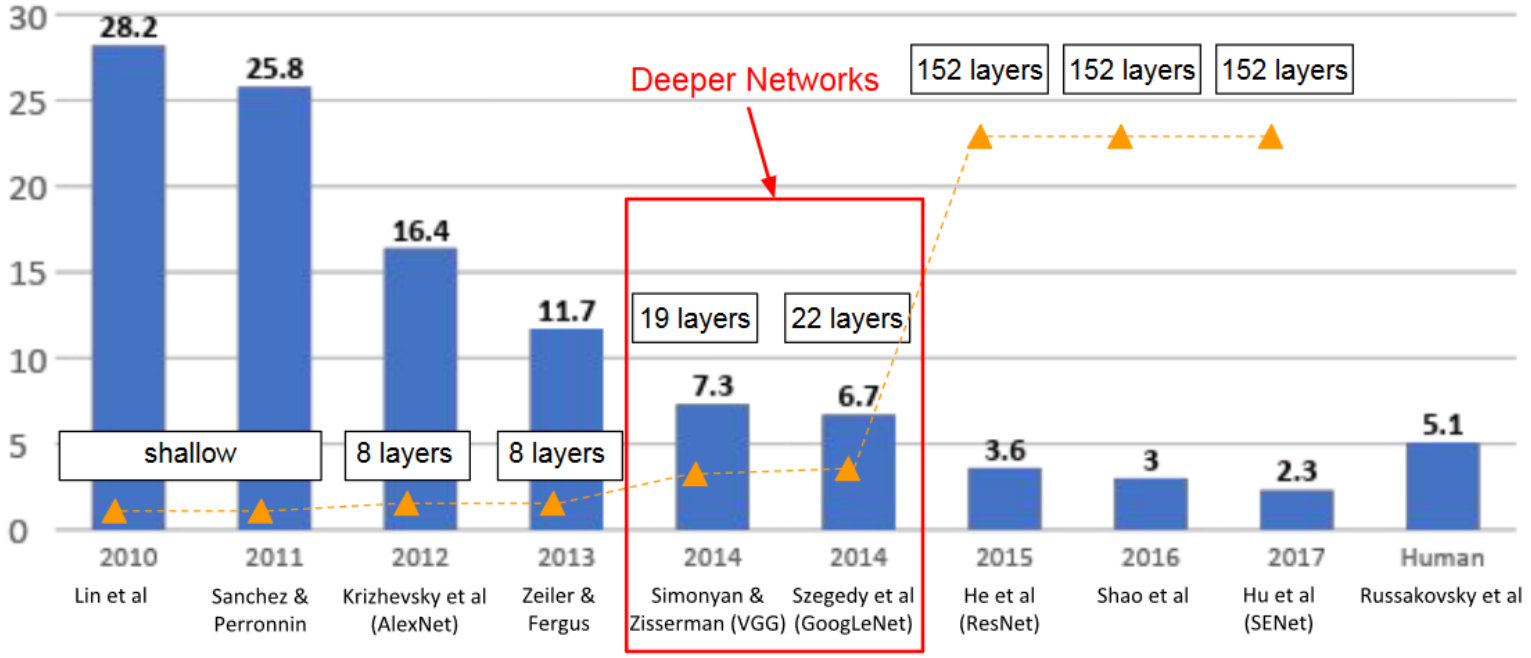
Top-5 error: Percentage of images for which the correct class is not among the model's 5 highest-ranked predictions.



Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung. http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture09.pdf

Some popular models

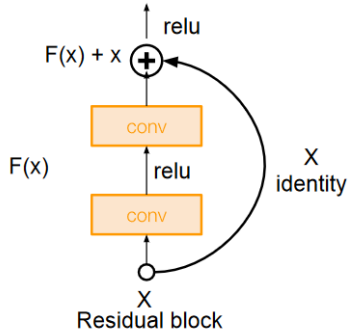
- VGGNet [Simonyan and Zisserman, 2014]
  - Smaller filters (3x3), deeper networks (more non-linearity)
- GoogLeNet [Szegedy et al., 2014]
  - Combines different receptive fields (Inception modules)
  - Uses 1x1 convolutions to reduce computational cost
  - Auxiliary intermediate outputs to “inject” additional gradient into lower layers

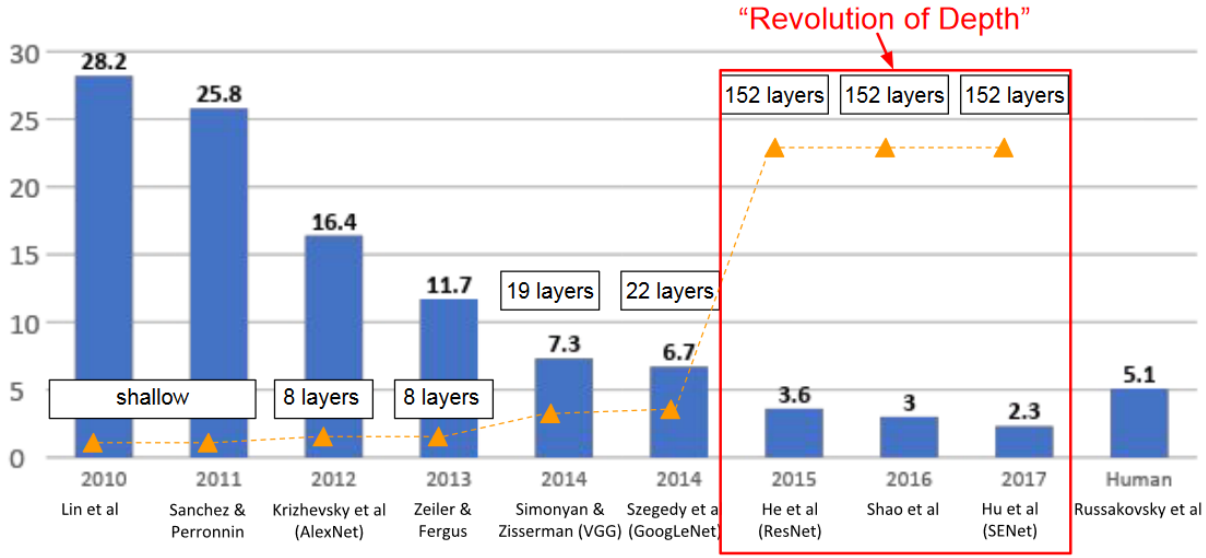


Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung. http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture09.pdf

Some popular models

- ResNet [He et al., 2015]
  - Very Deep networks using residual connections (*skip connections* together with *batch norm and* 1x1 convolutions)
- ResNeXt [Xie et al., 2016]
  - ResNet + parallel routing strategy inspired by Inception module
- [Shao et al., 2016]: combination of models (Inception, ResNet,…)
- SENet [Hu et al., 2017]: feature recalibration module to learn how to re-weight *feature maps*.





Slide credit: Fei-Fei Li, Justin Johnson and Serena Yeung. http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture09.pdf

Main conclusions

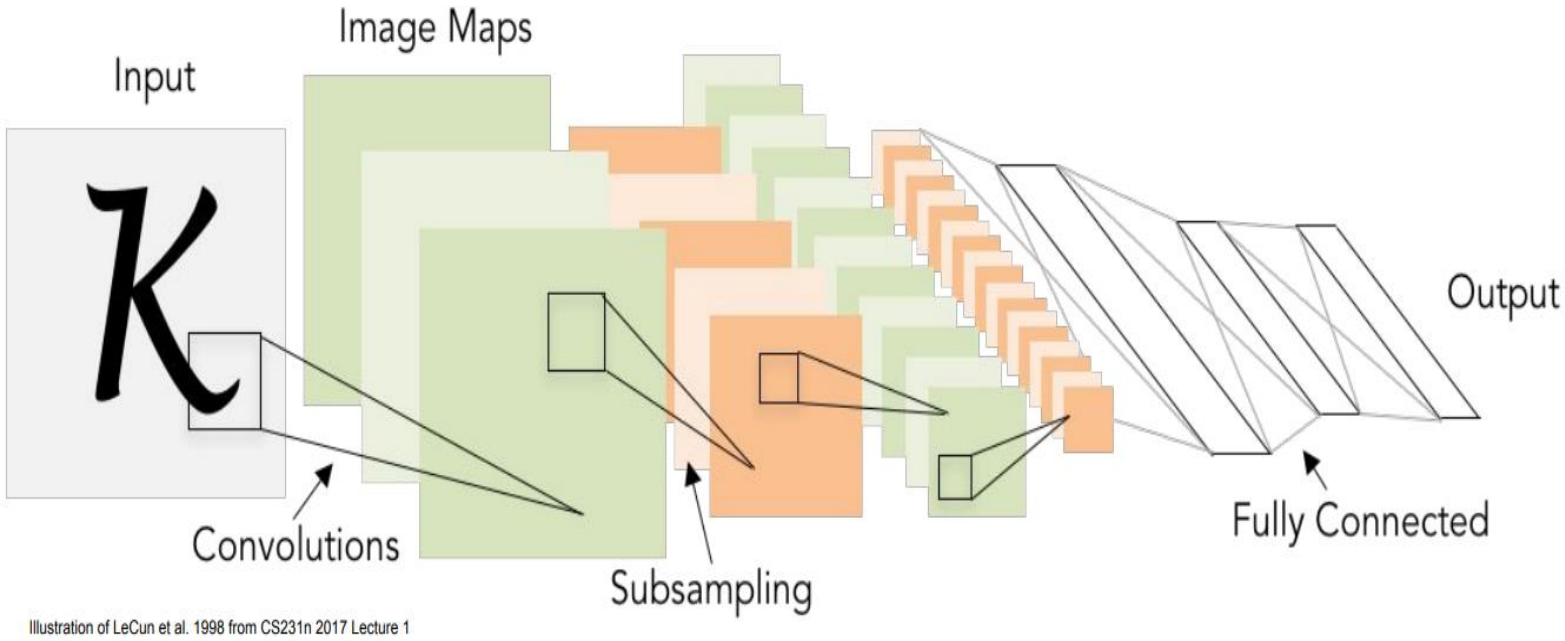
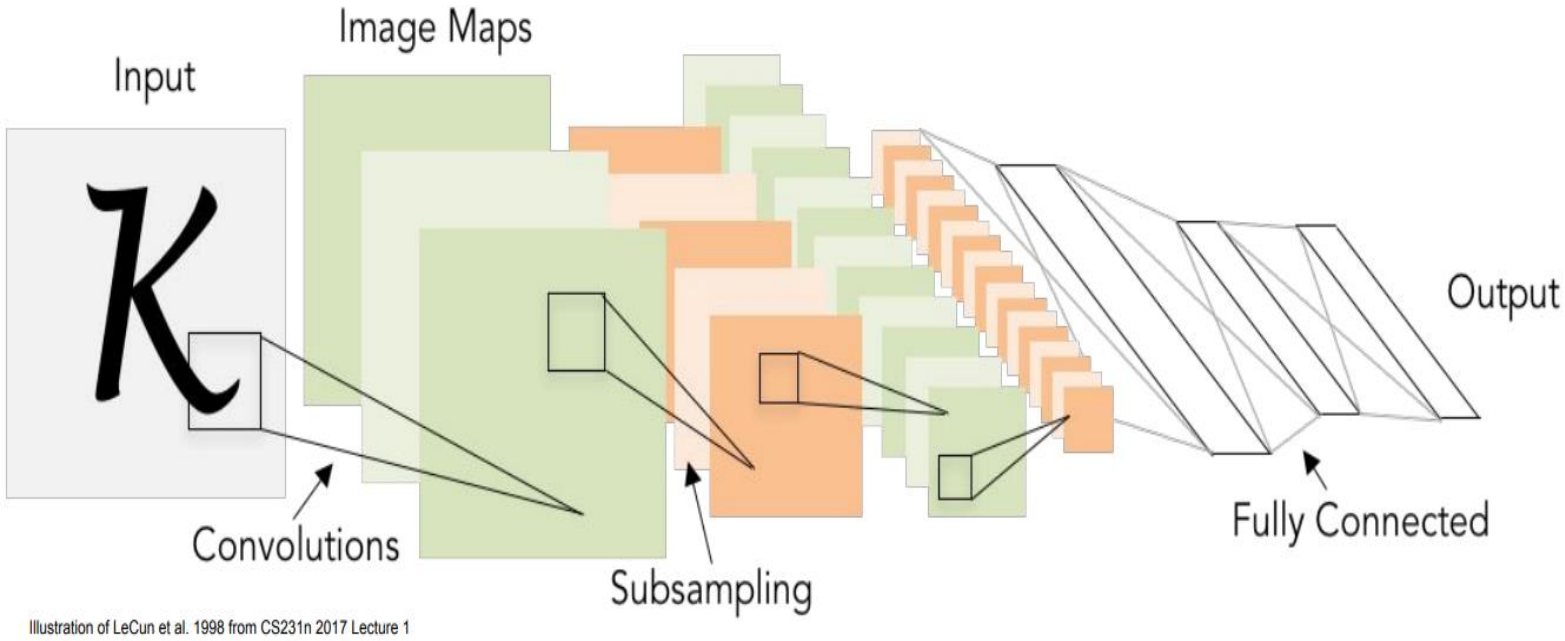


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

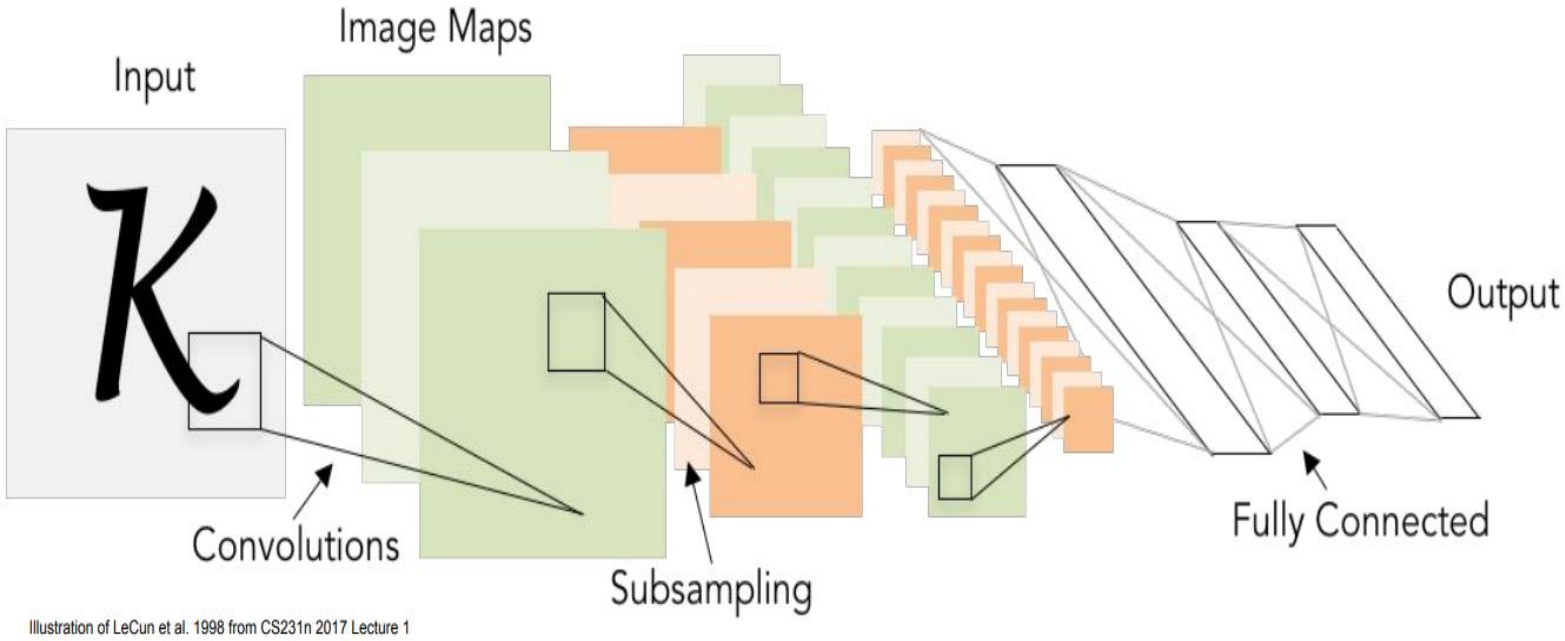
- Convolutional networks (ConvNets) are neural networks commonly applied to image analysis.
- They use convolutions to extract visual features (*feature maps*).
- Masks/kernels/filters are learned (network weights).

Main conclusions (2)



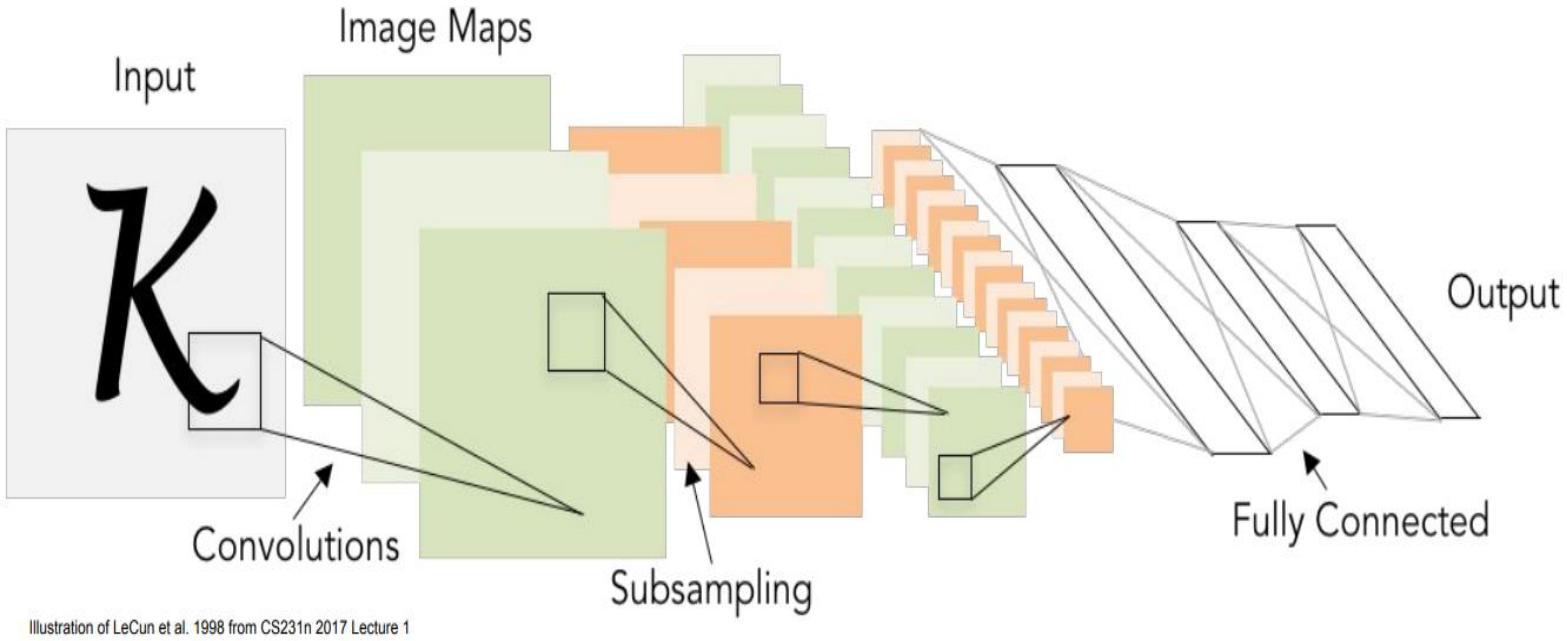
- They may or may not contain *fully connected layers*.
- They may include pooling layers to reduce spatial resolution (subsampling).

Main conclusions (3)



- They use the image itself (pixels directly) as input.
- Linear (convolution) and non-linear (activation function) operations are combined.
- Progressively more complex representations are learned.

Main conclusions (and 4)



- Training is *end-to-end,* minimizing one function (*loss*).
- When learning is supervised, it usually requires lots of data.

Some useful references

- Montavon, Grégoire, Geneviève Orr, and Klaus-Robert Müller, eds. *Neural networks: tricks of the trade*. Vol. 7700. Springer, 2012.
  - Bengio, Yoshua. “Practical recommendations for gradient-based training of deep architectures”, pages 437-478.
  - Bottou, Léon. “Stochastic gradient descent tricks”, 421-436.
  - LeCun, Yann, et al. “Efficient backprop”, pages 9-50.
  - Neuneier, Ralph, and Zimmermann, Hans Georg. “How to train neural networks”, pages 369-418.
- Deep Learning Tips and Tricks cheatsheet: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks#

Appendix

The following slides contain supplementary (but hopefully useful and interesting) material for students to review.

On the importance of ablation studies in Deep Learning research

On the importance of ablation studies in deep learning research

Deep learning architectures are often more *evolved* than designed—they were developed by repeatedly trying things and selecting what seemed to work. Much like in biological systems, if you take any complicated experimental deep learning setup, chances are you can remove a few modules (or replace some trained features with random ones) with no loss of performance.

This is made worse by the incentives that deep learning researchers face: by making a system more complex than necessary, they can make it appear more interesting or more novel, and thus increase their chances of getting a paper through the peer-review process. If you read lots of deep learning papers, you will notice that they're often optimized for peer review in both style and content in ways that actively hurt clarity of explanation and reliability of results. For instance, mathematics in deep learning papers is rarely used for clearly formalizing concepts or deriving non-obvious results—rather, it gets leveraged as a *signal of seriousness*, like an expensive suit on a salesman.

The goal of research shouldn't be merely to publish, but to generate reliable knowledge. Crucially, understanding *causality* in your system is the most straightforward way to generate reliable knowledge. And there's a very low-effort way to look into causality: *ablation studies*. Ablation studies consist of systematically trying to remove parts of a system—making it simpler—to identify where its performance actually comes from. If you find that X + Y + Z gives you good results, also try X, Y, Z, X + Y, X + Z, and Y + Z, and see what happens.

If you become a deep learning researcher, cut through the noise in the research process: do ablation studies for your models. Always ask, "Could there be a simpler explanation? Is this added complexity really necessary? Why?"

Francois Chollet. https://x.com/fchollet/status/1956443383662260536

Progress in Deep Learning (and science in general)



This paper says “X”



But this paper proposes “Y”
(and even “no X”)



And this says that “Z” is the correct thing to do (and suggests that “X” and “Y” are both wrong)



In science, apparently contradictory findings are common, especially in rapidly evolving research fields such as deep learning (DL).

Scientific progress often involves examining and reconciling conflicting findings.

What to do? My advice:

- Prioritize conclusions supported by rigorous experimental evidence, preferably corroborated by independent studies.
  - Scientific conclusions are provisional and may be revised as new evidence emerges.
- The devil is in the details.
  - Regarding DL: some recommendations may be for a particular type of network, or may only be valid in combination with other elements (such as the problem or a set of layers). Pay much attention to details.
- Consider the reputation of authors and publication venues, but prioritize methodological rigor and reproducibility over prestige or citation counts.

Furthermore, in general, and Deep Learning is not an exception, theory often lags behind practice.

"In global terms, mathematics is very far behind computer science [in making fast progress in deep learning]. That's a very classic situation in science. If you look the history of physics, from the first experimental discoveries to the theory, you typically have several centuries. Here, we are speaking of 10, 20, 30 years. This is not only normal but it's great for maths. Because that means you have new problems, and I think what is the beauty of these problems is that the range of mathematics is very diverse (Fourier transform, convolution, harmonic analysis, algebra, differential equations, high-dimensional statistics, geometry,...). It's going to take some time, but I'm impressed by how fast the field is moving, including the math's side. Mathematics has always been moving slow, but it's moving... there are a lot of ideas coming in."

(Stéphane Mallat, DaSCI seminar, 16/05/2023)



Regarding batch sizes, learning rates, and the interaction between hyperparameters

- Intuitively, larger batch size should increase “effectiveness”
  - Because gradient update takes into account a significant portion of the dataset.
- Smaller batch size → train with gradients estimated from a smaller portion of the dataset.
  - A smaller “chunk” of the dataset will be less representative of the overall relationship between the features and the labels.

- However, **some studies have reported poorer generalization with larger batch sizes** (although this effect depends on the training configuration).

- Wilson and Martinez. "The general inefficiency of batch training for gradient descent learning." *Neural networks* 16.10 (2003): 1429-1451.

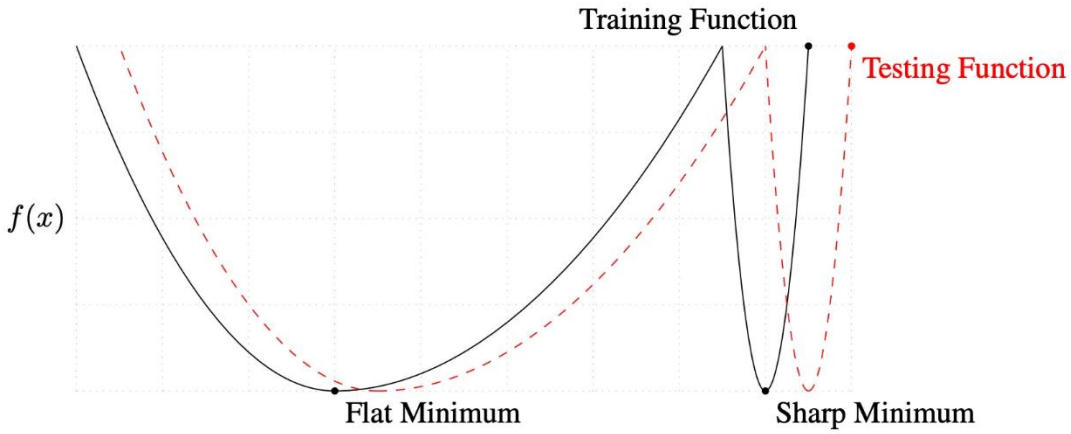
Learning rate	Batch size	Max word accuracy (%)	Training epochs
0.1	1	96.49	21
0.1	10	96.13	41
0.1	100	95.39	43
0.1	1000	84.13 +	4747 +
0.01	1	96.49	27
0.01	10	96.49	27
0.01	100	95.76	46
0.01	1000	95.20	1612
0.01	20,000	23.25 +	4865 +
0.001	1	96.49	402
0.001	100	96.68	468
0.001	1000	96.13	405
0.001	20,000	90.77	1966
0.0001	1	96.68	4589
0.0001	100	96.49	5340
0.0001	1000	96.49	5520
0.0001	20,000	96.31	8343

In these experiments, smaller batches often achieved comparable or better accuracy with fewer training epochs.

- Keskar et al. "On large-batch training for deep learning: Generalization gap and sharp minima." *ICLR 2017*.

– In their experiments, **large-batch training was associated with poorer generalization than small-batch training.**

– Large batches tend to zoom in on the closest relative minima that it finds, whereas networks trained with a **smaller batch size tend to "explore" the loss landscape** before settling on a promising minimum.



Small batch size adds noise to training → This noise seems to be beneficial to avoid sharp minima.

- Hoffer et al. "Train longer, generalize better: closing the generalization gap in large batch training of neural networks." *NeurIPS* 2017.
  - The larger the batch size, the smaller the number of updates → With fewer parameter updates, the model may require additional training epochs to achieve comparable optimization and generalization.
  - *"The "generalization gap" stems from the relatively small number of updates rather than the batch size"*.
  - Hoffer et al. (2017) argue that **the generalization gap observed with large batches is not necessarily caused by batch size itself**. They show that, in their experiments, **it can be reduced by training for more epochs** (thus increasing the number of parameter updates), **appropriately adjusting the learning rate, and using Ghost Batch Normalization**.

In any case, keep in mind that larger batches can train faster,
but require more memory.

- Smith, Samuel L. et al. "Don't decay the learning rate, increase the batch size." *ICLR 2018*.
  - Some relevant observations:
    - Small learning rates and large batch sizes can reduce the effective stochasticity of SGD.
    - This may weaken implicit regularization and, in some settings, increase the risk of overfitting.
    - However, these effects depend on the interaction between hyperparameters and the training procedure.
    - It's common to decay the learning rate → first you explore, then you exploit
  - In this paper, **instead of reducing the learning rate by a factor of k, the authors increase the batch size by the same factor k, keeping the learning rate constant**.

"It reaches equivalent test accuracies after the **same number of training epochs**, but with **fewer parameter updates**, leading to greater parallelism and **shorter training times**. "

- Smith, Leslie N. "A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay." *US Naval Research Laboratory Technical Report 5510-026* (2018).
  - Small batch sizes can provide implicit regularization.
  - The 1cycle policy increases the learning rate and then progressively decreases it to a very small value.
  - Goal: “obtaining the highest performance while minimizing the needed computational time.”
  - Faster training:
    - Larger batch sizes (computational efficiency) + larger learning rates (faster optimization and potential regularization)



Yann LeCun ∞

@ylecun

Training with large minibatches is bad for your health.
More importantly, it's bad for your test error.
Friends dont let friends use minibatches larger than 32.

Traducir post

arxiv.org

Revisiting Small Batch Training for Deep Neural Networks

Modern deep neural network training is typically based on mini-batch stochastic gradient optimization. While the us...

11:00 p. m. · 26 abr. 2018

https://arxiv.org/abs/1804.07612

"The best performance has been consistently obtained for mini-batch sizes between m=2 and m=32, which contrasts with recent work advocating the use of mini-batch sizes in the thousands."

And… what about, for instance, the percentage of dropout?

- Either **you use some reference value from the literature**,
- Or **you start from some values recommended in the literature** and perform **some experiments to select the suitable value for your problem**.
  - Park & Kwak. Analysis on the dropout effect in convolutional neural networks. *ACCV'16*
    - Best results documented with $p < 0.5$
  - Original paper: Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research.*
    - "0.5 […] seems to be close to optimal for a wide range of networks and tasks."

Introduction to Deep Learning and Convolutional Neural Networks

Pablo Mesejo
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA

